

Objektinis-prog-3

3.0

Sugeneruota Doxygen 1.17.0

1 Hierarchijos Indeksas	1
1.1 Klasių hierarchija	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Klasės Dokumentacija	7
4.1 Studentas Klasė	7
4.1.1 Smulkus aprašymas	9
4.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	10
4.1.2.1 Studentas() [1/4]	10
4.1.2.2 Studentas() [2/4]	10
4.1.2.3 Studentas() [3/4]	10
4.1.2.4 Studentas() [4/4]	11
4.1.2.5 ~Studentas()	11
4.1.3 Metodų Dokumentacija	11
4.1.3.1 addPazymys()	11
4.1.3.2 clearPazymiai()	12
4.1.3.3 getEgz()	12
4.1.3.4 getPaz()	12
4.1.3.5 getPazSkaicius()	13
4.1.3.6 getRez()	13
4.1.3.7 islaike()	13
4.1.3.8 mediana()	14
4.1.3.9 operator!=()	14
4.1.3.10 operator+=()	14
4.1.3.11 operator<()	15
4.1.3.12 operator<=()	15
4.1.3.13 operator=() [1/2]	16
4.1.3.14 operator=() [2/2]	16
4.1.3.15 operator==()	17
4.1.3.16 operator>()	17
4.1.3.17 operator>=()	18
4.1.3.18 operator[]() [1/2]	18
4.1.3.19 operator[]() [2/2]	18
4.1.3.20 print()	18
4.1.3.21 setEgz()	19
4.1.3.22 setRez()	19
4.1.3.23 skaiciuotiRez()	19
4.1.3.24 vidurkis()	20

4.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	20
4.1.4.1 operator<<	20
4.1.4.2 operator>>	21
4.1.5 Atributų Dokumentacija	22
4.1.5.1 egz_	22
4.1.5.2 paz_	22
4.1.5.3 rez_	23
4.2 TestRow Struktūra	23
4.2.1 Smulkus aprašymas	23
4.2.2 Atributų Dokumentacija	23
4.2.2.1 bendras	23
4.2.2.2 failas	23
4.2.2.3 konteineris	24
4.2.2.4 rusiavimas	24
4.2.2.5 skaitymas	24
4.2.2.6 skirstymas	24
4.2.2.7 strategija	24
4.2.2.8 studentu_sk	24
4.3 Vector< T, Allocator > Klasė Šablonas	24
4.3.1 Smulkus aprašymas	28
4.3.2 Tipo Aprašymo Dokumentacija	29
4.3.2.1 allocator_type	29
4.3.2.2 AllocTraits	29
4.3.2.3 const_iterator	29
4.3.2.4 const_pointer	29
4.3.2.5 const_reference	29
4.3.2.6 const_reverse_iterator	30
4.3.2.7 difference_type	30
4.3.2.8 iterator	30
4.3.2.9 pointer	30
4.3.2.10 reference	30
4.3.2.11 reverse_iterator	30
4.3.2.12 size_type	30
4.3.2.13 value_type	31
4.3.3 Konstruktorius ir Destruktorius Dokumentacija	31
4.3.3.1 Vector() [1/7]	31
4.3.3.2 Vector() [2/7]	31
4.3.3.3 Vector() [3/7]	31
4.3.3.4 Vector() [4/7]	32
4.3.3.5 Vector() [5/7]	32
4.3.3.6 Vector() [6/7]	33
4.3.3.7 Vector() [7/7]	33

4.3.3.8 ~Vector()	33
4.3.4 Metodų Dokumentacija	34
4.3.4.1 assign() [1/3]	34
4.3.4.2 assign() [2/3]	34
4.3.4.3 assign() [3/3]	34
4.3.4.4 at() [1/2]	35
4.3.4.5 at() [2/2]	35
4.3.4.6 back() [1/2]	35
4.3.4.7 back() [2/2]	35
4.3.4.8 begin() [1/2]	36
4.3.4.9 begin() [2/2]	36
4.3.4.10 capacity()	36
4.3.4.11 cbegin()	36
4.3.4.12 cend()	36
4.3.4.13 clear()	37
4.3.4.14 crbegin()	37
4.3.4.15 crend()	37
4.3.4.16 data() [1/2]	37
4.3.4.17 data() [2/2]	37
4.3.4.18 deallocate_all()	38
4.3.4.19 emplace()	38
4.3.4.20 emplace_back()	38
4.3.4.21 empty()	39
4.3.4.22 end() [1/2]	39
4.3.4.23 end() [2/2]	39
4.3.4.24 erase() [1/2]	39
4.3.4.25 erase() [2/2]	40
4.3.4.26 front() [1/2]	40
4.3.4.27 front() [2/2]	40
4.3.4.28 get_allocator()	40
4.3.4.29 insert() [1/5]	40
4.3.4.30 insert() [2/5]	41
4.3.4.31 insert() [3/5]	41
4.3.4.32 insert() [4/5]	42
4.3.4.33 insert() [5/5]	42
4.3.4.34 max_size()	42
4.3.4.35 operator=() [1/3]	43
4.3.4.36 operator=() [2/3]	43
4.3.4.37 operator=() [3/3]	43
4.3.4.38 operator[]() [1/2]	43
4.3.4.39 operator[]() [2/2]	44
4.3.4.40 pop_back()	44

4.3.4.41 push_back() [1/2]	44
4.3.4.42 push_back() [2/2]	44
4.3.4.43 rbegin() [1/2]	45
4.3.4.44 rbegin() [2/2]	45
4.3.4.45 reallocate()	45
4.3.4.46 rend() [1/2]	45
4.3.4.47 rend() [2/2]	46
4.3.4.48 reserve()	46
4.3.4.49 resize()	46
4.3.4.50 shrink_to_fit()	47
4.3.4.51 size()	47
4.3.4.52 swap()	47
4.3.5 Atributų Dokumentacija	47
4.3.5.1 alloc_	47
4.3.5.2 capacity_	48
4.3.5.3 data_	48
4.3.5.4 size_	48
4.4 Zmogus Klasė	48
4.4.1 Smulkus aprašymas	49
4.4.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	50
4.4.2.1 Zmogus() [1/4]	50
4.4.2.2 Zmogus() [2/4]	50
4.4.2.3 Zmogus() [3/4]	50
4.4.2.4 Zmogus() [4/4]	51
4.4.2.5 ~Zmogus()	51
4.4.3 Metodų Dokumentacija	51
4.4.3.1 getPavarde()	51
4.4.3.2 getVardas()	51
4.4.3.3 operator=() [1/2]	52
4.4.3.4 operator=() [2/2]	52
4.4.3.5 print()	52
4.4.3.6 setPavarde()	53
4.4.3.7 setVardas()	53
4.4.4 Atributų Dokumentacija	53
4.4.4.1 pavarde_	53
4.4.4.2 vardas_	53
5 Failo Dokumentacija	55
5.1 deque_ops.cpp Failo Nuoroda	55
5.1.1 Funkcijos Dokumentacija	55
5.1.1.1 isvedimas_i_faila_d()	55
5.1.1.2 pasirinkimo_metodas_d()	56

5.1.1.3 rusiavimas_d()	56
5.1.1.4 skaitymas_is_failo_d()	56
5.1.1.5 skirstymas_i_grupes_d()	57
5.1.1.6 skirstymas_s1_d()	57
5.1.1.7 skirstymas_s2_d()	57
5.1.1.8 skirstymas_s3_d()	58
5.2 deque_ops.cpp	58
5.3 deque_ops.h Failo Nuoroda	59
5.3.1 Funkcijos Dokumentacija	60
5.3.1.1 isvedimas_i_faila_d()	60
5.3.1.2 pasirinkimo_metodas_d()	60
5.3.1.3 rusiavimas_d()	60
5.3.1.4 skaitymas_is_failo_d()	61
5.3.1.5 skirstymas_i_grupes_d()	61
5.3.1.6 skirstymas_s1_d()	61
5.3.1.7 skirstymas_s2_d()	62
5.3.1.8 skirstymas_s3_d()	62
5.4 deque_ops.h	62
5.5 generator.cpp Failo Nuoroda	63
5.5.1 Funkcijos Dokumentacija	63
5.5.1.1 generuoti_faila()	63
5.5.1.2 generuoti_pazymi()	64
5.5.1.3 generuoti_testu_failus()	64
5.6 generator.cpp	64
5.7 list_ops.cpp Failo Nuoroda	65
5.7.1 Funkcijos Dokumentacija	65
5.7.1.1 isvedimas_i_faila_l()	65
5.7.1.2 pasirinkimo_metodas_l()	66
5.7.1.3 rusiavimas_l()	66
5.7.1.4 skaitymas_is_failo_l()	66
5.7.1.5 skirstymas_i_grupes_l()	67
5.7.1.6 skirstymas_s1_l()	67
5.7.1.7 skirstymas_s2_l()	67
5.7.1.8 skirstymas_s3_l()	68
5.8 list_ops.cpp	68
5.9 list_ops.h Failo Nuoroda	69
5.9.1 Funkcijos Dokumentacija	70
5.9.1.1 isvedimas_i_faila_l()	70
5.9.1.2 pasirinkimo_metodas_l()	70
5.9.1.3 rusiavimas_l()	70
5.9.1.4 skaitymas_is_failo_l()	71
5.9.1.5 skirstymas_i_grupes_l()	71

5.9.1.6 skirstymas_s1_l()	71
5.9.1.7 skirstymas_s2_l()	72
5.9.1.8 skirstymas_s3_l()	72
5.10 list_ops.h	72
5.11 main.cpp Failo Nuoroda	73
5.11.1 Funkcijos Dokumentacija	73
5.11.1.1 atlikti_klases_testus()	73
5.11.1.2 atlikti_visus_testus()	74
5.11.1.3 generuoti_faila()	74
5.11.1.4 generuoti_testu_failus()	75
5.11.1.5 main()	75
5.11.1.6 rankinis_ivedimas()	76
5.11.1.7 skaityti_deque()	76
5.11.1.8 skaityti_list()	77
5.11.1.9 skaityti_vector()	77
5.12 main.cpp	78
5.13 studentas.cpp Failo Nuoroda	81
5.13.1 Funkcijos Dokumentacija	81
5.13.1.1 operator<<()	81
5.13.1.2 operator>>()	82
5.14 studentas.cpp	83
5.15 studentas.h Failo Nuoroda	85
5.16 studentas.h	85
5.17 testavimas.cpp Failo Nuoroda	88
5.17.1 Funkcijos Dokumentacija	89
5.17.1.1 atlikti_visus_testus()	89
5.17.1.2 spausdinti_lentele()	89
5.17.1.3 testuoti_deque()	90
5.17.1.4 testuoti_list()	90
5.17.1.5 testuoti_vector()	91
5.18 testavimas.cpp	91
5.19 testavimas_klases.cpp Failo Nuoroda	93
5.19.1 Funkcijos Dokumentacija	94
5.19.1.1 atlikti_klases_testus()	94
5.19.1.2 sekcija()	95
5.19.1.3 testuoti_konstruktorius()	95
5.19.1.4 testuoti_metodus()	96
5.19.1.5 testuoti_palyginimo_operatorius()	96
5.19.1.6 testuoti_priskyrima()	97
5.19.1.7 testuoti_srautu_operatorius()	97
5.19.1.8 testuoti_su_stl()	98
5.19.1.9 testuoti_zmogus_abstrakti()	98

5.19.1.10 tikrinti()	99
5.19.2 Kintamojo Dokumentacija	99
5.19.2.1 testai_is_viso	99
5.19.2.2 testai_pavyke	99
5.20 testavimas_klases.cpp	100
5.21 unit_tests.cpp Failo Nuoroda	104
5.21.1 Funkcijos Dokumentacija	104
5.21.1.1 main()	104
5.21.1.2 TEST() [1/13]	105
5.21.1.3 TEST() [2/13]	105
5.21.1.4 TEST() [3/13]	105
5.21.1.5 TEST() [4/13]	105
5.21.1.6 TEST() [5/13]	106
5.21.1.7 TEST() [6/13]	106
5.21.1.8 TEST() [7/13]	106
5.21.1.9 TEST() [8/13]	106
5.21.1.10 TEST() [9/13]	107
5.21.1.11 TEST() [10/13]	107
5.21.1.12 TEST() [11/13]	107
5.21.1.13 TEST() [12/13]	107
5.21.1.14 TEST() [13/13]	108
5.22 unit_tests.cpp	108
5.23 utils.cpp Failo Nuoroda	109
5.23.1 Funkcijos Dokumentacija	110
5.23.1.1 galutinis_vid()	110
5.23.1.2 iversti_varda_ar_pavarde()	110
5.23.1.3 iversties_skaicius()	110
5.23.1.4 iversties_tikrinimas()	111
5.23.1.5 skaiciavimo_metodas()	111
5.24 utils.cpp	111
5.25 utils.h Failo Nuoroda	113
5.25.1 Funkcijos Dokumentacija	113
5.25.1.1 iversti_varda_ar_pavarde()	113
5.25.1.2 iversties_skaicius()	113
5.25.1.3 iversties_tikrinimas()	114
5.25.1.4 skaiciavimo_metodas()	114
5.26 utils.h	114
5.27 vector.cpp Failo Nuoroda	115
5.28 vector.cpp	115
5.29 vector.h Failo Nuoroda	115
5.29.1 Smulkus aprašymas	116
5.29.2 Funkcijos Dokumentacija	116

5.29.2.1 erase()	116
5.29.2.2 erase_if()	116
5.29.2.3 operator!=()	117
5.29.2.4 operator<()	117
5.29.2.5 operator<=()	117
5.29.2.6 operator==()	117
5.29.2.7 operator>()	118
5.29.2.8 operator>=()	118
5.29.2.9 swap()	118
5.30 vector.h	118
5.31 vector_ops.cpp Failo Nuoroda	126
5.31.1 Smulkus aprašymas	127
5.31.2 Funkcijos Dokumentacija	127
5.31.2.1 isvedimas()	127
5.31.2.2 isvedimas_i_faila()	127
5.31.2.3 pasirinkimas_rusiavimo_budo()	128
5.31.2.4 pasirinkimo_metodas()	128
5.31.2.5 rusiavimas()	129
5.31.2.6 skaitymas_is_failo()	129
5.31.2.7 skirstymas_i_grupes()	130
5.31.2.8 skirstymas_s1()	130
5.31.2.9 skirstymas_s2()	130
5.31.2.10 skirstymas_s3()	131
5.32 vector_ops.cpp	131
5.33 vector_ops.h Failo Nuoroda	133
5.33.1 Smulkus aprašymas	134
5.33.2 Funkcijos Dokumentacija	134
5.33.2.1 isvedimas()	134
5.33.2.2 isvedimas_i_faila()	135
5.33.2.3 pasirinkimas_rusiavimo_budo()	136
5.33.2.4 pasirinkimo_metodas()	136
5.33.2.5 rusiavimas()	137
5.33.2.6 skaitymas_is_failo()	137
5.33.2.7 skirstymas_i_grupes()	138
5.33.2.8 skirstymas_s1()	138
5.33.2.9 skirstymas_s2()	139
5.33.2.10 skirstymas_s3()	139
5.34 vector_ops.h	140
5.35 vector_unit_tests.cpp Failo Nuoroda	141
5.35.1 Smulkus aprašymas	142
5.35.2 Funkcijos Dokumentacija	142
5.35.2.1 TEST() [1/61]	142

5.35.2.2 TEST() [2/61]	143
5.35.2.3 TEST() [3/61]	143
5.35.2.4 TEST() [4/61]	143
5.35.2.5 TEST() [5/61]	143
5.35.2.6 TEST() [6/61]	144
5.35.2.7 TEST() [7/61]	144
5.35.2.8 TEST() [8/61]	144
5.35.2.9 TEST() [9/61]	144
5.35.2.10 TEST() [10/61]	145
5.35.2.11 TEST() [11/61]	145
5.35.2.12 TEST() [12/61]	145
5.35.2.13 TEST() [13/61]	145
5.35.2.14 TEST() [14/61]	146
5.35.2.15 TEST() [15/61]	146
5.35.2.16 TEST() [16/61]	146
5.35.2.17 TEST() [17/61]	146
5.35.2.18 TEST() [18/61]	147
5.35.2.19 TEST() [19/61]	147
5.35.2.20 TEST() [20/61]	147
5.35.2.21 TEST() [21/61]	147
5.35.2.22 TEST() [22/61]	148
5.35.2.23 TEST() [23/61]	148
5.35.2.24 TEST() [24/61]	148
5.35.2.25 TEST() [25/61]	148
5.35.2.26 TEST() [26/61]	149
5.35.2.27 TEST() [27/61]	149
5.35.2.28 TEST() [28/61]	149
5.35.2.29 TEST() [29/61]	149
5.35.2.30 TEST() [30/61]	150
5.35.2.31 TEST() [31/61]	150
5.35.2.32 TEST() [32/61]	150
5.35.2.33 TEST() [33/61]	150
5.35.2.34 TEST() [34/61]	151
5.35.2.35 TEST() [35/61]	151
5.35.2.36 TEST() [36/61]	151
5.35.2.37 TEST() [37/61]	151
5.35.2.38 TEST() [38/61]	152
5.35.2.39 TEST() [39/61]	152
5.35.2.40 TEST() [40/61]	152
5.35.2.41 TEST() [41/61]	152
5.35.2.42 TEST() [42/61]	153
5.35.2.43 TEST() [43/61]	153

5.35.2.44 TEST() [44/61]	153
5.35.2.45 TEST() [45/61]	153
5.35.2.46 TEST() [46/61]	154
5.35.2.47 TEST() [47/61]	154
5.35.2.48 TEST() [48/61]	154
5.35.2.49 TEST() [49/61]	154
5.35.2.50 TEST() [50/61]	155
5.35.2.51 TEST() [51/61]	155
5.35.2.52 TEST() [52/61]	155
5.35.2.53 TEST() [53/61]	155
5.35.2.54 TEST() [54/61]	156
5.35.2.55 TEST() [55/61]	156
5.35.2.56 TEST() [56/61]	156
5.35.2.57 TEST() [57/61]	156
5.35.2.58 TEST() [58/61]	157
5.35.2.59 TEST() [59/61]	157
5.35.2.60 TEST() [60/61]	157
5.35.2.61 TEST() [61/61]	158
5.36 vector_unit_tests.cpp	158
5.37 zmogus.cpp Failo Nuoroda	165
5.38 zmogus.cpp	165
5.39 zmogus.h Failo Nuoroda	165
5.39.1 Apibrėžimų Dokumentacija	166
5.39.1.1 ZMOGUS_H	166
5.39.2 Funkcijos Dokumentacija	166
5.39.2.1 operator<<()	166
5.40 zmogus.h	167
6 Pavyzdžiai	169
6.1 C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h	169
Rodyklė	173

skyrius 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

TestRow	23
Vector< T, Allocator >	24
Zmogus	48
Studentas	7

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	
Atstovauja studenta su pažymiais ir egzamino rezultatu	7
TestRow	23
Vector< T, Allocator >	
Nuosava Vector konteinerio klasė	24
Zmogus	
Abstrakti bazinė klase, atstovaujanti žmogui	48

skyrius 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

deque_ops.cpp	55
deque_ops.h	59
generator.cpp	63
list_ops.cpp	65
list_ops.h	69
main.cpp	73
studentas.cpp	81
studentas.h	85
testavimas.cpp	88
testavimas_klases.cpp	93
unit_tests.cpp	104
utils.cpp	109
utils.h	113
vector.cpp	115
vector.h	
Nuosavos Vector konteinerio klasės aprašas	115
vector_ops.cpp	
Vektoriaus operacijų realizacija naudojant nuosavą Vector konteinerį	126
vector_ops.h	
Vektoriaus operacijos naudojant nuosavą Vector konteinerį	133
vector_unit_tests.cpp	
Google Test testai nuosavai Vector klasei	141
zmogus.cpp	165
zmogus.h	165

skyrius 4

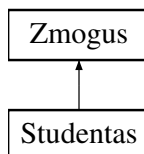
Klasės Dokumentacija

4.1 Studentas Klasė

Atstovauja studenta su pažymiais ir egzamino rezultatu.

```
#include <studentas.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas ()`
Numatytasis konstruktorius.
- `Studentas (const std::string &vardas, const std::string &pavarde, const Vector< int > &paz, int egz)`
- `Studentas (const Studentas &other)`
Kopijavimo konstruktorius (gilioji kopija).
- `Studentas (Studentas &&other) noexcept`
Perkelimo konstruktorius.
- `~Studentas ()`
Virtualus destruktorius.
- `Studentas & operator= (const Studentas &other)`
Kopijavimo priskyrimo operatorius.
- `Studentas & operator= (Studentas &&other) noexcept`
Perkelimo priskyrimo operatorius.
- `bool operator< (const Studentas &other) const`
Maziau uz (pagal galutini rezultata).
- `bool operator> (const Studentas &other) const`
Daugiau uz (pagal galutini rezultata).
- `bool operator== (const Studentas &other) const`
Lygybe (pagal varda ir pavarde).

- bool `operator<=` (const `Studentas` &other) const
Maziau arba lygu.
- bool `operator>=` (const `Studentas` &other) const
Daugiau arba lygu.
- bool `operator!=` (const `Studentas` &other) const
Nelygybe.
- `Studentas` & `operator+=` (int pazymys)
- int `operator[]` (size_t i) const
- int & `operator[]` (size_t i)
- void `print` (std::ostream &os) const override
Spausdina studento informacija (pakeicia `Zmogus::print`).
- const std::vector< int > & `getPaz` () const
- int `getEgz` () const
- double `getRez` () const
- size_t `getPazSkaicius` () const
- void `setEgz` (int e)
- void `setRez` (double r)
- void `addPazymys` (int p)
Prideda viena namu darbu pazymi.
- void `clearPazymiai` ()
Isvalo visus namu darbu pazymius.
- double `vidurkis` () const
Skaiciuoja namu darbu pazymiu vidurki.
- double `mediana` () const
Skaiciuoja namu darbu pazymiu mediana.
- void `skaiciuotiRez` (int tipas)
Skaiciuoja galutini rezultata.
- bool `islaike` () const
Tikrina ar studentas islaike.

Vieši Metodai inherited from `Zmogus`

- `Zmogus` ()
Numatytasis konstruktorius Sukuria tuscia zmogu su tusciais vardu ir pavarde.
- `Zmogus` (const std::string &vardas, const std::string &pavarde)
Parametrinis konstruktorius.
- `Zmogus` (const `Zmogus` &other)
Kopijavimo konstruktorius.
- `Zmogus` (`Zmogus` &&other) noexcept
Perkelimo konstruktorius.
- virtual `~Zmogus` ()=0
Virtualus destruktorius.
- `Zmogus` & `operator=` (const `Zmogus` &other)
Kopijavimo priskyrimo operatorius.
- `Zmogus` & `operator=` (`Zmogus` &&other) noexcept
Perkelimo priskyrimo operatorius.
- const std::string & `getVardas` () const
- const std::string & `getPavarde` () const
- void `setVardas` (const std::string &v)
- void `setPavarde` (const std::string &p)

Privatūs Atributai

- `std::vector< int > paz_`
- `int egz_`
- `double rez_`

Draugai

- `std::ostream & operator<< (std::ostream &os, const Studentas &s)`
Isvedimo i ekrana operatorius.
- `std::istream & operator>> (std::istream &is, Studentas &s)`
Ivedimo is failo operatorius.

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- `std::string vardas_`
- `std::string pavarde_`

4.1.1 Smulkus aprašymas

Atstovauja studenta su pazymiais ir egzamino rezultatu.

Si klase praplecia Zmogus klase ir igyvendina studentams budinga funkcionaluma, iskaitant pazymių valdyma, rezultatu skaiciavima (vidurkis/mediana) ir klasifikavima i grupes.

Autorius

Studentu Valdymo Sistema

Versija

2.0

Data

2026

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 24.

4.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.1.2.1 Studentas() [1/4]

```
Studentas::Studentas ()
```

Numatytasis konstruktorius.

Sukuria tuscia studenta su numatytosiomis reikšmemis. Visi pazymiai isvalomi, egzaminas = 0, rezultatas = 0.0

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 10.

```
00011 : Zmogus(), paz_(), egz_(0), rez_(0.0) {
00012 }
```

4.1.2.2 Studentas() [2/4]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    const Vector< int > & paz,
    int egz)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 15.

```
00019 : Zmogus(vardas, pavarde), paz_(paz.begin(), paz.end()), egz_(egz), rez_(0.0) {
00020 }
```

4.1.2.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & other)
```

Kopijavimo konstruktorius (gilioji kopija).

Parametrai

<i>other</i>	Studentas objektas, is kurio kopijuojama @kompleksiskumas O(n), kur n yra pazymių skaičius
--------------	--

Apibrėžimas failo [studentas.cpp](#) eilutėje 23.

```
00024 : Zmogus(other), // kopijuoja vardas_, pavarde_
00025 paz_(other.paz_),
00026 egz_(other.egz_),
00027 rez_(other.rez_) {
00028 }
```

4.1.2.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && other) [noexcept]
```

Perkelimo konstruktorius.

Parametrai

<i>other</i>	Studentas objektas, is kurio perkeliama @kompleksiskumas O(1)
--------------	---

Apibrėžimas failo [studentas.cpp](#) eilutėje 31.

```
00032 : Zmogus(std::move(other)), // perkelia vardas_, pavarde_
00033 paz_(std::move(other.paz_)),
00034 egz_(other.egz_),
00035 rez_(other.rez_) {
00036     other.egz_ = 0;
00037     other.rez_ = 0.0;
00038 }
```

4.1.2.5 ~Studentas()

```
Studentas::~~Studentas ()
```

Virtualus destruktorius.

Uztikrina tinkama isvestiniu klasiu isvalyma

Pavyzdžiai

[C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.](#)

Apibrėžimas failo [studentas.cpp](#) eilutėje 42.

```
00042 {
00043     paz_.clear();
00044 }
```

4.1.3 Metodų Dokumentacija

4.1.3.1 addPazymys()

```
void Studentas::addPazymys (
    int p)
```

Prideda viena namu darbu pazymi.

Parametrai

<i>p</i>	Pazymys (1-10)
----------	----------------

Pavyzdžiai

[C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.](#)

Apibrėžimas failo [studentas.cpp](#) eilutėje 135.

```
00135 {
00136     paz_.push_back(p);
00137 }
```

4.1.3.2 clearPazymiai()

```
void Studentas::clearPazymiai ()
```

Isvalo visus namu darbu pazymius.

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 139.

```
00139                                     {  
00140     paz_.clear();  
00141 }
```

4.1.3.3 getEgz()

```
int Studentas::getEgz () const [inline]
```

Gražina

Egzamino pazymys

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 189.

```
00189 { return egz_; }
```

4.1.3.4 getPaz()

```
const std::vector< int > & Studentas::getPaz () const [inline]
```

Gražina

Pastovi nuoroda į pazymių vektorių

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 186.

```
00186 { return paz_; }
```


4.1.3.5 getPazSkaicius()

```
size_t Studentas::getPazSkaicius () const [inline]
```

Gražina

Namu darbu pazymiu skaicius

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 195.

```
00195 { return paz_.size(); }
```

4.1.3.6 getRez()

```
double Studentas::getRez () const [inline]
```

Gražina

Galutinis rezultatas

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 192.

```
00192 { return rez_; }
```

4.1.3.7 islaike()

```
bool Studentas::islaike () const [inline]
```

Tikrina ar studentas islaike.

Gražina

true jei rez_ >= 5.0

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 243.

```
00243 { return rez_ >= 5.0; }
```

4.1.3.8 mediana()

```
double Studentas::mediana () const
```

Skaiciuoja namu darbu pazymiu mediana.

Gražina

Mediana kaip double

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 148.

```
00148 {
00149     if (paz_.empty()) return 0.0;
00150     std::vector<int> tmp = paz_;
00151     std::sort(tmp.begin(), tmp.end());
00152     size_t n = tmp.size();
00153     return (n % 2 == 0) ? (tmp[n / 2 - 1] + tmp[n / 2]) / 2.0 : tmp[n / 2];
00154 }
```

4.1.3.9 operator!=(())

```
bool Studentas::operator!= (
    const Studentas & other) const
```

Nelygybe.

Parametrai

<i>other</i>	Studentas su kuriuo lyginama
--------------	--

Gražina

true jei vardai ar pavardes skiriasi

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 80.

```
00080 {
00081     return !(*this == other);
00082 }
```

4.1.3.10 operator+=(())

```
Studentas & Studentas::operator+= (
    int pazymys)
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

4.1.3.11 operator<()

```
bool Studentas::operator< (
    const Studentas & other) const
```

Maziau uz (pagal galutini rezultata).

Parametrai

<i>other</i>	<code>Studentas</code> su kuriuo lyginama
--------------	---

Gražina

```
true jei si.rez_ < other.rez_
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo `studentas.cpp` eilutėje 71.

```
00071                                     {
00072     return rez_ < other.rez_;
00073 }
```

4.1.3.12 operator<=()

```
bool Studentas::operator<= (
    const Studentas & other) const
```

Maziau arba lygu.

Gražina

```
true jei si.rez_ <= other.rez_
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo `studentas.cpp` eilutėje 83.

```
00083                                     {
00084     return rez_ <= other.rez_;
00085 }
```

4.1.3.13 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Studentas objektas, is kurio kopijuojama
--------------	--

Gražina

Nuoroda i si objekta

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 47.

```
00047                                     {
00048     if (this != &other) {
00049         Zmogus::operator=(other); // kopijuoja vardas_, pavarde_
00050         paz_ = other.paz_;
00051         egz_ = other.egz_;
00052         rez_ = other.rez_;
00053     }
00054     return *this;
00055 }
```

4.1.3.14 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [noexcept]
```

Perkelimo priskyrimo operatorius.

Parametrai

<i>other</i>	Studentas objektas, is kurio perkeliama
--------------	---

Gražina

Nuoroda i si objekta

Apibrėžimas failo [studentas.cpp](#) eilutėje 58.

```
00058                                     {
00059     if (this != &other) {
00060         Zmogus::operator=(std::move(other)); // perkelia vardas_, pavarde_
00061         paz_ = std::move(other.paz_);
00062         egz_ = other.egz_;
00063         rez_ = other.rez_;
00064         other.egz_ = 0;
00065         other.rez_ = 0.0;
00066     }
00067     return *this;
00068 }
```

4.1.3.15 operator==()

```
bool Studentas::operator== (
    const Studentas & other) const
```

Lygybe (pagal vardą ir pavardę).

Parametrai

<i>other</i>	Studentas su kuriuo lyginama
--------------	------------------------------

Gražina

true jei vardai ir pavardės sutampa

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 77.

```
00077 {
00078     return vardas_ == other.vardas_ && pavarde_ == other.pavarde_;
00079 }
```

4.1.3.16 operator>()

```
bool Studentas::operator> (
    const Studentas & other) const
```

Daugiau už (pagal galutini rezultatą).

Parametrai

<i>other</i>	Studentas su kuriuo lyginama
--------------	------------------------------

Gražina

true jei si.rez_ > other.rez_

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 74.

```
00074 {
00075     return rez_ > other.rez_;
00076 }
```

4.1.3.17 operator>=()

```
bool Studentas::operator>= (
    const Studentas & other) const
```

Daugiau arba lygu.

Gražina

```
true jei si.rez_ >= other.rez_
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 86.

```
00086                                     {
00087     return rez_ >= other.rez_;
00088 }
```

4.1.3.18 operator[]() [1/2]

```
int & Studentas::operator[] (
    size_t i)
```

4.1.3.19 operator[]() [2/2]

```
int Studentas::operator[] (
    size_t i) const
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

4.1.3.20 print()

```
void Studentas::print (
    std::ostream & os) const [override], [virtual]
```

Spausdina studento informacija (pakeicia [Zmogus::print](#)).

Parametrai

<i>os</i>	Isvesties srautas
-----------	-------------------

Realizuoja [Zmogus](#).

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 130.

```
00130                                     {
00131     os << *this;
00132 }
```

4.1.3.21 setEgz()

```
void Studentas::setEgz (
    int e) [inline]
```

Parametrai

<i>e</i>	Naujas egzamino pazymys (0-10)
----------	--------------------------------

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 202.

```
00202 { egz_ = e; }
```

4.1.3.22 setRez()

```
void Studentas::setRez (
    double r) [inline]
```

Parametrai

<i>r</i>	Naujas galutinis rezultatas
----------	-----------------------------

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 205.

```
00205 { rez_ = r; }
```

4.1.3.23 skaiciuotiRez()

```
void Studentas::skaiciuotiRez (
    int tipas)
```

Skaiciuoja galutini rezultata.

Parametrai

<i>tipas</i>	1=vidurkis, 2=mediana @detales Formule: $0.4 * pazymys + 0.6 * egzaminas$
--------------	---

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 156.

```
00156 {
00157     double x = (tipas == 1) ? vidurkis() : mediana();
00158     rez_ = x * 0.4 + egz_ * 0.6;
00159 }
```

4.1.3.24 vidurkis()

```
double Studentas::vidurkis () const
```

Skaiciuoja namu darbu pazymiu vidurki.

Gražina

Aritmetinis vidurkis kaip double

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 143.

```
00143 {
00144     if (paz_.empty()) return 0.0;
00145     return std::accumulate(paz_.begin(), paz_.end(), 0.0) / paz_.size();
00146 }
```

4.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

Isvedimo i ekrana operatorius.

Parametrai

os	Isvesties srautas
s	Studentas objektas

Gražina

Nuoroda i isvesties srauta @detales Formatas: "Vardas Pavarde [pazymiai] Egz: E Rez: R"

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 92.

```
00092 {
00093     os << std::left << std::setw(15) << s.vardas_
00094         << std::setw(15) << s.pavarde_
00095         << "[";
00096     for (size_t i = 0; i < s.paz_.size(); ++i) {
00097         os << s.paz_[i];
00098         if (i + 1 < s.paz_.size()) os << " ";
00099     }
00100     os << "]" Egz: " << s.egz_
00101         << std::right << " Rez: "
00102         << std::fixed << std::setprecision(2) << s.rez_;
00103     return os;
00104 }
```


4.1.4.2 operator>>

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s) [friend]
```

Ivedimo is failo operatorius.

Parametrai

<i>is</i>	Ivesties srautas
-----------	------------------

s	Studentas objektas
---	--------------------

Gražina

Nuoroda i ivesties srauta @detales Skaito: "Vardas Pavarde paz1 paz2 ... pazN egzaminas"

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.cpp](#) eilutėje 107.

```

00107                                     {
00108         is » s.vardas_ » s.pavarde_;
00109         std::string line;
00110         std::getline(is, line);
00111         std::istringstream ss(line);
00112         s.paz_.clear();
00113         int v;
00114         while (ss » v) s.paz_.push_back(v);
00115         if (!s.paz_.empty()) {
00116             s.egz_ = s.paz_.back();
00117             s.paz_.pop_back();
00118         }
00119         else {
00120             s.egz_ = 0;
00121         }
00122         s.rez_ = 0.0;
00123         return is;
00124     }
```

4.1.5 Atributų Dokumentacija

4.1.5.1 egz_

```
int Studentas::egz_ [private]
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 27.

4.1.5.2 paz_

```
std::vector<int> Studentas::paz_ [private]
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 26.

4.1.5.3 rez_

```
double Studentas::rez_ [private]
```

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [studentas.h](#) eilutėje 28.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [studentas.h](#)
- [studentas.cpp](#)

4.2 TestRow Struktūra

Vieši Atributai

- std::string [konteineris](#)
- std::string [strategija](#)
- std::string [failas](#)
- int [studentu_sk](#) = 0
- double [skaitymas](#) = 0.0
- double [rusiavimas](#) = 0.0
- double [skirstymas](#) = 0.0
- double [bendras](#) = 0.0

4.2.1 Smulkus aprašymas

Apibrėžimas failo [testavimas.cpp](#) eilutėje 15.

4.2.2 Atributų Dokumentacija

4.2.2.1 bendras

```
double TestRow::bendras = 0.0
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 23.

4.2.2.2 failas

```
std::string TestRow::failas
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 18.

4.2.2.3 konteineris

```
std::string TestRow::konteineris
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 16.

4.2.2.4 rusiavimas

```
double TestRow::rusiavimas = 0.0
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 21.

4.2.2.5 skaitymas

```
double TestRow::skaitymas = 0.0
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 20.

4.2.2.6 skirstymas

```
double TestRow::skirstymas = 0.0
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 22.

4.2.2.7 strategija

```
std::string TestRow::strategija
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 17.

4.2.2.8 studentu_sk

```
int TestRow::studentu_sk = 0
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 19.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [testavimas.cpp](#)

4.3 Vector< T, Allocator > Klasė Šablonas

Nuosava [Vector](#) konteinerio klasė.

```
#include <vector.h>
```

Vieši Tipai

- using `value_type` = T
- using `allocator_type` = Allocator
- using `size_type` = std::size_t
- using `difference_type` = std::ptrdiff_t
- using `reference` = T&
- using `const_reference` = const T&
- using `pointer` = typename std::allocator_traits<Allocator>::pointer
- using `const_pointer` = typename std::allocator_traits<Allocator>::const_pointer
- using `iterator` = T*
- using `const_iterator` = const T*
- using `reverse_iterator` = std::reverse_iterator<iterator>
- using `const_reverse_iterator` = std::reverse_iterator<const_iterator>

Vieši Metodai

- `Vector` () noexcept
Numatytasis konstruktorius. Sukuria tuščią vektorių.
- `Vector` (const Allocator &alloc) noexcept
Konstruktorius su paskirstytoju.
- `Vector` (`size_type` count, const T &value=T(), const Allocator &alloc=Allocator())
Konstruktorius su dydžiu.
- template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<↔
InputIt>::iterator_category>>>>
`Vector` (InputIt first, InputIt last, const Allocator &alloc=Allocator())
Konstruktorius iš iteratorių diapazono.
- `Vector` (const `Vector` &other)
Kopijos konstruktorius.
- `Vector` (`Vector` &&other) noexcept
Perkėlimo konstruktorius.
- `Vector` (std::initializer_list< T > il, const Allocator &alloc=Allocator())
Konstruktorius iš std::initializer_list.
- `~Vector` ()
Destruktorius. Atlaisvina visą atmintį.
- `Vector` & `operator=` (const `Vector` &other)
Kopijos priskyrimo operatorius.
- `Vector` & `operator=` (`Vector` &&other) noexcept
Perkėlimo priskyrimo operatorius.
- `Vector` & `operator=` (std::initializer_list< T > il)
Priskyrimo operatorius iš initializer_list.
- void `assign` (`size_type` count, const T &value)
Priskiria count kopijų reikšmės value.
- template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<↔
InputIt>::iterator_category>>>>
void `assign` (InputIt first, InputIt last)
Priskiria elementus iš iteratorių diapazono.
- void `assign` (std::initializer_list< T > il)
Priskiria elementus iš initializer_list.
- `allocator_type` `get_allocator` () const noexcept
Grąžina paskirstytojo kopiją.
- `reference` `at` (`size_type` pos)

- Prieiga prie elemento su ribų tikrinimu.*
- `const_reference at (size_type pos) const`
- Prieiga prie elemento su ribų tikrinimu (const).*
- `reference operator[] (size_type pos)`
- Prieiga prie elemento pagal indeksą (be tikrinimo).*
- `const_reference operator[] (size_type pos) const`
- Prieiga prie elemento pagal indeksą (be tikrinimo, const).*
- `reference front ()`
- Grąžina pirmą elementą.*
- `const_reference front () const`
- `reference back ()`
- Grąžina paskutinį elementą.*
- `const_reference back () const`
- `T * data () noexcept`
- Grąžina rodyklę į vidinį duomenų masyvą.*
- `const T * data () const noexcept`
- `iterator begin () noexcept`
- Grąžina iteratorių į pradžią.*
- `const_iterator begin () const noexcept`
- `const_iterator cbegin () const noexcept`
- `iterator end () noexcept`
- Grąžina iteratorių į pabaigą (past-the-end).*
- `const_iterator end () const noexcept`
- `const_iterator cend () const noexcept`
- `reverse_iterator rbegin () noexcept`
- Grąžina atvirkštinį iteratorių.*
- `const_reverse_iterator rbegin () const noexcept`
- `const_reverse_iterator crbegin () const noexcept`
- `reverse_iterator rend () noexcept`
- Grąžina atvirkštinį past-the-end iteratorių.*
- `const_reverse_iterator rend () const noexcept`
- `const_reverse_iterator crend () const noexcept`
- `bool empty () const noexcept`
- Tikrina ar vektorius tuščias.*
- `size_type size () const noexcept`
- Grąžina elementų skaičių.*
- `size_type max_size () const noexcept`
- Grąžina maksimalų galimą elementų skaičių.*
- `void reserve (size_type new_cap)`
- Rezervuoja atmintį new_cap elementams.*
- `size_type capacity () const noexcept`
- Grąžina dabartiną capacity.*
- `void shrink_to_fit ()`
- Sumažina capacity iki size (shrink to fit).*
- `void clear () noexcept`
- Ištrina visus elementus (size = 0, capacity nesikeičia).*
- `iterator insert (const_iterator pos, const T &value)`
- Įterpia elementą prieš pos.*
- `iterator insert (const_iterator pos, T &&value)`
- `iterator insert (const_iterator pos, size_type count, const T &value)`
- Įterpia count kopijų reikšmės prieš pos.*

- `template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<↔
InputIt>::iterator_category>>>`
`iterator insert (const_iterator pos, InputIt first, InputIt last)`
Įterpia elementus iš diapazono prieš pos.
- `iterator insert (const_iterator pos, std::initializer_list< T > il)`
- `template<typename... Args>`
`iterator emplace (const_iterator pos, Args &&... args)`
Sukonstruoja elementą vietoje prieš pos.
- `iterator erase (const_iterator pos)`
Ištrina elementą ties pos.
- `iterator erase (const_iterator first, const_iterator last)`
Ištrina elementus diapazone [first, last).
- `void push_back (const T &value)`
Prideda elementą į galą (kopija).
- `void push_back (T &&value)`
Prideda elementą į galą (perkėlimas).
- `template<typename... Args>`
`reference emplace_back (Args &&... args)`
Sukonstruoja elementą gale vietoje.
- `void pop_back ()`
Pašalina paskutinį elementą.
- `void resize (size_type count, const T &value=T())`
Pakeičia vektoriaus dydį.
- `void swap (Vector &other) noexcept`
Sukeičia du vektorius vietomis (O(1)).

Privatūs Tipai

- `using AllocTraits = std::allocator_traits<Allocator>`

Privatūs Metodai

- `void deallocate_all ()`
Atlaisvina visą naudojamą atmintį.
- `void reallocate (size_type new_cap)`
Perskirsto atmintį naujam dydžiui.

Privatūs Atributai

- `pointer data_`
Rodyklė į duomenų masyvą
- `size_type size_`
Dabartinis elementų skaičius.
- `size_type capacity_`
Maksimalus galimas elementų skaičius be perskirstymo.
- `Allocator alloc_`
Atminties paskirstytojas.

4.3.1 Smulkus aprašymas

```
template<typename T, typename Allocator = std::allocator<T>>  
class Vector< T, Allocator >
```

Nuosava [Vector](#) konteinerio klasė.

Dinaminės atminties masyvas su automatine atminties valdymu. Veikia kaip pilnavertis `std::vector` pakaitalas.

Template Parameters

<i>T</i>	Elementų tipas
----------	----------------

<i>Allocator</i>	Atminties paskirstytojas (pagal nutylėjimą <code>std::allocator<T></code>)
------------------	---

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [vector.h](#) eilutėje 34.

4.3.2 Tipo Aprašymo Dokumentacija**4.3.2.1 allocator_type**

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::allocator_type = Allocator
```

Apibrėžimas failo [vector.h](#) eilutėje 38.

4.3.2.2 AllocTraits

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::AllocTraits = std::allocator_traits<Allocator> [private]
```

Apibrėžimas failo [vector.h](#) eilutėje 56.

4.3.2.3 const_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_iterator = const T*
```

Apibrėžimas failo [vector.h](#) eilutėje 46.

4.3.2.4 const_pointer

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_pointer = typename std::allocator_traits<Allocator>::const_pointer
```

Apibrėžimas failo [vector.h](#) eilutėje 44.

4.3.2.5 const_reference

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reference = const T&
```

Apibrėžimas failo [vector.h](#) eilutėje 42.

4.3.2.6 const_reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Apibrėžimas failo [vector.h](#) eilutėje 48.

4.3.2.7 difference_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

Apibrėžimas failo [vector.h](#) eilutėje 40.

4.3.2.8 iterator

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::iterator = T*
```

Apibrėžimas failo [vector.h](#) eilutėje 45.

4.3.2.9 pointer

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::pointer = typename std::allocator_traits<Allocator>::pointer
```

Apibrėžimas failo [vector.h](#) eilutėje 43.

4.3.2.10 reference

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::reference = T&
```

Apibrėžimas failo [vector.h](#) eilutėje 41.

4.3.2.11 reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
```

Apibrėžimas failo [vector.h](#) eilutėje 47.

4.3.2.12 size_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::size_type = std::size_t
```

Apibrėžimas failo [vector.h](#) eilutėje 39.

4.3.2.13 value_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::value_type = T
```

Apibrėžimas failo [vector.h](#) eilutėje 37.

4.3.3 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.3.3.1 Vector() [1/7]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector () [inline], [noexcept]
```

Numatytasis konstruktorius. Sukuria tuščią vektorių.

Apibrėžimas failo [vector.h](#) eilutėje 99.

```
00099 : data_(nullptr), size_(0), capacity_(0), alloc_() {}
```

4.3.3.2 Vector() [2/7]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Allocator & alloc) [inline], [explicit], [noexcept]
```

Konstruktorius su paskirstytoju.

Parametrai

<i>alloc</i>	Naudojamas paskirstytojas
--------------	---------------------------

Apibrėžimas failo [vector.h](#) eilutėje 106.

```
00107 : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00108 }
```

4.3.3.3 Vector() [3/7]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count,
    const T & value = T(),
    const Allocator & alloc = Allocator()) [inline], [explicit]
```

Konstruktorius su dydžiu.

Parametrai

<i>count</i>	Elementų skaičius
--------------	-------------------

<i>value</i>	Pradinė reikšmė (pagal nutylėjimą – numatytoji)
<i>alloc</i>	Paskirstytojas

Apibrėžimas failo [vector.h](#) eilutėje 116.

```
00118         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00119         assign(count, value);
00120     }
```

4.3.3.4 Vector() [4/7]

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator,
_tag, typename std::iterator_traits<InputIt>::iterator_category>>>
Vector< T, Allocator >::Vector (
    InputIt first,
    InputIt last,
    const Allocator & alloc = Allocator()) [inline]
```

Konstruktorius iš iteratorių diapazono.

Parametrai

<i>first</i>	Pradžios iteratorius
<i>last</i>	Pabaigos iteratorius
<i>alloc</i>	Paskirstytojas

Apibrėžimas failo [vector.h](#) eilutėje 133.

```
00134         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00135         assign(first, last);
00136     }
```

4.3.3.5 Vector() [5/7]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

Kopijos konstruktorius.

Parametrai

<i>other</i>	Kopijuojamas vektorius
--------------	------------------------

Apibrėžimas failo [vector.h](#) eilutėje 142.

```
00143         : data_(nullptr), size_(0), capacity_(0),
00144         alloc_(AllocTraits::select_on_container_copy_construction(other.alloc_)) {
00145         assign(other.begin(), other.end());
00146     }
```

4.3.3.6 Vector() [6/7]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo konstruktorius.

Parametrai

<i>other</i>	Perkeliamas vektorius
--------------	-----------------------

Apibrėžimas failo [vector.h](#) eilutėje 153.

```
00154         : data_(other.data_), size_(other.size_), capacity_(other.capacity_),
00155         alloc_(std::move(other.alloc_)) {
00156         other.data_ = nullptr;
00157         other.size_ = 0;
00158         other.capacity_ = 0;
00159     }
```

4.3.3.7 Vector() [7/7]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > il,
    const Allocator & alloc = Allocator()) [inline]
```

Konstruktorius iš std::initializer_list.

Parametrai

<i>il</i>	Pradinių reikšmių sąrašas
<i>alloc</i>	Paskirstytojas

Apibrėžimas failo [vector.h](#) eilutėje 166.

```
00167         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00168         assign(il.begin(), il.end());
00169     }
```

4.3.3.8 ~Vector()

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::~~Vector () [inline]
```

Destruktorius. Atlaisvina visą atmintį.

Apibrėžimas failo [vector.h](#) eilutėje 174.

```
00174 { deallocate_all(); }
```

4.3.4 Metodų Dokumentacija

4.3.4.1 assign() [1/3]

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator,
_tag, typename std::iterator_traits<InputIt>::iterator_category>>>
void Vector< T, Allocator >::assign (
    InputIt first,
    InputIt last) [inline]
```

Priskiria elementus iš iteratorių diapazono.

Apibrėžimas failo [vector.h](#) eilutėje 237.

```
00237                                     {
00238     clear();
00239     for (auto it = first; it != last; ++it)
00240         push_back(*it);
00241 }
```

4.3.4.2 assign() [2/3]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type count,
    const T & value) [inline]
```

Priskiria count kopijų reikšmės value.

Parametrai

<i>count</i>	Elementų skaičius
<i>value</i>	Reikšmė

Apibrėžimas failo [vector.h](#) eilutėje 222.

```
00222                                     {
00223     clear();
00224     reserve(count);
00225     for (size_type i = 0; i < count; ++i)
00226         AllocTraits::construct(alloc_, data_ + i, value);
00227     size_ = count;
00228 }
```

4.3.4.3 assign() [3/3]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    std::initializer_list< T > il) [inline]
```

Priskiria elementus iš initializer_list.

Apibrėžimas failo [vector.h](#) eilutėje 246.

```
00246                                     {
00247     assign(il.begin(), il.end());
00248 }
```

4.3.4.4 at() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type pos) [inline]
```

Prieiga prie elemento su ribų tikrinimu.

Išimtys

<code>std::out_of_range</code>	jei <code>pos >= size()</code>
--------------------------------	-----------------------------------

Apibrėžimas failo [vector.h](#) eilutėje 261.

```
00261 {
00262     if (pos >= size_) throw std::out_of_range("Vector::at - indeksas uz ribu");
00263     return data_[pos];
00264 }
```

4.3.4.5 at() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type pos) const [inline]
```

Prieiga prie elemento su ribų tikrinimu (const).

Apibrėžimas failo [vector.h](#) eilutėje 269.

```
00269 {
00270     if (pos >= size_) throw std::out_of_range("Vector::at - indeksas uz ribu");
00271     return data_[pos];
00272 }
```

4.3.4.6 back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::back () [inline]
```

Grąžina paskutinį elementą.

Apibrėžimas failo [vector.h](#) eilutėje 293.

```
00293 { return data_[size_ - 1]; }
```

4.3.4.7 back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::back () const [inline]
```

Apibrėžimas failo [vector.h](#) eilutėje 294.

```
00294 { return data_[size_ - 1]; }
```

4.3.4.8 begin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::begin () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 306.

```
00306 { return data_; }
```

4.3.4.9 begin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::begin () [inline], [noexcept]
```

Grąžina iteratorių į pradžia.

Apibrėžimas failo [vector.h](#) eilutėje 305.

```
00305 { return data_; }
```

4.3.4.10 capacity()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity () const [inline], [noexcept]
```

Grąžina dabartiną capacity.

Apibrėžimas failo [vector.h](#) eilutėje 350.

```
00350 { return capacity_; }
```

4.3.4.11 cbegin()

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cbegin () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 307.

```
00307 { return data_; }
```

4.3.4.12 cend()

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cend () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 312.

```
00312 { return data_ + size_; }
```


4.3.4.13 clear()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear () [inline], [noexcept]
```

Ištrina visus elementus (size = 0, capacity nesikeičia).

Apibrėžimas failo [vector.h](#) eilutėje 363.

```
00363         {
00364             for (size_type i = 0; i < size_; ++i)
00365                 AllocTraits::destroy(alloc_, data_ + i);
00366             size_ = 0;
00367         }
```

4.3.4.14 crbegin()

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crbegin () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 317.

```
00317 { return const_reverse_iterator(end()); }
```

4.3.4.15 crend()

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crend () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 322.

```
00322 { return const_reverse_iterator(begin()); }
```

4.3.4.16 data() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::data () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 300.

```
00300 { return data_; }
```

4.3.4.17 data() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T * Vector< T, Allocator >::data () [inline], [noexcept]
```

Grąžina rodyklę į vidinį duomenų masyvą.

Apibrėžimas failo [vector.h](#) eilutėje 299.

```
00299 { return data_; }
```

4.3.4.18 deallocate_all()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::deallocate_all () [inline], [private]
```

Atlaisvina visą naudojamą atmintį.

Apibrėžimas failo `vector.h` eilutėje 61.

```
00061         {
00062             if (data_) {
00063                 for (size_type i = 0; i < size_; ++i)
00064                     AllocTraits::destroy(alloc_, data_ + i);
00065                 AllocTraits::deallocate(alloc_, data_, capacity_);
00066                 data_ = nullptr;
00067             }
00068             size_ = 0;
00069             capacity_ = 0;
00070         }
```

4.3.4.19 emplace()

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename... Args>
iterator Vector< T, Allocator >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

Sukonstruoja elementą vietoje prieš pos.

Apibrėžimas failo `vector.h` eilutėje 462.

```
00462         {
00463             size_type idx = pos - cbegin();
00464             if (size_ == capacity_)
00465                 reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00466             for (size_type i = size_; i > idx; --i) {
00467                 size_type old_idx = i - 1;
00468                 size_type new_idx = old_idx + 1;
00469                 if (new_idx >= size_) {
00470                     AllocTraits::construct(alloc_, data_ + new_idx, std::move(data_[old_idx]));
00471                 }
00472                 else {
00473                     data_[new_idx] = std::move(data_[old_idx]);
00474                 }
00475             }
00476             if (idx < size_) {
00477                 AllocTraits::destroy(alloc_, data_ + idx);
00478             }
00479             AllocTraits::construct(alloc_, data_ + idx, std::forward<Args>(args)...);
00480             ++size_;
00481             return begin() + idx;
00482         }
```

4.3.4.20 emplace_back()

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args) [inline]
```

Sukonstruoja elementą gale vietoje.

Gražina

Nuoroda į sukonstruotą elementą

Apibrėžimas failo [vector.h](#) eilutėje 528.

```
00528                                     {
00529         if (size_ == capacity_)
00530             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00531         AllocTraits::construct(alloc_, data_ + size_, std::forward<Args>(args)...);
00532         return data_[size_++];
00533     }
```

4.3.4.21 empty()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty () const [inline], [nodiscard], [noexcept]
```

Tikrina ar vektorius tuščias.

Apibrėžimas failo [vector.h](#) eilutėje 327.

```
00327 { return size_ == 0; }
```

4.3.4.22 end() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::end () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 311.

```
00311 { return data_ + size_; }
```

4.3.4.23 end() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::end () [inline], [noexcept]
```

Gražina iteratorių į pabaigą (past-the-end).

Apibrėžimas failo [vector.h](#) eilutėje 310.

```
00310 { return data_ + size_; }
```

4.3.4.24 erase() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

Ištrina elementus diapazone [first, last).

Apibrėžimas failo [vector.h](#) eilutėje 498.

```
00498                                     {
00499         iterator f = begin() + (first - cbegin());
00500         iterator l = begin() + (last - cbegin());
00501         size_type count = l - f;
00502         std::move(l, end(), f);
00503         for (size_type i = 0; i < count; ++i)
00504             AllocTraits::destroy(alloc_, end() - 1 - i);
00505         size_ -= count;
00506         return f;
00507     }
```

4.3.4.25 erase() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator pos) [inline]
```

Ištrina elementą ties pos.

Gražina

Iteratorius į kitą elementą po ištrintojo

Apibrėžimas failo [vector.h](#) eilutėje 491.

```
00491 {
00492     return erase(pos, pos + 1);
00493 }
```

4.3.4.26 front() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::front () [inline]
```

Gražina pirmą elementą.

Apibrėžimas failo [vector.h](#) eilutėje 287.

```
00287 { return data_[0]; }
```

4.3.4.27 front() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::front () const [inline]
```

Apibrėžimas failo [vector.h](#) eilutėje 288.

```
00288 { return data_[0]; }
```

4.3.4.28 get_allocator()

```
template<typename T, typename Allocator = std::allocator<T>>
allocator_type Vector< T, Allocator >::get_allocator () const [inline], [noexcept]
```

Gražina paskirstytojo kopiją.

Apibrėžimas failo [vector.h](#) eilutėje 253.

```
00253 { return alloc_; }
```

4.3.4.29 insert() [1/5]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    const T & value) [inline]
```

Įterpia elementą prieš pos.

Parametrai

<i>pos</i>	Iteratorius į vietą
------------	---------------------

<i>value</i>	Įterpiama reikšmė
--------------	-------------------

Gražina

Iteratorius į įterptą elementą

Apibrėžimas failo [vector.h](#) eilutėje 375.

```
00375                                     {
00376         return emplace(pos, value);
00377     }
```

4.3.4.30 insert() [2/5]

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator,
_tag, typename std::iterator_traits<InputIt>::iterator_category>>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last) [inline]
```

Įterpia elementus iš diapazono prieš pos.

Apibrėžimas failo [vector.h](#) eilutėje 424.

```
00424                                     {
00425         size_type idx = pos - cbegin();
00426         Vector tmp(first, last);
00427         size_type count = tmp.size();
00428         if (count == 0) return begin() + idx;
00429         if (size_ + count > capacity_)
00430             reallocate(std::max(size_ + count, capacity_ * 2));
00431
00432         for (size_type i = size_; i > idx; --i) {
00433             size_type old_idx = i - 1;
00434             size_type new_idx = old_idx + count;
00435             if (new_idx >= size_) {
00436                 AllocTraits::construct(alloc_, data_ + new_idx, std::move(data_[old_idx]));
00437             }
00438             else {
00439                 data_[new_idx] = std::move(data_[old_idx]);
00440             }
00441         }
00442
00443         size_type affected_constructed = std::min(size_, idx + count);
00444         for (size_type i = idx; i < affected_constructed; ++i) {
00445             AllocTraits::destroy(alloc_, data_ + i);
00446         }
00447
00448         for (size_type i = 0; i < count; ++i)
00449             AllocTraits::construct(alloc_, data_ + idx + i, std::move(tmp[i]));
00450         size_ += count;
00451         return begin() + idx;
00452     }
```

4.3.4.31 insert() [3/5]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    size_type count,
    const T & value) [inline]
```

Įterpia count kopijų reikšmės prieš pos.

Apibrėžimas failo [vector.h](#) eilutėje 386.

```

00386                                     {
00387     size_type idx = pos - cbegin();
00388     if (count == 0) return begin() + idx;
00389     if (size_ + count > capacity_) {
00390         size_type new_cap = std::max(size_ + count, capacity_ * 2);
00391         reallocate(new_cap);
00392     }
00393
00394     for (size_type i = size_; i > idx; --i) {
00395         size_type old_idx = i - 1;
00396         size_type new_idx = old_idx + count;
00397         if (new_idx >= size_) {
00398             AllocTraits::construct(alloc_, data_ + new_idx, std::move(data_[old_idx]));
00399         }
00400         else {
00401             data_[new_idx] = std::move(data_[old_idx]);
00402         }
00403     }
00404
00405     size_type affected_constructed = std::min(size_, idx + count);
00406     for (size_type i = idx; i < affected_constructed; ++i) {
00407         AllocTraits::destroy(alloc_, data_ + i);
00408     }
00409
00410     for (size_type i = 0; i < count; ++i) {
00411         AllocTraits::construct(alloc_, data_ + idx + i, value);
00412     }
00413     size_ += count;
00414     return begin() + idx;
00415 }

```

4.3.4.32 insert() [4/5]

```

template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    std::initializer_list< T > il) [inline]

```

Apibrėžimas failo [vector.h](#) eilutėje 454.

```

00454                                     {
00455     return insert(pos, il.begin(), il.end());
00456 }

```

4.3.4.33 insert() [5/5]

```

template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    T && value) [inline]

```

Apibrėžimas failo [vector.h](#) eilutėje 379.

```

00379                                     {
00380     return emplace(pos, std::move(value));
00381 }

```

4.3.4.34 max_size()

```

template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline], [noexcept]

```

Grąžina maksimalų galimą elementų skaičių.

Apibrėžimas failo [vector.h](#) eilutėje 333.

```

00333                                     {
00334     return AllocTraits::max_size(alloc_);
00335 }

```

4.3.4.35 operator=() [1/3]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

Kopijos priskyrimo operatorius.

Apibrėžimas failo [vector.h](#) eilutėje 181.

```
00181 {
00182     if (this == &other) return *this;
00183     if constexpr (AllocTraits::propagate_on_container_copy_assignment::value)
00184         alloc_ = other.alloc_; // <- Čia ištaisyta klaida (buvo alloc = ...)
00185     assign(other.begin(), other.end());
00186     return *this;
00187 }
```

4.3.4.36 operator=() [2/3]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    std::initializer_list< T > il) [inline]
```

Priskyrimo operatorius iš initializer_list.

Apibrėžimas failo [vector.h](#) eilutėje 210.

```
00210 {
00211     assign(il.begin(), il.end());
00212     return *this;
00213 }
```

4.3.4.37 operator=() [3/3]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius.

Apibrėžimas failo [vector.h](#) eilutėje 193.

```
00193 {
00194     if (this == &other) return *this;
00195     deallocate_all();
00196     if constexpr (AllocTraits::propagate_on_container_move_assignment::value)
00197         alloc_ = std::move(other.alloc_);
00198     data_ = other.data_;
00199     size_ = other.size_;
00200     capacity_ = other.capacity_;
00201     other.data_ = nullptr;
00202     other.size_ = 0;
00203     other.capacity_ = 0;
00204     return *this;
00205 }
```

4.3.4.38 operator[]() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::operator[] (
    size_type pos) [inline]
```

Prieiga prie elemento pagal indeksą (be tikrinimo).

Apibrėžimas failo [vector.h](#) eilutėje 277.

```
00277 { return data_[pos]; }
```

4.3.4.39 operator[]() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::operator[] (
    size_type pos) const [inline]
```

Prieiga prie elemento pagal indeksą (be tikrinimo, const).

Apibrėžimas failo [vector.h](#) eilutėje 282.

```
00282 { return data_[pos]; }
```

4.3.4.40 pop_back()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back () [inline]
```

Pašalina paskutinį elementą.

Apibrėžimas failo [vector.h](#) eilutėje 538.

```
00538 {
00539     AllocTraits::destroy(alloc_, data_ + --size_);
00540 }
```

4.3.4.41 push_back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const T & value) [inline]
```

Prideda elementą į galą (kopija).

Parametrai

<i>value</i>	Pridedama reikšmė
--------------	-------------------

Apibrėžimas failo [vector.h](#) eilutėje 513.

```
00513 {
00514     emplace_back(value);
00515 }
```

4.3.4.42 push_back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    T && value) [inline]
```

Prideda elementą į galą (perkėlimas).

Apibrėžimas failo [vector.h](#) eilutėje 519.

```
00519 {
00520     emplace_back(std::move(value));
00521 }
```


4.3.4.43 rbegin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rbegin () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 316.

```
00316 { return const_reverse_iterator(end()); }
```

4.3.4.44 rbegin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rbegin () [inline], [noexcept]
```

Gražina atvirkštinį iteratorių.

Apibrėžimas failo [vector.h](#) eilutėje 315.

```
00315 { return reverse_iterator(end()); }
```

4.3.4.45 reallocate()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reallocate (
    size_type new_cap) [inline], [private]
```

Perskirsto atmintį naujam dydžiui.

Parametrai

<i>new_cap</i>	Naujas capacity dydis
----------------	-----------------------

Apibrėžimas failo [vector.h](#) eilutėje 75.

```
00075 {
00076     pointer new_data = AllocTraits::allocate(alloc_, new_cap);
00077     size_type i = 0;
00078     try {
00079         for (; i < size_; ++i)
00080             AllocTraits::construct(alloc_, new_data + i, std::move_if_noexcept(data_[i]));
00081     }
00082     catch (...) {
00083         for (size_type j = 0; j < i; ++j)
00084             AllocTraits::destroy(alloc_, new_data + j);
00085         AllocTraits::deallocate(alloc_, new_data, new_cap);
00086         throw;
00087     }
00088     deallocate_all();
00089     data_ = new_data;
00090     size_ = i;
00091     capacity_ = new_cap;
00092 }
```

4.3.4.46 rend() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rend () const [inline], [noexcept]
```

Apibrėžimas failo [vector.h](#) eilutėje 321.

```
00321 { return const_reverse_iterator(begin()); }
```

4.3.4.47 `rend()` [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rend () [inline], [noexcept]
```

Grąžina atvirkštinį past-the-end iteratorių.

Apibrėžimas failo [vector.h](#) eilutėje 320.

```
00320 { return reverse_iterator(begin()); }
```

4.3.4.48 `reserve()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type new_cap) [inline]
```

Rezervuoja atmintį `new_cap` elementams.

Parametrai

<code>new_cap</code>	Naujas capacity
----------------------	-----------------

Išimtys

<code>std::length_error</code>	jei <code>new_cap > max_size()</code>
--------------------------------	--

Apibrėžimas failo [vector.h](#) eilutėje 342.

```
00342 {
00343     if (new_cap <= capacity_) return;
00344     if (new_cap > max_size())
00345         throw std::length_error("Vector::reserve - per didelis dydis");
00346     reallocate(new_cap);
00347 }
```

4.3.4.49 `resize()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count,
    const T & value = T()) [inline]
```

Pakeičia vektoriaus dydį.

Parametrai

<code>count</code>	Naujas dydis
<code>value</code>	Reikšmė naujiems elementams (jei didėja)

Apibrėžimas failo [vector.h](#) eilutėje 547.

```
00547 {
00548     if (count < size_) {
00549         for (size_type i = count; i < size_; ++i)
00550             AllocTraits::destroy(alloc_, data_ + i);
00551         size_ = count;
00552     }
00553     else if (count > size_) {
00554         reserve(count);
00555         for (size_type i = size_; i < count; ++i)
00556             AllocTraits::construct(alloc_, data_ + i, value);
00557         size_ = count;
00558     }
00559 }
```

4.3.4.50 shrink_to_fit()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

Sumažina capacity iki size (shrink to fit).

Apibrėžimas failo [vector.h](#) eilutėje 355.

```
00355         {
00356             if (size_ < capacity_)
00357                 reallocate(size_);
00358     }
```

4.3.4.51 size()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size () const [inline], [noexcept]
```

Grąžina elementų skaičių.

Apibrėžimas failo [vector.h](#) eilutėje 330.

```
00330 { return size_; }
```

4.3.4.52 swap()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline], [noexcept]
```

Sukeičia du vektorius vietomis (O(1)).

Parametrai

<i>other</i>	Kitas vektorius
--------------	-----------------

Apibrėžimas failo [vector.h](#) eilutėje 565.

```
00565         {
00566             std::swap(data_, other.data_);
00567             std::swap(size_, other.size_);
00568             std::swap(capacity_, other.capacity_);
00569             if constexpr (AllocTraits::propagate_on_container_swap::value)
00570                 std::swap(alloc_, other.alloc_);
00571     }
```

4.3.5 Atributų Dokumentacija

4.3.5.1 alloc_

```
template<typename T, typename Allocator = std::allocator<T>>
Allocator Vector< T, Allocator >::alloc_ [private]
```

Atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 54.

4.3.5.2 capacity_

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity_ [private]
```

Maksimalus galimas elementų skaičius be perskirstymo.

Apibrėžimas failo [vector.h](#) eilutėje 53.

4.3.5.3 data_

```
template<typename T, typename Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::data_ [private]
```

Rodyklė į duomenų masyvą

Apibrėžimas failo [vector.h](#) eilutėje 51.

4.3.5.4 size_

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size_ [private]
```

Dabartinis elementų skaičius.

Apibrėžimas failo [vector.h](#) eilutėje 52.

Dokumentacija šiai klasei sugeneruota iš šio failo:

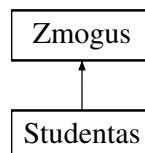
- [vector.h](#)

4.4 Zmogus Klasė

Abstrakti bazinė klase, atstovaujanti žmogui.

```
#include <zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) ()
Numatytasis konstruktorius Sukuria tuscia zmogu su tusciais vardu ir pavarde.
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
Parametrinis konstruktorius.
- [Zmogus](#) (const [Zmogus](#) &other)
Kopijavimo konstruktorius.
- [Zmogus](#) ([Zmogus](#) &&other) noexcept
Perkelimo konstruktorius.
- virtual ~[Zmogus](#) ()=0
Virtualus destruktorius.
- [Zmogus](#) & operator= (const [Zmogus](#) &other)
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) & operator= ([Zmogus](#) &&other) noexcept
Perkelimo priskyrimo operatorius.
- const std::string & [getVardas](#) () const
- const std::string & [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)
- virtual void [print](#) (std::ostream &os) const =0
Grynas virtualus metodas spausdinimui.

Apsaugoti Atributai

- std::string [vardas_](#)
- std::string [pavarde_](#)

4.4.1 Smulkus aprašymas

Abstrakti bazine klase, atstovaujanti zmogu.

Negalima sukurti [Zmogus](#) tipo objektu tiesiogiai - klase abstrakti del grynai virtualaus destruktoriaus. Is jos galima kurti tik isvestines klases (pvz. [Studentas](#)).

Autorius

Studentu Valdymo Sistema

Versija

2.0

Pavyzdžiai

C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h.

Apibrėžimas failo [zmogus.h](#) eilutėje 19.

4.4.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.4.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus ()
```

Numatytasis konstruktorius Sukuria tuscia zmogu su tusciais vardu ir pavarde.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 4.

```
00005      : vardas_(""), pavarde_("") {
00006  }
```

4.4.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde)
```

Parametrinis konstruktorius.

Parametrai

<i>vardas</i>	Asmens vardas
<i>pavarde</i>	Asmens pavarde

Apibrėžimas failo [zmogus.cpp](#) eilutėje 9.

```
00010      : vardas_(vardas), pavarde_(pavarde) {
00011  }
```

4.4.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & other)
```

Kopijavimo konstruktorius.

Parametrai

<i>other</i>	Zmogus objektas, is kurio kopijuojama
--------------	---

Apibrėžimas failo [zmogus.cpp](#) eilutėje 14.

```
00015      : vardas_(other.vardas_), pavarde_(other.pavarde_) {
00016  }
```

4.4.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && other) [noexcept]
```

Perkelimo konstruktorius.

Parametrai

<i>other</i>	Zmogus objektas, is kurio perkeliama
--------------	--------------------------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 19.

```
00020 : vardas_(std::move(other.vardas_)),
00021 pavarde_(std::move(other.pavarde_)) {
00022 }
```

4.4.2.5 ~Zmogus()

```
Zmogus::~Zmogus () [pure virtual]
```

Virtualus destruktorius.

Grynas virtualus destruktorius - daro klase abstrakcia. Isvestines klases privalo ji realizuoti.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 27.

```
00027 {}
```

4.4.3 Metodų Dokumentacija

4.4.3.1 getPavarde()

```
const std::string & Zmogus::getPavarde () const [inline]
```

Gražina

Asmens pavarde

Apibrėžimas failo [zmogus.h](#) eilutėje 83.

```
00083 { return pavarde_; }
```

4.4.3.2 getVardas()

```
const std::string & Zmogus::getVardas () const [inline]
```

Gražina

Asmens vardas

Apibrėžimas failo [zmogus.h](#) eilutėje 80.

```
00080 { return vardas_; }
```

4.4.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Zmogus objektas, is kurio kopijuojama
--------------	---------------------------------------

Gražina

Nuoroda i si objekta

Apibrėžimas failo [zmogus.cpp](#) eilutėje 30.

```
00030                                     {
00031     if (this != &other) {
00032         vardas_ = other.vardas_;
00033         pavarde_ = other.pavarde_;
00034     }
00035     return *this;
00036 }
```

4.4.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [noexcept]
```

Perkelimo priskyrimo operatorius.

Parametrai

<i>other</i>	Zmogus objektas, is kurio perkeliama
--------------	--------------------------------------

Gražina

Nuoroda i si objekta

Apibrėžimas failo [zmogus.cpp](#) eilutėje 39.

```
00039                                     {
00040     if (this != &other) {
00041         vardas_ = std::move(other.vardas_);
00042         pavarde_ = std::move(other.pavarde_);
00043     }
00044     return *this;
00045 }
```

4.4.3.5 print()

```
virtual void Zmogus::print (
    std::ostream & os) const [pure virtual]
```

Grynas virtualus metodus spausdinimui.

Parametrai

<i>os</i>	Isvesties srautas
-----------	-------------------

Isvestines klases privalo realizuoti si metoda. Naudojamas operator<< globaliame operatoriuje.

Realizuota [Studentas](#).

4.4.3.6 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p) [inline]
```

Parametrai

<i>p</i>	Nauja pavarde
----------	---------------

Apibrėžimas failo [zmogus.h](#) eilutėje 91.

```
00091 { pavarde_ = p; }
```

4.4.3.7 setVardas()

```
void Zmogus::setVardas (
    const std::string & v) [inline]
```

Parametrai

<i>v</i>	Naujas vardas
----------	---------------

Apibrėžimas failo [zmogus.h](#) eilutėje 88.

```
00088 { vardas_ = v; }
```

4.4.4 Atributų Dokumentacija

4.4.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Apibrėžimas failo [zmogus.h](#) eilutėje 22.

4.4.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Apibrėžimas failo [zmogus.h](#) eilutėje 21.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [zmogus.h](#)
- [zmogus.cpp](#)

skyrius 5

Failo Dokumentacija

5.1 deque_ops.cpp Failo Nuoroda

```
#include "deque_ops.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <iomanip>
#include <stdexcept>
#include <chrono>
#include "vector.h"
```

Funkcijos

- void `skaitymas_is_failo_d` (const std::string &filename, std::deque< `Studentas` > &studentai)
- void `isvedimas_i_faila_d` (const std::deque< `Studentas` > &studentai, const std::string &filename, const std::string &kategorija)
- void `pasirinkimo_metodas_d` (int tipas, std::deque< `Studentas` > &studentai)
- void `rusiavimas_d` (std::deque< `Studentas` > &studentai)
- double `skirstymas_i_grupes_d` (const std::deque< `Studentas` > &visi, std::deque< `Studentas` > &kieti, std::deque< `Studentas` > &vargsai)
- double `skirstymas_s1_d` (const std::deque< `Studentas` > &visi, std::deque< `Studentas` > &kieti, std::deque< `Studentas` > &vargsai)
- double `skirstymas_s2_d` (std::deque< `Studentas` > &studentai, std::deque< `Studentas` > &vargsai)
- double `skirstymas_s3_d` (std::deque< `Studentas` > &studentai, std::deque< `Studentas` > &vargsai)

5.1.1 Funkcijos Dokumentacija

5.1.1.1 isvedimas_i_faila_d()

```
void isvedimas_i_faila_d (
    const std::deque< Studentas > & studentai,
```

```
const std::string & filename,
const std::string & kategorija)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 36.

```
00038 {
00039     std::ofstream out(filename);
00040     if (!out.is_open())
00041         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00042     out << std::left << std::setw(20) << "Vardas"
00043         << std::setw(20) << "Pavarde"
00044         << std::right << std::setw(20) << "Galutinis\n"
00045         << std::string(60, '-') << "\n";
00046     for (const auto& s : studentai) out << s << "\n";
00047     std::cout << " [" << kategorija << "] " << studentai.size()
00048         << " studentu -> " << filename << "\n";
00049 }
```

5.1.1.2 pasirinkimo_metodas_d()

```
void pasirinkimo_metodas_d (
    int tipas,
    std::deque< Studentas > & studentai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 51.

```
00051 {
00052     for (auto& s : studentai) s.skaiciuotiRez(tipas);
00053 }
```

5.1.1.3 rusiavimas_d()

```
void rusiavimas_d (
    std::deque< Studentas > & studentai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 56.

```
00056 {
00057     std::sort(studentai.begin(), studentai.end(),
00058         [](const Studentas& a, const Studentas& b) { return a.getRez() < b.getRez(); });
00059 }
```

5.1.1.4 skaitymas_is_failo_d()

```
void skaitymas_is_failo_d (
    const std::string & filename,
    std::deque< Studentas > & studentai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 11.

```
00011 {
00012     std::ifstream file(filename);
00013     if (!file.is_open())
00014         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00015     studentai.clear();
00016     std::string header, line;
00017     std::getline(file, header);
00018     int nr = 0;
00019     while (std::getline(file, line)) {
00020         ++nr;
00021         std::stringstream ss(line);
00022         std::string vardas, pavarde;
00023         ss >> vardas >> pavarde;
00024         Vector<int> paz;
00025         int v;
00026         while (ss >> v) paz.push_back(v);
00027         if (paz.empty()) continue;
00028         int egz = paz.back(); paz.pop_back();
00029         studentai.emplace_back(vardas, pavarde, paz, egz);
00030         if (nr % 1000000 == 0)
00031             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00032     }
00033     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00034 }
```

5.1.1.5 skirstymas_i_grupes_d()

```
double skirstymas_i_grupes_d (
    const std::deque< Studentas > & visi,
    std::deque< Studentas > & kieti,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo [deque_ops.cpp](#) eilutėje 61.

```
00063 {
00064     auto t0 = std::chrono::high_resolution_clock::now();
00065     kieti.clear(); vargsai.clear();
00066     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00067         [](const Studentas& s) { return s.getRez() < 5.0; });
00068     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00069         [](const Studentas& s) { return s.getRez() >= 5.0; });
00070     return std::chrono::duration<double>{
00071         std::chrono::high_resolution_clock::now() - t0}.count();
00072 }
```

5.1.1.6 skirstymas_s1_d()

```
double skirstymas_s1_d (
    const std::deque< Studentas > & visi,
    std::deque< Studentas > & kieti,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo [deque_ops.cpp](#) eilutėje 76.

```
00078 {
00079     auto t0 = std::chrono::high_resolution_clock::now();
00080     kieti.clear(); vargsai.clear();
00081     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00082         [](const Studentas& s) { return s.getRez() >= 5.0; });
00083     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00084         [](const Studentas& s) { return s.getRez() < 5.0; });
00085     return std::chrono::duration<double>{
00086         std::chrono::high_resolution_clock::now() - t0}.count();
00087 }
```

5.1.1.7 skirstymas_s2_d()

```
double skirstymas_s2_d (
    std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo [deque_ops.cpp](#) eilutėje 90.

```
00091 {
00092     auto t0 = std::chrono::high_resolution_clock::now();
00093     vargsai.clear();
00094     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00095         [](const Studentas& s) { return s.getRez() < 5.0; });
00096     studentai.erase(
00097         std::remove_if(studentai.begin(), studentai.end(),
00098             [](const Studentas& s) { return s.getRez() < 5.0; })),
00099     studentai.end());
00100     return std::chrono::duration<double>{
00101         std::chrono::high_resolution_clock::now() - t0}.count();
00102 }
```

5.1.1.8 skirstymas_s3_d()

```
double skirstymas_s3_d (
    std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 105.

```
00106 {
00107     auto t0 = std::chrono::high_resolution_clock::now();
00108     vargsai.clear();
00109     auto pivot = std::stable_partition(studentai.begin(), studentai.end(),
00110         [](const Studentas& s) { return s.getRez() < 5.0; });
00111     vargsai.assign(studentai.begin(), pivot);
00112     studentai.erase(studentai.begin(), pivot);
00113     return std::chrono::duration<double>(
00114         std::chrono::high_resolution_clock::now() - t0).count();
00115 }
```

5.2 deque_ops.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "deque_ops.h"
00002 #include <iostream>
00003 #include <fstream>
00004 #include <sstream>
00005 #include <algorithm>
00006 #include <iomanip>
00007 #include <stdexcept>
00008 #include <chrono>
00009 #include "vector.h"
00010
00011 void skaitymas_is_failo_d(const std::string& filename, std::deque<Studentas>& studentai) {
00012     std::ifstream file(filename);
00013     if (!file.is_open())
00014         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00015     studentai.clear();
00016     std::string header, line;
00017     std::getline(file, header);
00018     int nr = 0;
00019     while (std::getline(file, line)) {
00020         ++nr;
00021         std::stringstream ss(line);
00022         std::string vardas, pavarde;
00023         ss >> vardas >> pavarde;
00024         Vector<int> paz;
00025         int v;
00026         while (ss >> v) paz.push_back(v);
00027         if (paz.empty()) continue;
00028         int egz = paz.back(); paz.pop_back();
00029         studentai.emplace_back(vardas, pavarde, paz, egz);
00030         if (nr % 1000000 == 0)
00031             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00032     }
00033     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00034 }
00035
00036 void isvedimas_i_faila_d(const std::deque<Studentas>& studentai,
00037     const std::string& filename,
00038     const std::string& kategorija) {
00039     std::ofstream out(filename);
00040     if (!out.is_open())
00041         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00042     out << std::left << std::setw(20) << "Vardas"
00043         << std::setw(20) << "Pavarde"
00044         << std::right << std::setw(20) << "Galutinis\n"
00045         << std::string(60, '-') << "\n";
00046     for (const auto& s : studentai) out << s << "\n";
00047     std::cout << " [" << kategorija << "]" << " studentai.size() "
00048         << " studentu -> " << filename << "\n";
00049 }
00050
00051 void pasirinkimo_metodas_d(int tipas, std::deque<Studentas>& studentai) {
00052     for (auto& s : studentai) s.skaiciuotiRez(tipas);
00053 }
00054
00055 // deque turi atsitiktine prieiga - naudojame std::sort
```

```

00056 void rusiavimas_d(std::deque<Studentas>& studentai) {
00057     std::sort(studentai.begin(), studentai.end(),
00058         [](const Studentas& a, const Studentas& b) { return a.getRez() < b.getRez(); });
00059 }
00060
00061 double skirstymas_i_grupes_d(const std::deque<Studentas>& visi,
00062     std::deque<Studentas>& kieti,
00063     std::deque<Studentas>& vargsai) {
00064     auto t0 = std::chrono::high_resolution_clock::now();
00065     kieti.clear(); vargsai.clear();
00066     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00067         [](const Studentas& s) { return s.getRez() < 5.0; });
00068     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00069         [](const Studentas& s) { return s.getRez() >= 5.0; });
00070     return std::chrono::duration<double>(
00071         std::chrono::high_resolution_clock::now() - t0).count();
00072 }
00073
00074 // STRATEGIJA 1 - du nauji konteineriai, originalas lieka
00075 // Naudoja std::copy_if
00076 double skirstymas_sl_d(const std::deque<Studentas>& visi,
00077     std::deque<Studentas>& kieti,
00078     std::deque<Studentas>& vargsai) {
00079     auto t0 = std::chrono::high_resolution_clock::now();
00080     kieti.clear(); vargsai.clear();
00081     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00082         [](const Studentas& s) { return s.getRez() >= 5.0; });
00083     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00084         [](const Studentas& s) { return s.getRez() < 5.0; });
00085     return std::chrono::duration<double>(
00086         std::chrono::high_resolution_clock::now() - t0).count();
00087 }
00088
00089 // STRATEGIJA 2 - vienas naujas konteineris + erase/remove_if
00090 double skirstymas_s2_d(std::deque<Studentas>& studentai,
00091     std::deque<Studentas>& vargsai) {
00092     auto t0 = std::chrono::high_resolution_clock::now();
00093     vargsai.clear();
00094     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00095         [](const Studentas& s) { return s.getRez() < 5.0; });
00096     studentai.erase(
00097         std::remove_if(studentai.begin(), studentai.end(),
00098             [](const Studentas& s) { return s.getRez() < 5.0; }),
00099         studentai.end());
00100     return std::chrono::duration<double>(
00101         std::chrono::high_resolution_clock::now() - t0).count();
00102 }
00103
00104 // STRATEGIJA 3 - stable_partition (greiciausia deque atveju)
00105 double skirstymas_s3_d(std::deque<Studentas>& studentai,
00106     std::deque<Studentas>& vargsai) {
00107     auto t0 = std::chrono::high_resolution_clock::now();
00108     vargsai.clear();
00109     auto pivot = std::stable_partition(studentai.begin(), studentai.end(),
00110         [](const Studentas& s) { return s.getRez() < 5.0; });
00111     vargsai.assign(studentai.begin(), pivot);
00112     studentai.erase(studentai.begin(), pivot);
00113     return std::chrono::duration<double>(
00114         std::chrono::high_resolution_clock::now() - t0).count();
00115 }

```

5.3 deque_ops.h Failo Nuoroda

```

#include "studentas.h"
#include <deque>
#include <string>

```

Funkcijos

- void `skaitymas_is_failo_d` (const std::string &filename, std::deque< [Studentas](#) > &studentai)
- void `isvedimas_i_faila_d` (const std::deque< [Studentas](#) > &studentai, const std::string &filename, const std::string &kategorija)
- void `pasirinkimo_metodas_d` (int tipas, std::deque< [Studentas](#) > &studentai)

- void rusiavimas_d (std::deque< Studentas > &studentai)
- double skirstymas_i_grupes_d (const std::deque< Studentas > &visi, std::deque< Studentas > &kieti, std::deque< Studentas > &vargsai)
- double skirstymas_s1_d (const std::deque< Studentas > &visi, std::deque< Studentas > &kieti, std::deque< Studentas > &vargsai)
- double skirstymas_s2_d (std::deque< Studentas > &studentai, std::deque< Studentas > &vargsai)
- double skirstymas_s3_d (std::deque< Studentas > &studentai, std::deque< Studentas > &vargsai)

5.3.1 Funkcijos Dokumentacija

5.3.1.1 isvedimas_i_faila_d()

```
void isvedimas_i_faila_d (
    const std::deque< Studentas > & studentai,
    const std::string & filename,
    const std::string & kategorija)
```

Apibrėžimas failo deque_ops.cpp eilutėje 36.

```
00038 {
00039     std::ofstream out(filename);
00040     if (!out.is_open())
00041         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00042     out << std::left << std::setw(20) << "Vardas"
00043         << std::setw(20) << "Pavarde"
00044         << std::right << std::setw(20) << "Galutinis\n"
00045         << std::string(60, '-') << "\n";
00046     for (const auto& s : studentai) out << s << "\n";
00047     std::cout << " [" << kategorija << "]" << studentai.size()
00048         << " studentu -> " << filename << "\n";
00049 }
```

5.3.1.2 pasirinkimo_metodas_d()

```
void pasirinkimo_metodas_d (
    int tipas,
    std::deque< Studentas > & studentai)
```

Apibrėžimas failo deque_ops.cpp eilutėje 51.

```
00051 {
00052     for (auto& s : studentai) s.skaiciuotiRez(tipas);
00053 }
```

5.3.1.3 rusiavimas_d()

```
void rusiavimas_d (
    std::deque< Studentas > & studentai)
```

Apibrėžimas failo deque_ops.cpp eilutėje 56.

```
00056 {
00057     std::sort(studentai.begin(), studentai.end(),
00058         [](const Studentas& a, const Studentas& b) { return a.getRez() < b.getRez(); });
00059 }
```


5.3.1.4 skaitymas_is_failo_d()

```
void skaitymas_is_failo_d (
    const std::string & filename,
    std::deque< Studentas > & studentai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 11.

```
00011 {
00012     std::ifstream file(filename);
00013     if (!file.is_open())
00014         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00015     studentai.clear();
00016     std::string header, line;
00017     std::getline(file, header);
00018     int nr = 0;
00019     while (std::getline(file, line)) {
00020         ++nr;
00021         std::stringstream ss(line);
00022         std::string vardas, pavarde;
00023         ss >> vardas >> pavarde;
00024         Vector<int> paz;
00025         int v;
00026         while (ss >> v) paz.push_back(v);
00027         if (paz.empty()) continue;
00028         int egz = paz.back(); paz.pop_back();
00029         studentai.emplace_back(vardas, pavarde, paz, egz);
00030         if (nr % 1000000 == 0)
00031             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00032     }
00033     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00034 }
```

5.3.1.5 skirstymas_i_grupes_d()

```
double skirstymas_i_grupes_d (
    const std::deque< Studentas > & visi,
    std::deque< Studentas > & kieti,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 61.

```
00063 {
00064     auto t0 = std::chrono::high_resolution_clock::now();
00065     kieti.clear(); vargsai.clear();
00066     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00067         [](const Studentas& s) { return s.getRez() < 5.0; });
00068     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00069         [](const Studentas& s) { return s.getRez() >= 5.0; });
00070     return std::chrono::duration<double>(
00071         std::chrono::high_resolution_clock::now() - t0).count();
00072 }
```

5.3.1.6 skirstymas_s1_d()

```
double skirstymas_s1_d (
    const std::deque< Studentas > & visi,
    std::deque< Studentas > & kieti,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo `deque_ops.cpp` eilutėje 76.

```
00078 {
00079     auto t0 = std::chrono::high_resolution_clock::now();
00080     kieti.clear(); vargsai.clear();
00081     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00082         [](const Studentas& s) { return s.getRez() >= 5.0; });
00083     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00084         [](const Studentas& s) { return s.getRez() < 5.0; });
00085     return std::chrono::duration<double>(
00086         std::chrono::high_resolution_clock::now() - t0).count();
00087 }
```

5.3.1.7 skirstymas_s2_d()

```
double skirstymas_s2_d (
    std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo [deque_ops.cpp](#) eilutėje 90.

```
00091 {
00092     auto t0 = std::chrono::high_resolution_clock::now();
00093     vargsai.clear();
00094     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00095         [](const Studentas& s) { return s.getRez() < 5.0; });
00096     studentai.erase(
00097         std::remove_if(studentai.begin(), studentai.end(),
00098             [](const Studentas& s) { return s.getRez() < 5.0; }),
00099         studentai.end());
00100     return std::chrono::duration<double>(
00101         std::chrono::high_resolution_clock::now() - t0).count();
00102 }
```

5.3.1.8 skirstymas_s3_d()

```
double skirstymas_s3_d (
    std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsai)
```

Apibrėžimas failo [deque_ops.cpp](#) eilutėje 105.

```
00106 {
00107     auto t0 = std::chrono::high_resolution_clock::now();
00108     vargsai.clear();
00109     auto pivot = std::stable_partition(studentai.begin(), studentai.end(),
00110         [](const Studentas& s) { return s.getRez() < 5.0; });
00111     vargsai.assign(studentai.begin(), pivot);
00112     studentai.erase(studentai.begin(), pivot);
00113     return std::chrono::duration<double>(
00114         std::chrono::high_resolution_clock::now() - t0).count();
00115 }
```

5.4 deque_ops.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef DEQUE_OPS_H
00002 #define DEQUE_OPS_H
00003
00004 #include "studentas.h"
00005 #include <deque>
00006 #include <string>
00007
00008 void skaitymas_is_failo_d(const std::string& filename, std::deque<Studentas>& studentai);
00009 void isvedimas_i_faila_d(const std::deque<Studentas>& studentai,
00010     const std::string& filename,
00011     const std::string& kategorija);
00012 void pasirinkimo_metodas_d(int tipas, std::deque<Studentas>& studentai);
00013 void rusiavimas_d(std::deque<Studentas>& studentai);
00014
00015 // Originali strategija
00016 double skirstymas_i_grupes_d(const std::deque<Studentas>& visi,
00017     std::deque<Studentas>& kieti,
00018     std::deque<Studentas>& vargsai);
00019
00020 // STRATEGIJA 1 du nauji konteineriai, originalas lieka nepakeistas
00021 double skirstymas_sl_d(const std::deque<Studentas>& visi,
00022     std::deque<Studentas>& kieti,
00023     std::deque<Studentas>& vargsai);
00024
00025 // STRATEGIJA 2 vienas naujas konteineris + erase/remove if
00026 double skirstymas_s2_d(std::deque<Studentas>& studentai,
00027     std::deque<Studentas>& vargsai);
00028
00029 // STRATEGIJA 3 std::partition
00030 double skirstymas_s3_d(std::deque<Studentas>& studentai,
00031     std::deque<Studentas>& vargsai);
00032
00033 #endif
```

5.5 generator.cpp Failo Nuoroda

```
#include <fstream>
#include <iostream>
#include <random>
#include <chrono>
#include <iomanip>
#include <string>
```

Funkcijos

- int [generuoti_pazymi\(\)](#)
- void [generuoti_faila](#) (const std::string &filename, int studentu_skaicius, int nd_skaicius=5)
- void [generuoti_testu_failus](#) ()

5.5.1 Funkcijos Dokumentacija

5.5.1.1 generuoti_faila()

```
void generuoti_faila (
    const std::string & filename,
    int studentu_skaicius,
    int nd_skaicius = 5)
```

Apibrėžimas failo [generator.cpp](#) eilutėje 15.

```
00015                                                                 {
00016     auto start = std::chrono::high_resolution_clock::now();
00017
00018     std::ofstream out(filename);
00019     if (!out.is_open()) {
00020         std::cerr << "Nepavyko sukurti failo: " << filename << "\n";
00021         return;
00022     }
00023
00024     out << std::left << std::setw(20) << "Vardas"
00025         << std::setw(20) << "Pavarde";
00026
00027     for (int i = 1; i <= nd_skaicius; i++) {
00028         out << std::setw(5) << ("ND" + std::to_string(i));
00029     }
00030     out << std::setw(5) << "Egz." << "\n";
00031
00032     for (int i = 1; i <= studentu_skaicius; i++) {
00033         out << std::left << std::setw(20) << ("Vardas" + std::to_string(i))
00034             << std::setw(20) << ("Pavarde" + std::to_string(i));
00035
00036         for (int j = 0; j < nd_skaicius; j++) {
00037             out << std::setw(5) << generuoti\_pazymi\(\);
00038         }
00039         out << std::setw(5) << generuoti\_pazymi\(\) << "\n";
00040         if (i % 100000 == 0) {
00041             std::cout << "Sugeneruota: " << i << " studentu...\n";
00042         }
00043     }
00044     out.close();
00045
00046     auto end = std::chrono::high_resolution_clock::now();
00047     std::chrono::duration<double> diff = end - start;
00048
00049     std::cout << "Failas '" << filename << "' sukurtas per "
00050         << std::fixed << std::setprecision(6) << diff.count()
00051         << " sekundziu\n";
00052 }
```

5.5.1.2 generuoti_pazymi()

```
int generuoti_pazymi ()
```

Apibrėžimas failo [generator.cpp](#) eilutėje 8.

```
00008 {
00009     static std::random_device rd;
00010     static std::mt19937 gen(rd());
00011     static std::uniform_int_distribution<> dis(1, 10);
00012     return dis(gen);
00013 }
```

5.5.1.3 generuoti_testu_failus()

```
void generuoti_testu_failus ()
```

Apibrėžimas failo [generator.cpp](#) eilutėje 53.

```
00053 {
00054     std::cout << "\n===== FAILU GENERAVIMAS =====\n\n";
00055
00056     generuoti_faila("studentai1000.txt", 1000);
00057     generuoti_faila("studentai10000.txt", 10000);
00058     generuoti_faila("studentai100000.txt", 100000);
00059     generuoti_faila("studentai1000000.txt", 1000000);
00060     generuoti_faila("studentai10000000.txt", 10000000);
00061
00062     std::cout << "\n===== \n";
00063 }
```

5.6 generator.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include <fstream>
00002 #include <iostream>
00003 #include <random>
00004 #include <chrono>
00005 #include <iomanip>
00006 #include <string>
00007
00008 int generuoti_pazymi() {
00009     static std::random_device rd;
00010     static std::mt19937 gen(rd());
00011     static std::uniform_int_distribution<> dis(1, 10);
00012     return dis(gen);
00013 }
00014
00015 void generuoti_faila(const std::string& filename, int studentu_skaicius, int nd_skaicius = 5) {
00016     auto start = std::chrono::high_resolution_clock::now();
00017
00018     std::ofstream out(filename);
00019     if (!out.is_open()) {
00020         std::cerr << "Nepavyko sukurti failo: " << filename << "\n";
00021         return;
00022     }
00023
00024     out << std::left << std::setw(20) << "Vardas"
00025         << std::setw(20) << "Pavarde";
00026
00027     for (int i = 1; i <= nd_skaicius; i++) {
00028         out << std::setw(5) << ("ND" + std::to_string(i));
00029     }
00030     out << std::setw(5) << "Egz." << "\n";
00031
00032     for (int i = 1; i <= studentu_skaicius; i++) {
00033         out << std::left << std::setw(20) << ("Vardas" + std::to_string(i))
00034             << std::setw(20) << ("Pavarde" + std::to_string(i));
00035
00036         for (int j = 0; j < nd_skaicius; j++) {
00037             out << std::setw(5) << generuoti_pazymi();
00038         }
00039         out << std::setw(5) << generuoti_pazymi() << "\n";
00040     }
00041 }
```

```

00040         if (i % 100000 == 0) {
00041             std::cout << "Sugeneruota: " << i << " studentu...\n";
00042         }
00043     }
00044     out.close();
00045
00046     auto end = std::chrono::high_resolution_clock::now();
00047     std::chrono::duration<double> diff = end - start;
00048
00049     std::cout << "Failas '" << filename << "' sukurtas per "
00050             << std::fixed << std::setprecision(6) << diff.count()
00051             << " sekundziu\n";
00052 }
00053 void generuoti_testu_failus() {
00054     std::cout << "\n===== FAILU GENERAVIMAS =====\n\n";
00055
00056     generuoti_faila("studentai1000.txt", 1000);
00057     generuoti_faila("studentai10000.txt", 10000);
00058     generuoti_faila("studentai100000.txt", 100000);
00059     generuoti_faila("studentai1000000.txt", 1000000);
00060     generuoti_faila("studentai10000000.txt", 10000000);
00061
00062     std::cout << "\n===== \n";
00063 }

```

5.7 list_ops.cpp Failo Nuoroda

```

#include "list_ops.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <iomanip>
#include <stdexcept>
#include <chrono>

```

Funkcijos

- void [skaitymas_is_failo_I](#) (const std::string &filename, std::list< [Studentas](#) > &studentai)
- void [isvedimas_i_faila_I](#) (const std::list< [Studentas](#) > &studentai, const std::string &filename, const std::string &kategorija)
- void [pasirinkimo_metodas_I](#) (int tipas, std::list< [Studentas](#) > &studentai)
- void [rusiavimas_I](#) (std::list< [Studentas](#) > &studentai)
- double [skirstymas_i_grupes_I](#) (const std::list< [Studentas](#) > &visi, std::list< [Studentas](#) > &kieti, std::list< [Studentas](#) > &vargsai)
- double [skirstymas_s1_I](#) (const std::list< [Studentas](#) > &visi, std::list< [Studentas](#) > &kieti, std::list< [Studentas](#) > &vargsai)
- double [skirstymas_s2_I](#) (std::list< [Studentas](#) > &studentai, std::list< [Studentas](#) > &vargsai)
- double [skirstymas_s3_I](#) (std::list< [Studentas](#) > &studentai, std::list< [Studentas](#) > &vargsai)

5.7.1 Funkcijos Dokumentacija

5.7.1.1 isvedimas_i_faila_I()

```

void isvedimas_i_faila_I (
    const std::list< Studentas > & studentai,

```

```
const std::string & filename,
const std::string & kategorija)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 35.

```
00037         {
00038     std::ofstream out(filename);
00039     if (!out.is_open())
00040         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00041     out << std::left << std::setw(20) << "Vardas"
00042         << std::setw(20) << "Pavarde"
00043         << std::right << std::setw(20) << "Galutinis\n"
00044         << std::string(60, '-') << "\n";
00045     for (const auto& s : studentai) out << s << "\n";
00046     std::cout << " [" << kategorija << "]" << studentai.size()
00047         << " studentu -> " << filename << "\n";
00048 }
```

5.7.1.2 pasirinkimo_metodas_l()

```
void pasirinkimo_metodas_l (
    int tipas,
    std::list< Studentas > & studentai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 50.

```
00050                                     {
00051     for (auto& s : studentai) s.skaiciuotiRez(tipas);
00052 }
```

5.7.1.3 rusiavimas_l()

```
void rusiavimas_l (
    std::list< Studentas > & studentai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 55.

```
00055                                     {
00056     studentai.sort([](const Studentas& a, const Studentas& b) {
00057         if (a.getPavarde() != b.getPavarde()) return a.getPavarde() < b.getPavarde();
00058         return a.getVardas() < b.getVardas();
00059     });
00060 }
```

5.7.1.4 skaitymas_is_failo_l()

```
void skaitymas_is_failo_l (
    const std::string & filename,
    std::list< Studentas > & studentai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 10.

```
00010                                     {
00011     std::ifstream file(filename);
00012     if (!file.is_open())
00013         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00014     studentai.clear();
00015     std::string header, line;
00016     std::getline(file, header);
00017     int nr = 0;
00018     while (std::getline(file, line)) {
00019         ++nr;
00020         std::stringstream ss(line);
00021         std::string vardas, pavarde;
00022         ss >> vardas >> pavarde;
00023         Vector<int> paz;
00024         int v;
00025         while (ss >> v) paz.push_back(v);
00026         if (paz.empty()) continue;
00027         int egz = paz.back(); paz.pop_back();
00028         studentai.emplace_back(vardas, pavarde, paz, egz);
00029         if (nr % 1000000 == 0)
00030             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00031     }
00032     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00033 }
```

5.7.1.5 skirstymas_i_grupes_l()

```
double skirstymas_i_grupes_l (
    const std::list< Studentas > & visi,
    std::list< Studentas > & kieti,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 63.

```
00065 {
00066     auto t0 = std::chrono::high_resolution_clock::now();
00067     kieti.clear(); vargsai.clear();
00068     for (const auto& s : visi) {
00069         if (s.getRez() < 5.0) vargsai.push_back(s);
00070         else kieti.push_back(s);
00071     }
00072     return std::chrono::duration<double>{
00073         std::chrono::high_resolution_clock::now() - t0}.count();
00074 }
```

5.7.1.6 skirstymas_s1_l()

```
double skirstymas_s1_l (
    const std::list< Studentas > & visi,
    std::list< Studentas > & kieti,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 77.

```
00079 {
00080     auto t0 = std::chrono::high_resolution_clock::now();
00081     kieti.clear(); vargsai.clear();
00082     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00083         [](const Studentas& s) { return s.getRez() >= 5.0; });
00084     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00085         [](const Studentas& s) { return s.getRez() < 5.0; });
00086     return std::chrono::duration<double>{
00087         std::chrono::high_resolution_clock::now() - t0}.count();
00088 }
```

5.7.1.7 skirstymas_s2_l()

```
double skirstymas_s2_l (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 91.

```
00092 {
00093     auto t0 = std::chrono::high_resolution_clock::now();
00094     vargsai.clear();
00095     for (auto it = studentai.begin(); it != studentai.end(); ) {
00096         if (it->getRez() < 5.0) {
00097             vargsai.push_back(*it);
00098             it = studentai.erase(it);
00099         }
00100         else { ++it; }
00101     }
00102     return std::chrono::duration<double>{
00103         std::chrono::high_resolution_clock::now() - t0}.count();
00104 }
```

5.7.1.8 skirstymas_s3_l()

```
double skirstymas_s3_l (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 107.

```
00108 {
00109     auto t0 = std::chrono::high_resolution_clock::now();
00110     vargsai.clear();
00111     for (auto it = studentai.begin(); it != studentai.end(); ) {
00112         if (it->getRez() < 5.0) {
00113             vargsai.splice(vargsai.end(), studentai, it++);
00114         }
00115         else { ++it; }
00116     }
00117     return std::chrono::duration<double>(
00118         std::chrono::high_resolution_clock::now() - t0).count();
00119 }
```

5.8 list_ops.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "list_ops.h"
00002 #include <iostream>
00003 #include <fstream>
00004 #include <sstream>
00005 #include <algorithm>
00006 #include <iomanip>
00007 #include <stdexcept>
00008 #include <chrono>
00009
00010 void skaitymas_is_failo_l(const std::string& filename, std::list<Studentas>& studentai) {
00011     std::ifstream file(filename);
00012     if (!file.is_open())
00013         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00014     studentai.clear();
00015     std::string header, line;
00016     std::getline(file, header);
00017     int nr = 0;
00018     while (std::getline(file, line)) {
00019         ++nr;
00020         std::stringstream ss(line);
00021         std::string vardas, pavarde;
00022         ss >> vardas >> pavarde;
00023         Vector<int> paz;
00024         int v;
00025         while (ss >> v) paz.push_back(v);
00026         if (paz.empty()) continue;
00027         int egz = paz.back(); paz.pop_back();
00028         studentai.emplace_back(vardas, pavarde, paz, egz);
00029         if (nr % 1000000 == 0)
00030             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00031     }
00032     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00033 }
00034
00035 void isvedimas_i_faila_l(const std::list<Studentas>& studentai,
00036     const std::string& filename,
00037     const std::string& kategorija) {
00038     std::ofstream out(filename);
00039     if (!out.is_open())
00040         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00041     out << std::left << std::setw(20) << "Vardas"
00042         << std::setw(20) << "Pavarde"
00043         << std::right << std::setw(20) << "Galutinis\n"
00044         << std::string(60, '-') << "\n";
00045     for (const auto& s : studentai) out << s << "\n";
00046     std::cout << " [" << kategorija << "]" << " << studentai.size()
00047         << " studentu -> " << filename << "\n";
00048 }
00049
00050 void pasirinkimo_metodas_l(int tipas, std::list<Studentas>& studentai) {
00051     for (auto& s : studentai) s.skaiciuotiRez(tipas);
00052 }
00053
```



```

00054 //list rusiavimas pagal pavarde, vardas
00055 void rusiavimas_l(std::list<Studentas>& studentai) {
00056     studentai.sort([](const Studentas& a, const Studentas& b) {
00057         if (a.getPavarde() != b.getPavarde()) return a.getPavarde() < b.getPavarde();
00058         return a.getVardas() < b.getVardas();
00059     });
00060 }
00061
00062 // Originali strategija - du nauji konteineriai, originalas lieka
00063 double skirstymas_i_grupes_l(const std::list<Studentas>& visi,
00064     std::list<Studentas>& kieti,
00065     std::list<Studentas>& vargsai) {
00066     auto t0 = std::chrono::high_resolution_clock::now();
00067     kieti.clear(); vargsai.clear();
00068     for (const auto& s : visi) {
00069         if (s.getRez() < 5.0) vargsai.push_back(s);
00070         else kieti.push_back(s);
00071     }
00072     return std::chrono::duration<double>{
00073         std::chrono::high_resolution_clock::now() - t0}.count();
00074 }
00075
00076 // S1: du nauji konteineriai su kopijomis (std::copy_if)
00077 double skirstymas_s1_l(const std::list<Studentas>& visi,
00078     std::list<Studentas>& kieti,
00079     std::list<Studentas>& vargsai) {
00080     auto t0 = std::chrono::high_resolution_clock::now();
00081     kieti.clear(); vargsai.clear();
00082     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00083         [](const Studentas& s) { return s.getRez() >= 5.0; });
00084     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00085         [](const Studentas& s) { return s.getRez() < 5.0; });
00086     return std::chrono::duration<double>{
00087         std::chrono::high_resolution_clock::now() - t0}.count();
00088 }
00089
00090 // S2: vienas naujas konteineris + erase kilpa
00091 double skirstymas_s2_l(std::list<Studentas>& studentai,
00092     std::list<Studentas>& vargsai) {
00093     auto t0 = std::chrono::high_resolution_clock::now();
00094     vargsai.clear();
00095     for (auto it = studentai.begin(); it != studentai.end(); ) {
00096         if (it->getRez() < 5.0) {
00097             vargsai.push_back(*it);
00098             it = studentai.erase(it);
00099         }
00100         else { ++it; }
00101     }
00102     return std::chrono::duration<double>{
00103         std::chrono::high_resolution_clock::now() - t0}.count();
00104 }
00105
00106 // S3: splice - nulines kopijos, O(n)
00107 double skirstymas_s3_l(std::list<Studentas>& studentai,
00108     std::list<Studentas>& vargsai) {
00109     auto t0 = std::chrono::high_resolution_clock::now();
00110     vargsai.clear();
00111     for (auto it = studentai.begin(); it != studentai.end(); ) {
00112         if (it->getRez() < 5.0) {
00113             vargsai.splice(vargsai.end(), studentai, it++);
00114         }
00115         else { ++it; }
00116     }
00117     return std::chrono::duration<double>{
00118         std::chrono::high_resolution_clock::now() - t0}.count();
00119 }

```

5.9 list_ops.h Failo Nuoroda

```

#include "studentas.h"
#include <list>
#include <string>

```

Funkcijos

- void [skaitymas_is_failo_l](#) (const std::string &filename, std::list< [Studentas](#) > &studentai)

- void `isvedimas_i_faila_l` (const std::list< `Studentas` > &studentai, const std::string &filename, const std::string &kategorija)
- void `pasirinkimo_metodas_l` (int tipas, std::list< `Studentas` > &studentai)
- void `rusiavimas_l` (std::list< `Studentas` > &studentai)
- double `skirstymas_i_grupes_l` (const std::list< `Studentas` > &visi, std::list< `Studentas` > &kieti, std::list< `Studentas` > &vargsai)
- double `skirstymas_s1_l` (const std::list< `Studentas` > &visi, std::list< `Studentas` > &kieti, std::list< `Studentas` > &vargsai)
- double `skirstymas_s2_l` (std::list< `Studentas` > &studentai, std::list< `Studentas` > &vargsai)
- double `skirstymas_s3_l` (std::list< `Studentas` > &studentai, std::list< `Studentas` > &vargsai)

5.9.1 Funkcijos Dokumentacija

5.9.1.1 isvedimas_i_faila_l()

```
void isvedimas_i_faila_l (
    const std::list< Studentas > & studentai,
    const std::string & filename,
    const std::string & kategorija)
```

Apibrėžimas failo `list_ops.cpp` eilutėje 35.

```
00037     {
00038         std::ofstream out(filename);
00039         if (!out.is_open())
00040             throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00041         out << std::left << std::setw(20) << "Vardas"
00042             << std::setw(20) << "Pavarde"
00043             << std::right << std::setw(20) << "Galutinis\n"
00044             << std::string(60, '-') << "\n";
00045         for (const auto& s : studentai) out << s << "\n";
00046         std::cout << " [" << kategorija << "]" << studentai.size()
00047             << " studentu -> " << filename << "\n";
00048     }
```

5.9.1.2 pasirinkimo_metodas_l()

```
void pasirinkimo_metodas_l (
    int tipas,
    std::list< Studentas > & studentai)
```

Apibrėžimas failo `list_ops.cpp` eilutėje 50.

```
00050     {
00051         for (auto& s : studentai) s.skaiciuotiRez(tipas);
00052     }
```

5.9.1.3 rusiavimas_l()

```
void rusiavimas_l (
    std::list< Studentas > & studentai)
```

Apibrėžimas failo `list_ops.cpp` eilutėje 55.

```
00055     {
00056         studentai.sort([](const Studentas& a, const Studentas& b) {
00057             if (a.getPavarde() != b.getPavarde()) return a.getPavarde() < b.getPavarde();
00058             return a.getVardas() < b.getVardas();
00059         });
00060     }
```

5.9.1.4 skaitymas_is_failo_l()

```
void skaitymas_is_failo_l (
    const std::string & filename,
    std::list< Studentas > & studentai)
```

Apibrėžimas failo `list_ops.cpp` eilutėje 10.

```
00010                                     {
00011     std::ifstream file(filename);
00012     if (!file.is_open())
00013         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00014     studentai.clear();
00015     std::string header, line;
00016     std::getline(file, header);
00017     int nr = 0;
00018     while (std::getline(file, line)) {
00019         ++nr;
00020         std::stringstream ss(line);
00021         std::string vardas, pavarde;
00022         ss >> vardas >> pavarde;
00023         Vector<int> paz;
00024         int v;
00025         while (ss >> v) paz.push_back(v);
00026         if (paz.empty()) continue;
00027         int egz = paz.back(); paz.pop_back();
00028         studentai.emplace_back(vardas, pavarde, paz, egz);
00029         if (nr % 1000000 == 0)
00030             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00031     }
00032     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00033 }
```

5.9.1.5 skirstymas_i_grupes_l()

```
double skirstymas_i_grupes_l (
    const std::list< Studentas > & visi,
    std::list< Studentas > & kieti,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo `list_ops.cpp` eilutėje 63.

```
00065                                     {
00066     auto t0 = std::chrono::high_resolution_clock::now();
00067     kieti.clear(); vargsai.clear();
00068     for (const auto& s : visi) {
00069         if (s.getRez() < 5.0) vargsai.push_back(s);
00070         else kieti.push_back(s);
00071     }
00072     return std::chrono::duration<double>(
00073         std::chrono::high_resolution_clock::now() - t0).count();
00074 }
```

5.9.1.6 skirstymas_s1_l()

```
double skirstymas_s1_l (
    const std::list< Studentas > & visi,
    std::list< Studentas > & kieti,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo `list_ops.cpp` eilutėje 77.

```
00079                                     {
00080     auto t0 = std::chrono::high_resolution_clock::now();
00081     kieti.clear(); vargsai.clear();
00082     std::copy_if(visi.begin(), visi.end(), std::back_inserter(kieti),
00083         [](const Studentas& s) { return s.getRez() >= 5.0; });
00084     std::copy_if(visi.begin(), visi.end(), std::back_inserter(vargsai),
00085         [](const Studentas& s) { return s.getRez() < 5.0; });
00086     return std::chrono::duration<double>(
00087         std::chrono::high_resolution_clock::now() - t0).count();
00088 }
```

5.9.1.7 skirstymas_s2_l()

```
double skirstymas_s2_l (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 91.

```
00092         {
00093             auto t0 = std::chrono::high_resolution_clock::now();
00094             vargsai.clear();
00095             for (auto it = studentai.begin(); it != studentai.end(); ) {
00096                 if (it->getRez() < 5.0) {
00097                     vargsai.push_back(*it);
00098                     it = studentai.erase(it);
00099                 }
00100                 else { ++it; }
00101             }
00102             return std::chrono::duration<double>(
00103                 std::chrono::high_resolution_clock::now() - t0).count();
00104         }
```

5.9.1.8 skirstymas_s3_l()

```
double skirstymas_s3_l (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsai)
```

Apibrėžimas failo [list_ops.cpp](#) eilutėje 107.

```
00108         {
00109             auto t0 = std::chrono::high_resolution_clock::now();
00110             vargsai.clear();
00111             for (auto it = studentai.begin(); it != studentai.end(); ) {
00112                 if (it->getRez() < 5.0) {
00113                     vargsai.splice(vargsai.end(), studentai, it++);
00114                 }
00115                 else { ++it; }
00116             }
00117             return std::chrono::duration<double>(
00118                 std::chrono::high_resolution_clock::now() - t0).count();
00119         }
```

5.10 list_ops.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef LIST_OPS_H
00002 #define LIST_OPS_H
00003
00004 #include "studentas.h"
00005 #include <list>
00006 #include <string>
00007
00008 void skaitymas_is_failo_l(const std::string& filename, std::list<Studentas>& studentai);
00009 void isvedimas_i_faila_l(const std::list<Studentas>& studentai,
00010     const std::string& filename,
00011     const std::string& kategorija);
00012 void pasirinkimo_metodas_l(int tipas, std::list<Studentas>& studentai);
00013 void rusiavimas_l(std::list<Studentas>& studentai);
00014
00015 // Originali strategija
00016 double skirstymas_i_grupes_l(const std::list<Studentas>& visi,
00017     std::list<Studentas>& kieti,
00018     std::list<Studentas>& vargsai);
00019
00020 // STRATEGIJA 1 du nauji konteineriai, originalas lieka nepakeistas
00021 double skirstymas_sl_l(const std::list<Studentas>& visi,
00022     std::list<Studentas>& kieti,
00023     std::list<Studentas>& vargsai);
00024
00025 // STRATEGIJA 2 vienas naujas konteineris + erase/remove if
```

```

00026 double skirstymas_s2_1(std::list<Studentas>& studentai,
00027     std::list<Studentas>& vargsai);
00028
00029 // STRATEGIJA 3 splice - nulines kopijos
00030 double skirstymas_s3_1(std::list<Studentas>& studentai,
00031     std::list<Studentas>& vargsai);
00032
00033 #endif

```

5.11 main.cpp Failo Nuoroda

```

#include <iostream>
#include <limits>
#include <string>
#include <list>
#include <deque>
#include <chrono>
#include "studentas.h"
#include "vector_ops.h"
#include "list_ops.h"
#include "deque_ops.h"
#include "utils.h"
#include "vector.h"

```

Funkcijos

- void [generuoti_testu_failus](#) ()
- void [generuoti_faila](#) (const std::string &, int, int)
- void [atlikti_visus_testus](#) ()
- void [atlikti_klases_testus](#) ()
- static void [skaityti_vector](#) ()
- static void [skaityti_list](#) ()
- static void [skaityti_deque](#) ()
- static void [rankinis_ivedimas](#) ()
- int [main](#) ()

5.11.1 Funkcijos Dokumentacija

5.11.1.1 atlikti_klases_testus()

```
void atlikti_klases_testus ()
```

Apibrėžimas failo [testavimas_klases.cpp](#) eilutėje 330.

```

00330 {
00331     std::cout << "\n";
00332     std::cout << std::string(55, '=') << "\n";
00333     std::cout << "    STUDENTAS KLASĖS TESTAVIMAS    (v1.2)\n";
00334     std::cout << std::string(55, '=') << "\n";
00335
00336     testuoti_konstruktorius();
00337     testuoti_priskyrima();
00338     testuoti_palyginimo_operatorius();
00339     testuoti_srautu_operatorius();
00340     testuoti_metodus();
00341     testuoti_su_stl();
00342     testuoti_zmogus_abstrakti();

```

```

00343
00344     std::cout << "\n" << std::string(55, '=') << "\n";
00345     std::cout << " Rezultatas: " << testai_pavyke << " / "
00346         << testai_is_viso << " testu pavyko\n";
00347     if (testai_pavyke == testai_is_viso)
00348         std::cout << " Visi testai pavyko!\n";
00349     else
00350         std::cout << " DEMESIO: "
00351             << (testai_is_viso - testai_pavyke)
00352             << " testai nepavyko!\n";
00353     std::cout << std::string(55, '=') << "\n";
00354 }

```

5.11.1.2 atlikti_visus_testus()

```
void atlikti_visus_testus ()
```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 136.

```

00136     {
00137         const std::vector<std::string> failai = {
00138             "studentai000.txt",
00139             "studentai10000.txt",
00140             "studentai100000.txt",
00141             "studentai1000000.txt",
00142             "studentai10000000.txt"
00143         };
00144
00145         std::vector<TestRow> rezultatai;
00146         std::cout << "\n##### PRADEDAMAS TESTAVIMAS #####\n\n";
00147
00148         for (const auto& f : failai) {
00149             for (int s = 1; s <= 3; ++s) {
00150                 std::cout << "> vector S" << s << " | " << f << "\n";
00151                 try { rezultatai.push_back(testuoti_vector(f, s)); }
00152                 catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00153
00154                 std::cout << "> list S" << s << " | " << f << "\n";
00155                 try { rezultatai.push_back(testuoti_list(f, s)); }
00156                 catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00157
00158                 std::cout << "> deque S" << s << " | " << f << "\n";
00159                 try { rezultatai.push_back(testuoti_deque(f, s)); }
00160                 catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00161             }
00162         }
00163
00164         std::cout << "\n##### TESTAVIMAS BAIGTAS #####\n";
00165         spausdinti_lentele(rezultatai);
00166     }

```

5.11.1.3 generuoti_faila()

```

void generuoti_faila (
    const std::string & filename,
    int studentu_skaicius,
    int nd_skaicius)

```

Apibrėžimas failo [generator.cpp](#) eilutėje 15.

```

00015
00016     auto start = std::chrono::high_resolution_clock::now();
00017
00018     std::ofstream out(filename);
00019     if (!out.is_open()) {
00020         std::cerr << "Nepavyko sukurti failo: " << filename << "\n";
00021         return;
00022     }
00023
00024     out << std::left << std::setw(20) << "Vardas"
00025         << std::setw(20) << "Pavarde";
00026
00027     for (int i = 1; i <= nd_skaicius; i++) {
00028         out << std::setw(5) << ("ND" + std::to_string(i));
00029     }

```

```

00030     out << std::setw(5) << "Egz." << "\n";
00031
00032     for (int i = 1; i <= studentu_skaicius; i++) {
00033         out << std::left << std::setw(20) << ("Vardas" + std::to_string(i))
00034             << std::setw(20) << ("Pavarde" + std::to_string(i));
00035
00036         for (int j = 0; j < nd_skaicius; j++) {
00037             out << std::setw(5) << generuoti_pazymi();
00038         }
00039         out << std::setw(5) << generuoti_pazymi() << "\n";
00040         if (i % 100000 == 0) {
00041             std::cout << "Sugeneruota: " << i << " studentu...\n";
00042         }
00043     }
00044     out.close();
00045
00046     auto end = std::chrono::high_resolution_clock::now();
00047     std::chrono::duration<double> diff = end - start;
00048
00049     std::cout << "Failas '" << filename << "' sukurtas per "
00050         << std::fixed << std::setprecision(6) << diff.count()
00051         << " sekundziu\n";
00052 }

```

5.11.1.4 generuoti_testu_failus()

```
void generuoti_testu_failus ()
```

Apibrėžimas failo [generator.cpp](#) eilutėje 53.

```

00053     {
00054         std::cout << "\n===== FAILU GENERAVIMAS =====\n\n";
00055
00056         generuoti_faila("studentail000.txt", 1000);
00057         generuoti_faila("studentail0000.txt", 10000);
00058         generuoti_faila("studentail00000.txt", 100000);
00059         generuoti_faila("studentail000000.txt", 1000000);
00060         generuoti_faila("studentail0000000.txt", 10000000);
00061
00062         std::cout << "\n===== \n";
00063     }

```

5.11.1.5 main()

```
int main ()
```

Apibrėžimas failo [main.cpp](#) eilutėje 174.

```

00174     {
00175         while (true) {
00176             std::cout << "\n===== STUDENTU PROGRAMA v3.0 (Vector<T>) =====\n"
00177                 << "1 - Rankinis ivedimas\n"
00178                 << "2 - Skaityti is failo (Vector)\n"
00179                 << "3 - Skaityti is failo (list)\n"
00180                 << "4 - Skaityti is failo (deque)\n"
00181                 << "5 - Testavimas (visi konteineriai)\n"
00182                 << "6 - Generuoti studentu failus\n"
00183                 << "7 - Testavimas (klases)\n"
00184                 << "8 - Baigti\n"
00185                 << "Pasirinkimas: ";
00186
00187             int p;
00188             std::cin >> p;
00189             if (std::cin.fail()) {
00190                 std::cin.clear();
00191                 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00192                 continue;
00193             }
00194
00195             switch (p) {
00196                 case 1: rankinis_ivedimas(); break;
00197                 case 2: skaityti_vector(); break;
00198                 case 3: skaityti_list(); break;
00199                 case 4: skaityti_deque(); break;
00200                 case 5: atlikti_visus_testus(); break;
00201                 case 6: {
00202                     std::cout << "1 - Visus testu failus (1k-10M)\n"

```

```

00203         "2 - Viena faila\nPasirinkimas: ";
00204     int sub; std::cin >> sub;
00205     if (sub == 1) {
00206         generuoti_testu_failus();
00207     }
00208     else {
00209         int n; std::string fn;
00210         std::cout << "Kiek studentu? "; std::cin >> n;
00211         std::cout << "Failo pavadinimas: "; std::cin >> fn;
00212         generuoti_faila(fn, n, 5);
00213     }
00214     break;
00215 }
00216 case 7: atlikti_klases_testus(); break;
00217 case 8: std::cout << "Programa baigta.\n"; return 0;
00218 default: std::cerr << "Tokio pasirinkimo nera.\n";
00219 }
00220 }
00221 }

```

5.11.1.6 rankinis_ivedimas()

```
void rankinis_ivedimas () [static]
```

Apibrėžimas failo `main.cpp` eilutėje 147.

```

00147     {
00148         Vector<Studentas> studentai;
00149         int chosen = 1;
00150         do {
00151             std::string vardas = iversti_varda_ar_pavarde("Vardas: ");
00152             std::string pavarde = iversti_varda_ar_pavarde("Pavarde: ");
00153             Vector<int> paz; // laikinas Vector pazymiu nuskaitymui
00154             std::cout << "Pazymiai (0-10, -1 baigti):\n";
00155             while (true) {
00156                 int p = iversties_tikrinimas("Pazymys: ");
00157                 if (p == -1) break;
00158                 paz.push_back(p);
00159             }
00160             int egz = iversties_tikrinimas("Egzamino pazymys: ");
00161             studentai.push_back(Studentas(vardas, pavarde, paz, egz));
00162             std::cout << "Dar vienas? (1-taip, 0-ne): "; std::cin >> chosen;
00163         } while (chosen == 1);
00164
00165         if (!studentai.empty()) {
00166             int t = skaiciavimo_metodas();
00167             pasirinkimo_metodas(t, studentai);
00168             isvedimas(studentai, t);
00169         }
00170     }

```

5.11.1.7 skaityti_deque()

```
void skaityti_deque () [static]
```

Apibrėžimas failo `main.cpp` eilutėje 106.

```

00106     {
00107         std::string fn;
00108         std::cout << "Failo pavadinimas: "; std::cin >> fn;
00109
00110         std::deque<Studentas> studentai;
00111         try {
00112             auto t0 = std::chrono::high_resolution_clock::now();
00113             skaitymas_is_failo_d(fn, studentai);
00114             double dt = std::chrono::duration<double>(
00115                 std::chrono::high_resolution_clock::now() - t0).count();
00116             if (studentai.empty()) throw std::runtime_error("Failas tuscias");
00117             std::cout << "Skaitymas: " << dt << " s\n";
00118         }
00119         catch (const std::exception& e) { std::cerr << e.what() << "\n"; return; }
00120
00121         pasirinkimo_metodas_d(1, studentai);
00122
00123         auto t0 = std::chrono::high_resolution_clock::now();
00124         rusiavimas_d(studentai);
00125         std::cout << "Rusiavimas: "

```



```

00126         « std::chrono::duration<double>{
00127             std::chrono::high_resolution_clock::now() - t0}.count()
00128         « " s\n";
00129
00130     std::deque<Studentas> kieti, vargsai;
00131     t0 = std::chrono::high_resolution_clock::now();
00132     skirstymas_i_grupes_d(studentai, kieti, vargsai);
00133     std::cout « "Skirstymas: "
00134         « std::chrono::duration<double>{
00135             std::chrono::high_resolution_clock::now() - t0}.count()
00136         « " s\n";
00137     std::cout « "Kietiakai: " « kieti.size()
00138         « " | Vargsai: " « vargsai.size() « "\n";
00139
00140     std::string base = fn.substr(0, fn.find_last_of('.'));
00141     isvedimas_i_faila_d(kieti, base + "_kietiakai.txt", "Kietiakai");
00142     isvedimas_i_faila_d(vargsai, base + "_vargsiukai.txt", "Vargsiukai");
00143 }

```

5.11.1.8 skaityti_list()

```
void skaityti_list () [static]
```

Apibrėžimas failo [main.cpp](#) eilutėje 65.

```

00065     {
00066         std::string fn;
00067         std::cout « "Failo pavadinimas: "; std::cin » fn;
00068
00069         std::list<Studentas> studentai;
00070         try {
00071             auto t0 = std::chrono::high_resolution_clock::now();
00072             skaitymas_is_failo_l(fn, studentai);
00073             double dt = std::chrono::duration<double>{
00074                 std::chrono::high_resolution_clock::now() - t0}.count();
00075             if (studentai.empty()) throw std::runtime_error("Failas tuscias");
00076             std::cout « "Skaitymas: " « dt « " s\n";
00077         }
00078         catch (const std::exception& e) { std::cerr « e.what() « "\n"; return; }
00079
00080         pasirinkimo_metodas_l(1, studentai);
00081
00082         auto t0 = std::chrono::high_resolution_clock::now();
00083         rusiavimas_l(studentai);
00084         std::cout « "Rusiavimas: "
00085             « std::chrono::duration<double>{
00086                 std::chrono::high_resolution_clock::now() - t0}.count()
00087             « " s\n";
00088
00089         std::list<Studentas> kieti, vargsai;
00090         t0 = std::chrono::high_resolution_clock::now();
00091         skirstymas_i_grupes_l(studentai, kieti, vargsai);
00092         std::cout « "Skirstymas: "
00093             « std::chrono::duration<double>{
00094                 std::chrono::high_resolution_clock::now() - t0}.count()
00095             « " s\n";
00096         std::cout « "Kietiakai: " « kieti.size()
00097             « " | Vargsai: " « vargsai.size() « "\n";
00098
00099         std::string base = fn.substr(0, fn.find_last_of('.'));
00100         isvedimas_i_faila_l(kieti, base + "_kietiakai.txt", "Kietiakai");
00101         isvedimas_i_faila_l(vargsai, base + "_vargsiukai.txt", "Vargsiukai");
00102     }

```

5.11.1.9 skaityti_vector()

```
void skaityti_vector () [static]
```

Apibrėžimas failo [main.cpp](#) eilutėje 22.

```

00022     {
00023         std::string fn;
00024         std::cout « "Failo pavadinimas: "; std::cin » fn;
00025
00026         Vector<Studentas> studentai;
00027         try {
00028             auto t0 = std::chrono::high_resolution_clock::now();

```

```

00029         skaitymas_is_failo(fn, studentai);
00030         double dt = std::chrono::duration<double>(
00031             std::chrono::high_resolution_clock::now() - t0).count();
00032         if (studentai.empty()) throw std::runtime_error("Failas tuscias");
00033         std::cout << "Skaitymas: " << dt << " s\n";
00034     }
00035     catch (const std::exception& e) { std::cerr << e.what() << "\n"; return; }
00036
00037     int met = skaiciavimo_metodas();
00038     pasirinkimo_metodas(met, studentai);
00039
00040     int rus = pasirinkimas_rusiavimo_budo();
00041     auto t0 = std::chrono::high_resolution_clock::now();
00042     rusiavimas(studentai, rus);
00043     std::cout << "Rusiavimas: "
00044         << std::chrono::duration<double>(
00045             std::chrono::high_resolution_clock::now() - t0).count()
00046         << " s\n";
00047
00048     Vector<Studentas> kieti, vargsai;
00049     t0 = std::chrono::high_resolution_clock::now();
00050     skirstymas_i_grupes(studentai, kieti, vargsai);
00051     std::cout << "Skirstymas: "
00052         << std::chrono::duration<double>(
00053             std::chrono::high_resolution_clock::now() - t0).count()
00054         << " s\n";
00055     std::cout << "Kietiakai: " << kieti.size()
00056         << " | Vargsai: " << vargsai.size() << "\n";
00057
00058     std::string base = fn.substr(0, fn.find_last_of('.'));
00059     isvedimas_i_faila(kieti, base + "_kietiakai.txt", "Kietiakai");
00060     isvedimas_i_faila(vargsai, base + "_vargsiukai.txt", "Vargsiukai");
00061 }

```

5.12 main.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include <iostream>
00002 #include <limits>
00003 #include <string>
00004 #include <list>
00005 #include <deque>
00006 #include <chrono>
00007
00008 #include "studentas.h"
00009 #include "vector_ops.h"
00010 #include "list_ops.h"
00011 #include "deque_ops.h"
00012 #include "utils.h"
00013 #include "vector.h"
00014
00015 void generuoti_testu_failus();
00016 void generuoti_faila(const std::string&, int, int);
00017 void atlikti_visus_testus();
00018 void atlikti_klases_testus();
00019
00020 // Vector
00021
00022 static void skaityti_vector() {
00023     std::string fn;
00024     std::cout << "Failo pavadinimas: "; std::cin >> fn;
00025
00026     Vector<Studentas> studentai;
00027     try {
00028         auto t0 = std::chrono::high_resolution_clock::now();
00029         skaitymas_is_failo(fn, studentai);
00030         double dt = std::chrono::duration<double>(
00031             std::chrono::high_resolution_clock::now() - t0).count();
00032         if (studentai.empty()) throw std::runtime_error("Failas tuscias");
00033         std::cout << "Skaitymas: " << dt << " s\n";
00034     }
00035     catch (const std::exception& e) { std::cerr << e.what() << "\n"; return; }
00036
00037     int met = skaiciavimo_metodas();
00038     pasirinkimo_metodas(met, studentai);
00039
00040     int rus = pasirinkimas_rusiavimo_budo();
00041     auto t0 = std::chrono::high_resolution_clock::now();
00042     rusiavimas(studentai, rus);
00043     std::cout << "Rusiavimas: "
00044         << std::chrono::duration<double>(

```

```

00045         std::chrono::high_resolution_clock::now() - t0).count()
00046         « " s\n";
00047
00048     Vector<Studentas> kieti, vargsai;
00049     t0 = std::chrono::high_resolution_clock::now();
00050     skirstymas_i_grupes(studentai, kieti, vargsai);
00051     std::cout « "Skirstymas: "
00052             « std::chrono::duration<double>(
00053                 std::chrono::high_resolution_clock::now() - t0).count()
00054             « " s\n";
00055     std::cout « "Kietiakai: " « kieti.size()
00056             « " | Vargsai: " « vargsai.size() « "\n";
00057
00058     std::string base = fn.substr(0, fn.find_last_of('.'));
00059     isvedimas_i_faila(kieti, base + "_kietiakai.txt", "Kietiakai");
00060     isvedimas_i_faila(vargsai, base + "_vargsiukai.txt", "Vargsiukai");
00061 }
00062
00063 // List
00064
00065 static void skaityti_list() {
00066     std::string fn;
00067     std::cout « "Failo pavadinimas: "; std::cin » fn;
00068
00069     std::list<Studentas> studentai;
00070     try {
00071         auto t0 = std::chrono::high_resolution_clock::now();
00072         skaitymas_is_failo_l(fn, studentai);
00073         double dt = std::chrono::duration<double>(
00074             std::chrono::high_resolution_clock::now() - t0).count();
00075         if (studentai.empty()) throw std::runtime_error("Failas tuscias");
00076         std::cout « "Skaitymas: " « dt « " s\n";
00077     }
00078     catch (const std::exception& e) { std::cerr « e.what() « "\n"; return; }
00079
00080     pasirinkimo_metodas_l(1, studentai);
00081
00082     auto t0 = std::chrono::high_resolution_clock::now();
00083     rusiavimas_l(studentai);
00084     std::cout « "Rusiavimas: "
00085             « std::chrono::duration<double>(
00086                 std::chrono::high_resolution_clock::now() - t0).count()
00087             « " s\n";
00088
00089     std::list<Studentas> kieti, vargsai;
00090     t0 = std::chrono::high_resolution_clock::now();
00091     skirstymas_i_grupes_l(studentai, kieti, vargsai);
00092     std::cout « "Skirstymas: "
00093             « std::chrono::duration<double>(
00094                 std::chrono::high_resolution_clock::now() - t0).count()
00095             « " s\n";
00096     std::cout « "Kietiakai: " « kieti.size()
00097             « " | Vargsai: " « vargsai.size() « "\n";
00098
00099     std::string base = fn.substr(0, fn.find_last_of('.'));
00100     isvedimas_i_faila_l(kieti, base + "_kietiakai.txt", "Kietiakai");
00101     isvedimas_i_faila_l(vargsai, base + "_vargsiukai.txt", "Vargsiukai");
00102 }
00103
00104 // Deque
00105
00106 static void skaityti_deque() {
00107     std::string fn;
00108     std::cout « "Failo pavadinimas: "; std::cin » fn;
00109
00110     std::deque<Studentas> studentai;
00111     try {
00112         auto t0 = std::chrono::high_resolution_clock::now();
00113         skaitymas_is_failo_d(fn, studentai);
00114         double dt = std::chrono::duration<double>(
00115             std::chrono::high_resolution_clock::now() - t0).count();
00116         if (studentai.empty()) throw std::runtime_error("Failas tuscias");
00117         std::cout « "Skaitymas: " « dt « " s\n";
00118     }
00119     catch (const std::exception& e) { std::cerr « e.what() « "\n"; return; }
00120
00121     pasirinkimo_metodas_d(1, studentai);
00122
00123     auto t0 = std::chrono::high_resolution_clock::now();
00124     rusiavimas_d(studentai);
00125     std::cout « "Rusiavimas: "
00126             « std::chrono::duration<double>(
00127                 std::chrono::high_resolution_clock::now() - t0).count()
00128             « " s\n";
00129
00130     std::deque<Studentas> kieti, vargsai;
00131     t0 = std::chrono::high_resolution_clock::now();

```

```

00132     skirstymas_i_grupes_d(studentai, kieti, vargsai);
00133     std::cout << "Skirstymas: "
00134         << std::chrono::duration<double>{
00135             std::chrono::high_resolution_clock::now() - t0}.count()
00136         << " s\n";
00137     std::cout << "Kietiakai: " << kieti.size()
00138         << " | Vargsai: " << vargsai.size() << "\n";
00139
00140     std::string base = fn.substr(0, fn.find_last_of('.'));
00141     isvedimas_i_faila_d(kieti, base + "_kietiakai.txt", "Kietiakai");
00142     isvedimas_i_faila_d(vargsai, base + "_vargsiukai.txt", "Vargsiukai");
00143 }
00144
00145 // Rankinis ivedimas
00146
00147 static void rankinis_ivedimas() {
00148     Vector<Studentas> studentai;
00149     int chosen = 1;
00150     do {
00151         std::string vardas = ivesiti_varda_ar_pavarde("Vardas: ");
00152         std::string pavarde = ivesiti_varda_ar_pavarde("Pavarde: ");
00153         Vector<int> paz; // laikinas Vector pazymiu nuskaitymui
00154         std::cout << "Pazymiai (0-10, -1 baigti):\n";
00155         while (true) {
00156             int p = ivesities_tikrinimas("Pazymys: ");
00157             if (p == -1) break;
00158             paz.push_back(p);
00159         }
00160         int egz = ivesities_tikrinimas("Egzamino pazymys: ");
00161         studentai.push_back(Studentas(vardas, pavarde, paz, egz));
00162         std::cout << "Dar vienas? (1-taip, 0-ne): "; std::cin >> chosen;
00163     } while (chosen == 1);
00164
00165     if (!studentai.empty()) {
00166         int t = skaiciavimo_metodas();
00167         pasirinkimo_metodas(t, studentai);
00168         isvedimas(studentai, t);
00169     }
00170 }
00171
00172 // main
00173
00174 int main() {
00175     while (true) {
00176         std::cout << "\n===== STUDENTU PROGRAMA v3.0 (Vector<T>) =====\n"
00177             << "1 - Rankinis ivedimas\n"
00178             << "2 - Skaityti is failo (Vector)\n"
00179             << "3 - Skaityti is failo (list)\n"
00180             << "4 - Skaityti is failo (deque)\n"
00181             << "5 - Testavimas (visi konteineriai)\n"
00182             << "6 - Generuoti studentu failus\n"
00183             << "7 - Testavimas (klases)\n"
00184             << "8 - Baigti\n"
00185             << "Pasirinkimas: ";
00186
00187         int p;
00188         std::cin >> p;
00189         if (std::cin.fail()) {
00190             std::cin.clear();
00191             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00192             continue;
00193         }
00194
00195         switch (p) {
00196             case 1: rankinis_ivedimas(); break;
00197             case 2: skaityti_vector(); break;
00198             case 3: skaityti_list(); break;
00199             case 4: skaityti_deque(); break;
00200             case 5: atlikti_visus_testus(); break;
00201             case 6: {
00202                 std::cout << "1 - Visus testu failus (1k-10M)\n"
00203                     << "2 - Viena faila\nPasirinkimas: ";
00204                 int sub; std::cin >> sub;
00205                 if (sub == 1) {
00206                     generuoti_testu_failus();
00207                 }
00208                 else {
00209                     int n; std::string fn;
00210                     std::cout << "Kiek studentu? "; std::cin >> n;
00211                     std::cout << "Failo pavadinimas: "; std::cin >> fn;
00212                     generuoti_faila(fn, n, 5);
00213                 }
00214                 break;
00215             }
00216             case 7: atlikti_klases_testus(); break;
00217             case 8: std::cout << "Programa baigta.\n"; return 0;
00218             default: std::cerr << "Tokio pasirinkimo nera.\n";

```

```
00219     }
00220     }
00221 }
```

5.13 studentas.cpp Failo Nuoroda

```
#include "studentas.h"
#include <algorithm>
#include <numeric>
#include <stdexcept>
#include <sstream>
#include <iomanip>
#include "vector.h"
```

Funkcijos

- `std::ostream & operator<< (std::ostream &os, const Studentas &s)`
- `std::istream & operator>> (std::istream &is, Studentas &s)`

5.13.1 Funkcijos Dokumentacija

5.13.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s)
```

Parametrai

<i>os</i>	Išvesties srautas
<i>s</i>	Studentas objektas

Gražina

Nuoroda i išvesties srauta @detales Formatas: "Vardas Pavarde [pazymiai] Egz: E Rez: R"

Apibrėžimas failo [studentas.cpp](#) eilutėje 92.

```
00092     {
00093         os << std::left << std::setw(15) << s.vardas_
00094             << std::setw(15) << s.pavarde_
00095             << "[";
00096         for (size_t i = 0; i < s.paz_.size(); ++i) {
00097             os << s.paz_[i];
00098             if (i + 1 < s.paz_.size()) os << " ";
00099         }
00100         os << "]" Egz: " << s.egz_
00101             << std::right << " Rez: "
00102             << std::fixed << std::setprecision(2) << s.rez_;
00103         return os;
00104     }
```

5.13.1.2 operator>>()

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s)
```

Parametrai

<i>is</i>	Ivesties srautas
-----------	------------------

s	Studentas objektas
---	--------------------

Gražina

Nuoroda i ivesties srauta @detales Skaito: "Vardas Pavarde paz1 paz2 ... pazN egzaminas"

Apibrėžimas failo `studentas.cpp` eilutėje 107.

```

00107                                     {
00108         is » s.vardas_ » s.pavarde_;
00109         std::string line;
00110         std::getline(is, line);
00111         std::istringstream ss(line);
00112         s.paz_.clear();
00113         int v;
00114         while (ss » v) s.paz_.push_back(v);
00115         if (!s.paz_.empty()) {
00116             s.egz_ = s.paz_.back();
00117             s.paz_.pop_back();
00118         }
00119         else {
00120             s.egz_ = 0;
00121         }
00122         s.rez_ = 0.0;
00123         return is;
00124     }

```

5.14 studentas.cpp

Eiti Å – Åjio failo dokumentacijÅ.

```

00001 #include "studentas.h"
00002 #include <algorithm>
00003 #include <numeric>
00004 #include <stdexcept>
00005 #include <sstream>
00006 #include <iomanip>
00007 #include "vector.h"
00008
00009 // Numatytasis konstruktorius
00010 Studentas::Studentas()
00011     : Zmogus(), paz_(), egz_(0), rez_(0.0) {
00012 }
00013
00014 // Parametrinis konstruktorius
00015 Studentas::Studentas(const std::string& vardas,
00016     const std::string& pavarde,
00017     const Vector<int>& paz,
00018     int egz)
00019     : Zmogus(vardas, pavarde), paz_(paz.begin(), paz.end()), egz_(egz), rez_(0.0) {
00020 }
00021
00022 // Kopijavimo konstruktorius
00023 Studentas::Studentas(const Studentas& other)
00024     : Zmogus(other), // kopijuoja vardas_, pavarde_
00025     paz_(other.paz_),
00026     egz_(other.egz_),
00027     rez_(other.rez_) {
00028 }
00029
00030 // Perkelimo konstruktorius
00031 Studentas::Studentas(Studentas&& other) noexcept
00032     : Zmogus(std::move(other)), // perkelia vardas_, pavarde_
00033     paz_(std::move(other.paz_)),
00034     egz_(other.egz_),
00035     rez_(other.rez_) {
00036     other.egz_ = 0;
00037     other.rez_ = 0.0;
00038 }
00039
00040 // Destruktorius - paz_ isvalomas automatiskai
00041 // grandinine tvarka isskviecia Zmogus::~Zmogus()
00042 Studentas::~Studentas() {
00043     paz_.clear();
00044 }

```

```

00045
00046 // Kopijavimo priskyrimo operatorius
00047 Studentas& Studentas::operator=(const Studentas& other) {
00048     if (this != &other) {
00049         Zmogus::operator=(other); // kopijuoja vardas_, pavarde_
00050         paz_ = other.paz_;
00051         egz_ = other.egz_;
00052         rez_ = other.rez_;
00053     }
00054     return *this;
00055 }
00056
00057 // Perkelimo priskyrimo operatorius
00058 Studentas& Studentas::operator=(Studentas&& other) noexcept {
00059     if (this != &other) {
00060         Zmogus::operator=(std::move(other)); // perkeliu vardas_, pavarde_
00061         paz_ = std::move(other.paz_);
00062         egz_ = other.egz_;
00063         rez_ = other.rez_;
00064         other.egz_ = 0;
00065         other.rez_ = 0.0;
00066     }
00067     return *this;
00068 }
00069
00070 // Palyginimo operatoriai (pagal rezultata)
00071 bool Studentas::operator<(const Studentas& other) const {
00072     return rez_ < other.rez_;
00073 }
00074 bool Studentas::operator>(const Studentas& other) const {
00075     return rez_ > other.rez_;
00076 }
00077 bool Studentas::operator==(const Studentas& other) const {
00078     return vardas_ == other.vardas_ && pavarde_ == other.pavarde_;
00079 }
00080 bool Studentas::operator!=(const Studentas& other) const {
00081     return !(*this == other);
00082 }
00083 bool Studentas::operator<=(const Studentas& other) const {
00084     return rez_ <= other.rez_;
00085 }
00086 bool Studentas::operator>=(const Studentas& other) const {
00087     return rez_ >= other.rez_;
00088 }
00089
00090 // Isvedimo operatorius
00091 // Formatas: Vardas Pavarde [p1 p2 ... pN] Egz: E Rez: R
00092 std::ostream& operator<(std::ostream& os, const Studentas& s) {
00093     os << std::left << std::setw(15) << s.vardas_
00094         << std::setw(15) << s.pavarde_
00095         << "[";
00096     for (size_t i = 0; i < s.paz_.size(); ++i) {
00097         os << s.paz_[i];
00098         if (i + 1 < s.paz_.size()) os << " ";
00099     }
00100     os << "]" Egz: " << s.egz_
00101         << std::right << " Rez: "
00102         << std::fixed << std::setprecision(2) << s.rez_;
00103     return os;
00104 }
00105
00106 // Ivedimo operatorius
00107 std::istream& operator>(std::istream& is, Studentas& s) {
00108     is >> s.vardas_ >> s.pavarde_;
00109     std::string line;
00110     std::getline(is, line);
00111     std::istringstream ss(line);
00112     s.paz_.clear();
00113     int v;
00114     while (ss >> v) s.paz_.push_back(v);
00115     if (!s.paz_.empty()) {
00116         s.egz_ = s.paz_.back();
00117         s.paz_.pop_back();
00118     }
00119     else {
00120         s.egz_ = 0;
00121     }
00122     s.rez_ = 0.0;
00123     return is;
00124 }
00125 //
00126 // VIRTUALUS METODAS
00127 //
00128
00129 // Realizuoja Zmogus::print() naudojamas operator<(Zmogus&)
00130 void Studentas::print(std::ostream& os) const {
00131     os << *this;

```



```

00132 }
00133
00134 // Metodai
00135 void Studentas::addPazymys(int p) {
00136     paz_.push_back(p);
00137 }
00138
00139 void Studentas::clearPazymiai() {
00140     paz_.clear();
00141 }
00142
00143 double Studentas::vidurkis() const {
00144     if (paz_.empty()) return 0.0;
00145     return std::accumulate(paz_.begin(), paz_.end(), 0.0) / paz_.size();
00146 }
00147
00148 double Studentas::mediana() const {
00149     if (paz_.empty()) return 0.0;
00150     std::vector<int> tmp = paz_;
00151     std::sort(tmp.begin(), tmp.end());
00152     size_t n = tmp.size();
00153     return (n % 2 == 0) ? (tmp[n / 2 - 1] + tmp[n / 2]) / 2.0 : tmp[n / 2];
00154 }
00155
00156 void Studentas::skaiciuotiRez(int tipas) {
00157     double x = (tipas == 1) ? vidurkis() : mediana();
00158     rez_ = x * 0.4 + egz_ * 0.6;
00159 }

```

5.15 studentas.h Failo Nuoroda

```

#include "zmogus.h"
#include "vector.h"
#include <vector>
#include <iostream>
#include <iomanip>
#include <sstream>

```

Klasės

- class [Studentas](#)

Atstovauja studenta su pazymiais ir egzamino rezultatu.

5.16 studentas.h

Eiti [↗](#) Åjio failo dokumentacijÅ.

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "zmogus.h"
00005 #include "vector.h"
00006 #include <vector>
00007 #include <iostream>
00008 #include <iomanip>
00009 #include <sstream>
00010
00011 /**
00012  * @class Studentas
00013  * @brief Atstovauja studenta su pazymiais ir egzamino rezultatu
00014  *
00015  * Si klase praplecia Zmogus klase ir igyvendina studentams budinga funkcionaluma,
00016  * iskaitant pazymiu valdyma, rezultatu skaiciavima (vidurkis/mediana) ir
00017  * klasifikavima i grupes.
00018  *
00019  * @author Studentu Valdymo Sistema
00020  * @version 2.0

```

```

00021  * @date 2026
00022  */
00023
00024 class Studentas : public Zmogus {
00025 private:
00026     std::vector<int> paz_; // Pazymiai
00027     int egz_; // Egzamino rezultatas
00028     double rez_; // Galutinis rezultatas (vidurkis arba mediana)
00029
00030 public:
00031     // Konstruktoriai
00032     /**
00033      * @brief Numatytasis konstruktorius
00034      *
00035      * Sukuria tuscia studenta su numatytosiomis reikšmemis.
00036      * Visi pazymiai isvalomi, egzaminas = 0, rezultatas = 0.0
00037      */
00038
00039     Studentas();
00040
00041     /**
00042      * @brief Parametrinis konstruktorius
00043      * @param vardas Studento vardas
00044      * @param pavarde Studento pavarde
00045      * @param paz Namu darbu pazymiu vektorius
00046      * @param egz Egzamino pazymys
00047      *
00048      * @example
00049      * std::vector<int> pazymiai = {8, 9, 7};
00050      * Studentas s("Jonas", "Jonaitis", pazymiai, 10);
00051      */
00052
00053
00054     Studentas(const std::string& vardas,
00055               const std::string& pavarde,
00056               const Vector<int>& paz,
00057               int egz);
00058
00059     /**
00060      * @brief Kopijavimo konstruktorius (gilioji kopija)
00061      * @param other Studentas objektas, is kurio kopijuojama
00062      * @kompleksiskumas O(n), kur n yra pazymiu skaicius
00063      */
00064
00065     // Kopijavimo konstruktorius
00066     Studentas(const Studentas& other);
00067
00068     /**
00069      * @brief Perkelimo konstruktorius
00070      * @param other Studentas objektas, is kurio perkeliama
00071      * @kompleksiskumas O(1)
00072      */
00073
00074
00075     Studentas(Studentas&& other) noexcept;
00076
00077     /**
00078      * @brief Virtualus destruktorius
00079      *
00080      * Uztikrina tinkama isvestiniu klasiu isvalyma
00081      */
00082     ~Studentas();
00083
00084     // Priskyrimo operatoriai
00085
00086     /**
00087      * @brief Kopijavimo priskyrimo operatorius
00088      * @param other Studentas objektas, is kurio kopijuojama
00089      * @return Nuoroda i si objekta
00090      */
00091
00092     Studentas& operator=(const Studentas& other);
00093
00094     /**
00095      * @brief Perkelimo priskyrimo operatorius
00096      * @param other Studentas objektas, is kurio perkeliama
00097      * @return Nuoroda i si objekta
00098      */
00099     Studentas& operator=(Studentas&& other) noexcept;
00100
00101     // Palyginimo operatoriai
00102
00103     /**
00104      * @brief Maziau uz (pagal galutini rezultata)
00105      * @param other Studentas su kuriuo lyginama
00106      * @return true jei si.rez_ < other.rez_
00107      */

```

```

00108     bool operator<(const Studentas& other) const;
00109
00110     /**
00111      * @brief Daugiau uz (pagal galutini rezultata)
00112      * @param other Studentas su kuriuo lyginama
00113      * @return true jei si.rez_ > other.rez_
00114      */
00115     bool operator>(const Studentas& other) const;
00116
00117     /**
00118      * @brief Lygybe (pagal varda ir pavarde)
00119      * @param other Studentas su kuriuo lyginama
00120      * @return true jei vardai ir pavardes sutampa
00121      */
00122     bool operator==(const Studentas& other) const;
00123
00124     /**
00125      * @brief Maziau arba lygu
00126      * @return true jei si.rez_ <= other.rez_
00127      */
00128     bool operator<=(const Studentas& other) const;
00129
00130     /**
00131      * @brief Daugiau arba lygu
00132      * @return true jei si.rez_ >= other.rez_
00133      */
00134     bool operator>=(const Studentas& other) const;
00135
00136     /**
00137      * @brief Nelygybe
00138      * @param other Studentas su kuriuo lyginama
00139      * @return true jei vardai ar pavardes skiriasi
00140      */
00141     bool operator!=(const Studentas& other) const;
00142
00143     // Pridėti pazymi: s += 8
00144     Studentas& operator+=(int pazymys);
00145
00146     // Prieiga prie paz_[i]
00147     int operator[](size_t i) const;
00148     int& operator[](size_t i);
00149
00150     // SRAUTU OPERATORIAI
00151
00152     /**
00153      * @brief Išvedimo į ekrano operatorius
00154      * @param os Išvesties srautas
00155      * @param s Studentas objektas
00156      * @return Nuoroda į išvesties srauta
00157      * @detales Formatas: "Vardas Pavarde [pazymiai] Egz: E Rez: R"
00158      */
00159     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00160
00161     /**
00162      * @brief Įvedimo iš failo operatorius
00163      * @param is Įvesties srautas
00164      * @param s Studentas objektas
00165      * @return Nuoroda į įvesties srauta
00166      * @detales Skaito: "Vardas Pavarde paz1 paz2 ... pazN egzaminas"
00167      */
00168     friend std::istream& operator>>(std::istream& is, Studentas& s);
00169
00170     //
00171     // Virtualus metodas realizuoja bazinės klasės gryną virtualų metodą
00172     //
00173
00174
00175     /**
00176      * @brief Spausdina studento informaciją (pakeičia Zmogus::print)
00177      * @param os Išvesties srautas
00178      */
00179     void print(std::ostream& os) const override;
00180
00181     //
00182     // GETTERIAI
00183     //
00184
00185     /** @return Pastovi nuoroda į pazymių vektorių */
00186     const std::vector<int>& getPaz() const { return paz_; }
00187
00188     /** @return Egzamino pazymys */
00189     int getEgz() const { return egz_; }
00190
00191     /** @return Galutinis rezultatas */
00192     double getRez() const { return rez_; }
00193
00194     /** @return Namų darbų pazymių skaičius */

```

```

00195     size_t                               getPazSkaicius() const { return paz_.size(); }
00196
00197     //
00198     // SETTERIAI
00199     //
00200
00201     /** @param e Naujas egzamino pazymys (0-10) */
00202     void setEgz(int e) { egz_ = e; }
00203
00204     /** @param r Naujas galutinis rezultatas */
00205     void setRez(double r) { rez_ = r; }
00206
00207     //
00208     // METODAI
00209     //
00210
00211     /**
00212      * @brief Prideda viena namu darbu pazymi
00213      * @param p Pazymys (1-10)
00214      */
00215     void addPazymys(int p);
00216
00217     /** @brief Isvalo visus namu darbu pazymius */
00218     void clearPazymiai();
00219
00220     /**
00221      * @brief Skaiciuoja namu darbu pazymiu vidurki
00222      * @return Aritmetinis vidurkis kaip double
00223      */
00224     double vidurkis() const;
00225
00226     /**
00227      * @brief Skaiciuoja namu darbu pazymiu mediana
00228      * @return Mediana kaip double
00229      */
00230     double mediana() const;
00231
00232     /**
00233      * @brief Skaiciuoja galutini rezultata
00234      * @param tipas 1=vidurkis, 2=mediana
00235      * @detales Formule: 0.4 * pazymys + 0.6 * egzaminas
00236      */
00237     void skaiciuotiRez(int tipas); // 1=vidurkis, 2=mediana
00238
00239     /**
00240      * @brief Tikrina ar studentas islaike
00241      * @return true jei rez_ >= 5.0
00242      */
00243     bool islaike() const { return rez_ >= 5.0; }
00244 };
00245
00246 #endif

```

5.17 testavimas.cpp Failo Nuoroda

```

#include "vector_ops.h"
#include "list_ops.h"
#include "deque_ops.h"
#include "Vector.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <list>
#include <deque>
#include <string>
#include <chrono>
#include <algorithm>

```

Klasės

- struct [TestRow](#)

Funkcijos

- static void `spausdinti_lentele` (const std::vector< `TestRow` > &eilutes)
- static `TestRow` `testuoti_vector` (const std::string &f, int strat)
- static `TestRow` `testuoti_list` (const std::string &f, int strat)
- static `TestRow` `testuoti_deque` (const std::string &f, int strat)
- void `atlikti_visus_testus` ()

5.17.1 Funkcijos Dokumentacija

5.17.1.1 atlikti_visus_testus()

void atlikti_visus_testus ()

Apibrėžimas failo `testavimas.cpp` eilutėje 136.

```
00136                                     {
00137     const std::vector<std::string> failai = {
00138         "studentail000.txt",
00139         "studentail0000.txt",
00140         "studentail00000.txt",
00141         "studentail000000.txt",
00142         "studentail0000000.txt"
00143     };
00144
00145     std::vector<TestRow> rezultatai;
00146     std::cout << "\n##### PRADEDAMOS TESTAVIMAS #####\n\n";
00147
00148     for (const auto& f : failai) {
00149         for (int s = 1; s <= 3; ++s) {
00150             std::cout << "> vector S" << s << " | " << f << "\n";
00151             try { rezultatai.push_back(testuoti_vector(f, s)); }
00152             catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00153
00154             std::cout << "> list S" << s << " | " << f << "\n";
00155             try { rezultatai.push_back(testuoti_list(f, s)); }
00156             catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00157
00158             std::cout << "> deque S" << s << " | " << f << "\n";
00159             try { rezultatai.push_back(testuoti_deque(f, s)); }
00160             catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00161         }
00162     }
00163
00164     std::cout << "\n##### TESTAVIMAS BAIGTAS #####\n";
00165     spausdinti_lentele(rezultatai);
00166 }
```

5.17.1.2 spausdinti_lentele()

void spausdinti_lentele (
 const std::vector< `TestRow` > &eilutes) [static]

Apibrėžimas failo `testavimas.cpp` eilutėje 26.

```
00026                                     {
00027     std::cout << "\n" << std::string(90, '=') << "\n";
00028     std::cout << std::left
00029         << std::setw(8) << "Kont."
00030         << std::setw(5) << "Strat"
00031         << std::setw(24) << "Failas"
00032         << std::right
00033         << std::setw(10) << "Stud."
00034         << std::setw(12) << "Skait.(s)"
00035         << std::setw(11) << "Rusi.(s)"
00036         << std::setw(12) << "Skirsti.(s)"
00037         << std::setw(12) << "Bendras(s)"
00038         << "\n" << std::string(90, '-') << "\n";
00039     for (const auto& r : eilutes) {
00040         std::cout << std::left
```

```

00041         « std::setw(8) « r.konteineris
00042         « std::setw(5) « r.strategija
00043         « std::setw(24) « r.failas
00044         « std::right « std::fixed « std::setprecision(6)
00045         « std::setw(10) « r.studentu_sk
00046         « std::setw(12) « r.skaitymas
00047         « std::setw(11) « r.rusiavimas
00048         « std::setw(12) « r.skirstymas
00049         « std::setw(12) « r.bendras
00050         « "\n";
00051     }
00052     std::cout « std::string(90, '=') « "\n";
00053 }

```

5.17.1.3 testuoti_deque()

```

TestRow testuoti_deque (
    const std::string & f,
    int strat) [static]

```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 109.

```

00109                                     {
00110     TestRow r;
00111     r.konteineris = "deque";
00112     r.strategija = "S" + std::to_string(strat);
00113     r.failas = f;
00114     auto total = std::chrono::high_resolution_clock::now();
00115
00116     std::deque<Studentas> studentai;
00117     auto t0 = std::chrono::high_resolution_clock::now();
00118     skaitymas_is_failo_d(f, studentai);
00119     r.skaitymas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
t0).count();
00120     r.studentu_sk = (int)studentai.size();
00121     pasirinkimo_metodas_d(1, studentai);
00122
00123     t0 = std::chrono::high_resolution_clock::now();
00124     rusiavimas_d(studentai);
00125     r.rusiavimas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
t0).count();
00126
00127     std::deque<Studentas> kieti, vargsai, tmp;
00128     if (strat == 1) { r.skirstymas = skirstymas_s1_d(studentai, kieti, vargsai); }
00129     else if (strat == 2) { tmp = studentai; r.skirstymas = skirstymas_s2_d(tmp, vargsai); }
00130     else { tmp = studentai; r.skirstymas = skirstymas_s3_d(tmp, vargsai); }
00131
00132     r.bendras = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
total).count();
00133     return r;
00134 }

```

5.17.1.4 testuoti_list()

```

TestRow testuoti_list (
    const std::string & f,
    int strat) [static]

```

Apibrėžimas failo [testavimas.cpp](#) eilutėje 82.

```

00082                                     {
00083     TestRow r;
00084     r.konteineris = "list";
00085     r.strategija = "S" + std::to_string(strat);
00086     r.failas = f;
00087     auto total = std::chrono::high_resolution_clock::now();
00088
00089     std::list<Studentas> studentai;
00090     auto t0 = std::chrono::high_resolution_clock::now();
00091     skaitymas_is_failo_l(f, studentai);
00092     r.skaitymas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
t0).count();
00093     r.studentu_sk = (int)studentai.size();
00094     pasirinkimo_metodas_l(1, studentai);
00095

```

```

00096     t0 = std::chrono::high_resolution_clock::now();
00097     rusiavimas_1(studentai);
00098     r.rusiavimas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00099     t0).count();
00100     std::list<Studentas> kieti, vargsai, tmp;
00101     if (strat == 1) { r.skirstymas = skirstymas_s1_1(studentai, kieti, vargsai); }
00102     else if (strat == 2) { tmp = studentai; r.skirstymas = skirstymas_s2_1(tmp, vargsai); }
00103     else { tmp = studentai; r.skirstymas = skirstymas_s3_1(tmp, vargsai); }
00104
00105     r.bendras = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00106     total).count();
00107     return r;
00107 }

```

5.17.1.5 testuoti_vector()

```

TestRow testuoti_vector (
    const std::string & f,
    int strat) [static]

```

Apibrėžimas failo `testavimas.cpp` eilutėje 55.

```

00055                                     {
00056     TestRow r;
00057     r.konteineris = "vector";
00058     r.strategija = "S" + std::to_string(strat);
00059     r.failas = f;
00060     auto total = std::chrono::high_resolution_clock::now();
00061
00062     Vector<Studentas> studentai; // <-- Vector, ne std::vector
00063     auto t0 = std::chrono::high_resolution_clock::now();
00064     skaitymas_is_failo(f, studentai);
00065     r.skaitymas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00066     t0).count();
00067     r.studentu_sk = (int)studentai.size();
00068     pasirinkimo_metodas(1, studentai);
00069
00070     t0 = std::chrono::high_resolution_clock::now();
00071     rusiavimas(studentai, 5);
00072     r.rusiavimas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00073     t0).count();
00074
00075     Vector<Studentas> kieti, vargsai, tmp; // <-- Vector, ne std::vector
00076     if (strat == 1) { r.skirstymas = skirstymas_s1(studentai, kieti, vargsai); }
00077     else if (strat == 2) { tmp = studentai; r.skirstymas = skirstymas_s2(tmp, vargsai); }
00078     else { tmp = studentai; r.skirstymas = skirstymas_s3(tmp, vargsai); }
00079
00080     r.bendras = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00081     total).count();
00082     return r;
00082 }

```

5.18 testavimas.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "vector_ops.h"
00002 #include "list_ops.h"
00003 #include "deque_ops.h"
00004 #include "Vector.h"
00005
00006 #include <iostream>
00007 #include <iomanip>
00008 #include <vector>
00009 #include <list>
00010 #include <deque>
00011 #include <string>
00012 #include <chrono>
00013 #include <algorithm>
00014
00015 struct TestRow {
00016     std::string konteineris;
00017     std::string strategija;
00018     std::string failas;
00019     int studentu_sk = 0;

```

```

00020     double skaitymas = 0.0;
00021     double rusiavimas = 0.0;
00022     double skirstymas = 0.0;
00023     double bendras = 0.0;
00024 };
00025
00026 static void spausdinti_lentele(const std::vector<TestRow>& eilutes) {
00027     std::cout << "\n" << std::string(90, '-') << "\n";
00028     std::cout << std::left
00029         << std::setw(8) << "Kont."
00030         << std::setw(5) << "Strat"
00031         << std::setw(24) << "Failas"
00032         << std::right
00033         << std::setw(10) << "Stud."
00034         << std::setw(12) << "Skait.(s)"
00035         << std::setw(11) << "Rusi.(s)"
00036         << std::setw(12) << "Skirsti.(s)"
00037         << std::setw(12) << "Bendras(s)"
00038         << "\n" << std::string(90, '-') << "\n";
00039     for (const auto& r : eilutes) {
00040         std::cout << std::left
00041             << std::setw(8) << r.konteineris
00042             << std::setw(5) << r.strategija
00043             << std::setw(24) << r.failas
00044             << std::right << std::fixed << std::setprecision(6)
00045             << std::setw(10) << r.studentu_sk
00046             << std::setw(12) << r.skaitymas
00047             << std::setw(11) << r.rusiavimas
00048             << std::setw(12) << r.skirstymas
00049             << std::setw(12) << r.bendras
00050             << "\n";
00051     }
00052     std::cout << std::string(90, '-') << "\n";
00053 }
00054
00055 static TestRow testuoti_vector(const std::string& f, int strat) {
00056     TestRow r;
00057     r.konteineris = "vector";
00058     r.strategija = "S" + std::to_string(strat);
00059     r.failas = f;
00060     auto total = std::chrono::high_resolution_clock::now();
00061
00062     Vector<Studentas> studentai; // <-- Vector, ne std::vector
00063     auto t0 = std::chrono::high_resolution_clock::now();
00064     skaitymas_is_failo(f, studentai);
00065     r.skaitymas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00066 t0).count();
00067     r.studentu_sk = (int)studentai.size();
00068     pasirinkimo_metodas(1, studentai);
00069
00069     t0 = std::chrono::high_resolution_clock::now();
00070     rusiavimas(studentai, 5);
00071     r.rusiavimas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00072 t0).count();
00073     Vector<Studentas> kieti, vargsai, tmp; // <-- Vector, ne std::vector
00074     if (strat == 1) { r.skirstymas = skirstymas_s1(studentai, kieti, vargsai); }
00075     else if (strat == 2) { tmp = studentai; r.skirstymas = skirstymas_s2(tmp, vargsai); }
00076     else { tmp = studentai; r.skirstymas = skirstymas_s3(tmp, vargsai); }
00077
00078     r.bendras = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00079 total).count();
00080     return r;
00081 }
00082
00082 static TestRow testuoti_list(const std::string& f, int strat) {
00083     TestRow r;
00084     r.konteineris = "list";
00085     r.strategija = "S" + std::to_string(strat);
00086     r.failas = f;
00087     auto total = std::chrono::high_resolution_clock::now();
00088
00089     std::list<Studentas> studentai;
00090     auto t0 = std::chrono::high_resolution_clock::now();
00091     skaitymas_is_failo_l(f, studentai);
00092     r.skaitymas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00093 t0).count();
00094     r.studentu_sk = (int)studentai.size();
00095     pasirinkimo_metodas_l(1, studentai);
00096
00097     t0 = std::chrono::high_resolution_clock::now();
00098     rusiavimas_l(studentai);
00099     r.rusiavimas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00100 t0).count();
00101
00102     std::list<Studentas> kieti, vargsai, tmp;
00103     if (strat == 1) { r.skirstymas = skirstymas_sl_l(studentai, kieti, vargsai); }

```



```

00102     else if (strat == 2) { tmp = studentai; r.skirstymas = skirstymas_s2_l(tmp, vargsai); }
00103     else { tmp = studentai; r.skirstymas = skirstymas_s3_l(tmp, vargsai); }
00104
00105     r.bendras = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
total).count();
00106     return r;
00107 }
00108
00109 static TestRow testuoti_deque(const std::string& f, int strat) {
00110     TestRow r;
00111     r.konteineris = "deque";
00112     r.strategija = "S" + std::to_string(strat);
00113     r.failas = f;
00114     auto total = std::chrono::high_resolution_clock::now();
00115
00116     std::deque<Studentas> studentai;
00117     auto t0 = std::chrono::high_resolution_clock::now();
00118     skaitymas_is_failo_d(f, studentai);
00119     r.skaitymas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
t0).count();
00120     r.studentu_sk = (int)studentai.size();
00121     pasirinkimo_metodas_d(1, studentai);
00122
00123     t0 = std::chrono::high_resolution_clock::now();
00124     rusiavimas_d(studentai);
00125     r.rusiavimas = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
t0).count();
00126
00127     std::deque<Studentas> kieti, vargsai, tmp;
00128     if (strat == 1) { r.skirstymas = skirstymas_s1_d(studentai, kieti, vargsai); }
00129     else if (strat == 2) { tmp = studentai; r.skirstymas = skirstymas_s2_d(tmp, vargsai); }
00130     else { tmp = studentai; r.skirstymas = skirstymas_s3_d(tmp, vargsai); }
00131
00132     r.bendras = std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
total).count();
00133     return r;
00134 }
00135
00136 void atlikti_visus_testus() {
00137     const std::vector<std::string> failai = {
00138         "studentai000.txt",
00139         "studentai0000.txt",
00140         "studentai000000.txt",
00141         "studentai0000000.txt",
00142         "studentai00000000.txt"
00143     };
00144
00145     std::vector<TestRow> rezultatai;
00146     std::cout << "\n##### PRADEAMAS TESTAVIMAS #####\n\n";
00147
00148     for (const auto& f : failai) {
00149         for (int s = 1; s <= 3; ++s) {
00150             std::cout << "> vector S" << s << " | " << f << "\n";
00151             try { rezultatai.push_back(testuoti_vector(f, s)); }
00152             catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00153
00154             std::cout << "> list S" << s << " | " << f << "\n";
00155             try { rezultatai.push_back(testuoti_list(f, s)); }
00156             catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00157
00158             std::cout << "> deque S" << s << " | " << f << "\n";
00159             try { rezultatai.push_back(testuoti_deque(f, s)); }
00160             catch (const std::exception& e) { std::cerr << " KLAIDA: " << e.what() << "\n"; }
00161         }
00162     }
00163
00164     std::cout << "\n##### TESTAVIMAS BAIGTAS #####\n";
00165     spausdinti_lentele(rezultatai);
00166 }

```

5.19 testavimas_klases.cpp Failo Nuoroda

```

#include "studentas.h"
#include "zmogus.h"
#include <iostream>
#include <sstream>
#include <cassert>
#include <vector>

```

```
#include <algorithm>
#include <iomanip>
```

Funkcijos

- static void [tikrinti](#) (bool salyga, const std::string &pavadinimas)
- static void [sekcija](#) (const std::string &pavadinimas)
- static void [testuoti_konstruktorius](#) ()
- static void [testuoti_priskyrima](#) ()
- static void [testuoti_palyginimo_operatorius](#) ()
- static void [testuoti_srautu_operatorius](#) ()
- static void [testuoti_metodus](#) ()
- static void [testuoti_su_stl](#) ()
- static void [testuoti_zmogus_abstrakti](#) ()
- void [atlikti_klases_testus](#) ()

Kintamieji

- static int [testai_is_viso](#) = 0
- static int [testai_pavyke](#) = 0

5.19.1 Funkcijos Dokumentacija

5.19.1.1 atlikti_klases_testus()

```
void atlikti_klases_testus ()
```

Apibrėžimas failo [testavimas_klases.cpp](#) eilutėje 330.

```
00330                                     {
00331     std::cout << "\n";
00332     std::cout << std::string(55, '=') << "\n";
00333     std::cout << "    STUDENTAS KLASES TESTAVIMAS    (v1.2)\n";
00334     std::cout << std::string(55, '=') << "\n";
00335
00336     testuoti_konstruktorius();
00337     testuoti_priskyrima();
00338     testuoti_palyginimo_operatorius();
00339     testuoti_srautu_operatorius();
00340     testuoti_metodus();
00341     testuoti_su_stl();
00342     testuoti_zmogus_abstrakti();
00343
00344     std::cout << "\n" << std::string(55, '=') << "\n";
00345     std::cout << "    Rezultatas: " << testai_pavyke << " / "
00346               << testai_is_viso << " testu pavyko\n";
00347     if (testai_pavyke == testai_is_viso)
00348         std::cout << "    Visi testai pavyko!\n";
00349     else
00350         std::cout << "    DEMESIO: "
00351               << (testai_is_viso - testai_pavyke)
00352               << " testai nepavyko!\n";
00353     std::cout << std::string(55, '=') << "\n";
00354 }
```

5.19.1.2 sekcija()

```
void sekcija (
    const std::string & pavadinimas) [static]
```

Apibrėžimas failo `testavimas_klases.cpp` eilutėje 26.

```
00026     {
00027         std::cout << "\n===== " << pavadinimas << " =====\n";
00028     }
```

5.19.1.3 testuoti_konstruktorius()

```
void testuoti_konstruktorius () [static]
```

Apibrėžimas failo `testavimas_klases.cpp` eilutėje 32.

```
00032     {
00033         sekcija("Konstruktoriai (Rules of five)");
00034
00035         //Numatyti konstruktoriai
00036
00037         Studentas a;
00038         tikrinti(a.getVardas() == "", "Numatytasis vardas tuscias");
00039         tikrinti(a.getPavarde() == "", "Numatytasis pavarde tuscias");
00040         tikrinti(a.getEgz() == 0, "Numatytasis egzaminas 0");
00041         tikrinti(a.getRez() == 0.0, "Numatytasis rezultatas 0.0");
00042         tikrinti(a.getPazSkaicius() == 0, "Numatytasis paz saras tuscias");
00043
00044         //Parametriniai konstruktoriai
00045
00046         Studentas b("Jonas", "Jonaitis", { 8, 9, 7 }, 10);
00047         tikrinti(b.getVardas() == "Jonas", "Parametrinis vardas");
00048         tikrinti(b.getPavarde() == "Jonaitis", "Parametrinis pavarde");
00049         tikrinti(b.getEgz() == 10, "Parametrinis egzaminas");
00050         tikrinti(b.getRez() == 0.0, "Parametrinis rezultatas (pries skaiciavima)");
00051         tikrinti(b.getPazSkaicius() == 3, "Parametrinis paz skaicius");
00052         tikrinti(b.getPaz()[0] == 8, "Parametrinis pazymiai paz[0] = 8");
00053
00054         //Kopijavimo konstruktorius
00055
00056         Studentas c(b);
00057
00058         tikrinti(c.getVardas() == b.getVardas(), "Kopijavimo vardas sutampa");
00059         tikrinti(c.getPavarde() == b.getPavarde(), "Kopijavimo pavarde sutampa");
00060         tikrinti(c.getEgz() == b.getEgz(), "Kopijavimo egzaminas sutampa");
00061         tikrinti(c.getPazSkaicius() == b.getPazSkaicius(), "Kopijavimo paz skaicius sutampa");
00062         tikrinti(c.getPaz() == b.getPaz(), "Kopijavimo pazymiai sutampa");
00063         c.setVardas("Petras");
00064         tikrinti(b.getVardas() == "Jonas", "Kopijavimo gili kopija (b) nepakito");
00065
00066         //Perkelimo konstruktorius
00067
00068         Studentas d_orig("Ona", "Onaite", {5, 6}, 7);
00069         Studentas d(std::move(d_orig));
00070         tikrinti(d.getVardas() == "Ona", "Perkelimo vardas");
00071         tikrinti(d.getPavarde() == "Onaite", "Perkelimo pavarde");
00072         tikrinti(d.getEgz() == 7, "Perkelimo egzaminas");
00073         tikrinti(d.getPazSkaicius() == 2, "Perkelimo paz skaicius");
00074         tikrinti(d_orig.getPazSkaicius() == 0, "Perkelimo originalas tuscias");
00075         tikrinti(d_orig.getVardas() == "", "Perkelimo originalo vardas tuscias");
00076
00077
00078         //Destruktorius.
00079
00080         {
00081             Studentas laikinas("Laikinas", "Laikinaitis", { 1, 2 }, 3);
00082             (void)laikinas; // Kad destruktoriaus nebutu pazymetas kaip nenaudojamas
00083         }
00084         tikrinti(true, "Destruktorius iskvieciamas (neuzstringa)");
00085     }
```

5.19.1.4 testuoti_metodus()

```
void testuoti_metodus () [static]
```

Apibrėžimas failo [testavimas_klases.cpp](#) eilutėje 201.

```
00201 {
00202     sekcija("Metodai: vidurkis, mediana, skaiciuotiRez, islaike");
00203
00204     // vidurkis
00205     Studentas a("X", "Y", { 6, 8, 10 }, 0);
00206     tikrinti(std::abs(a.vidurkis() - 8.0) < 1e-9, "vidurkis(): (6+8+10)/3 = 8.0");
00207
00208     // vidurkis su tusciu sarasu
00209     Studentas b;
00210     tikrinti(b.vidurkis() == 0.0, "vidurkis(): tuscias sarasas = 0.0");
00211
00212     // mediana nelyginis skaicius
00213     Studentas c("X", "Y", { 3, 9, 5 }, 0);
00214     tikrinti(std::abs(c.mediana() - 5.0) < 1e-9, "mediana(): {3,9,5} -> 5.0");
00215
00216     // mediana lyginis skaicius
00217     Studentas d("X", "Y", { 2, 4, 6, 8 }, 0);
00218     tikrinti(std::abs(d.mediana() - 5.0) < 1e-9, "mediana(): {2,4,6,8} -> 5.0");
00219
00220     // skaiciuotiRez su vidurkiu
00221     Studentas e("X", "Y", { 8, 10, 6 }, 8);
00222     e.skaiciuotiRez(1); // vidurkis = 8, rez = 8*0.4 + 8*0.6 = 8.0
00223     tikrinti(std::abs(e.getRez() - 8.0) < 1e-9, "skaiciuotiRez(1): rez = 8.0");
00224
00225     // skaiciuotiRez su mediana
00226     Studentas f("X", "Y", { 4, 6, 8 }, 10);
00227     f.skaiciuotiRez(2); // mediana = 6, rez = 6*0.4 + 10*0.6 = 2.4+6.0 = 8.4
00228     tikrinti(std::abs(f.getRez() - 8.4) < 1e-9, "skaiciuotiRez(2): rez = 8.4");
00229
00230     // islaike
00231     Studentas g("X", "Y", {}, 0);
00232     g.setRez(4.9);
00233     tikrinti(!g.islaike(), "islaike(): 4.9 < 5.0 -> false");
00234     g.setRez(5.0);
00235     tikrinti(g.islaike(), "islaike(): 5.0 >= 5.0 -> true");
00236     g.setRez(8.5);
00237     tikrinti(g.islaike(), "islaike(): 8.5 >= 5.0 -> true");
00238
00239     // addPazymys ir clearPazymiai
00240     Studentas h;
00241     h.addPazymys(7);
00242     h.addPazymys(9);
00243     tikrinti(h.getPazSkaicius() == 2, "addPazymys(): 2 pazymiai");
00244     h.clearPazymiai();
00245     tikrinti(h.getPazSkaicius() == 0, "clearPazymiai(): sarasas tuscias");
00246
00247     // getteriai ir setteriai
00248     Studentas i;
00249     i.setVardas("Inga");
00250     i.setPavarde("Ingaitė");
00251     i.setEgz(7);
00252     i.setRez(6.5);
00253     tikrinti(i.getVardas() == "Inga", "setter/getter: vardas");
00254     tikrinti(i.getPavarde() == "Ingaitė", "setter/getter: pavarde");
00255     tikrinti(i.getEgz() == 7, "setter/getter: egz");
00256     tikrinti(std::abs(i.getRez() - 6.5) < 1e-9, "setter/getter: rez");
00257 }
```

5.19.1.5 testuoti_palyginimo_operatorius()

```
void testuoti_palyginimo_operatorius () [static]
```

Apibrėžimas failo [testavimas_klases.cpp](#) eilutėje 124.

```
00124 {
00125
00126     sekcija("Palyginimo operatoriai");
00127
00128     Studentas a("Jonas", "Jonaitis", { 8, 9, 7 }, 10);
00129     Studentas b("Petras", "Petraitis", { 4, 3, 5 }, 4);
00130     a.skaiciuotiRez(1); // rez_ = 8.0*0.4 + 10*0.6 = 3.2+6 = 9.2 (vidurkis=(8+9+7)/3=8)
00131     b.skaiciuotiRez(1); // rez_ = 4.0*0.4 + 4*0.6 = 1.6+2.4 = 4.0
00132
00133     tikrinti(a > b, "operator> : a.rez > b.rez");
```

```

00134     tikrinti(b < a, "operator< : b.rez < a.rez");
00135     tikrinti(a >= b, "operator>= : a.rez >= b.rez");
00136     tikrinti(b <= a, "operator<= : b.rez <= a.rez");
00137
00138     //Lygybe - pagal varda ir pavarde
00139
00140     Studentas c("Jonas", "Jonaitis", { 1 }, 1);
00141     tikrinti(a == c, "operator== : tas pats vardas ir pavarde");
00142     tikrinti(a != b, "operator!= : skirtingas vardas ir pavarde");
00143
00144     //lygus sau
00145     tikrinti(a == a, "operator== : lygus sau");
00146     tikrinti(!(a != a), "operator!= : nelygus sau");
00147
00148     // <= ir >= su lygiais rezultatais
00149
00150     Studentas d("Dalia", "Dalaite", { 8, 9, 7 }, 10);
00151     d.skaiciuotiRez(1); //tas patas rezultatas kaip a, bet skirtingas vardas ir pavarde
00152     tikrinti(a <= d, "operator<= lygus rez (a <= d)");
00153     tikrinti(a >= d, "operator>= lygus rez (a >= d)");
00154
00155 }

```

5.19.1.6 testuoti_priskyrima()

```
void testuoti_priskyrima () [static]
```

Apibrąžimas failo `testavimas_klases.cpp` eilutėje 88.

```

00088     {
00089         sekcija("Priskyrimo operatoriai (Rule of Five)");
00090
00091         Studentas a("Jonas", "Jonaitis", { 8, 9 }, 10);
00092         Studentas b;
00093
00094         //kopijavimo priskyrimo operatorius
00095         b = a;
00096         tikrinti(b.getVardas() == "Jonas", "Kopijavimo priskyrimo vardas");
00097         tikrinti(b.getPavarde() == "Jonaitis", "Kopijavimo priskyrimo pavarde");
00098         tikrinti(b.getEgz() == 10, "Kopijavimas priskyrimas egz");
00099         tikrinti(b.getPaz() == a.getPaz(), "Kopijavimo priskyrimo pazymiai sutampa");
00100         //Gilios kopijos tikrinimas
00101         b.setVardas("kitas");
00102         tikrinti(a.getVardas() == "Jonas", "Kopijavimo priskyrimo gili kopija (a) nepakito");
00103
00104         //saves priskyrimo operatorius
00105         a = a;
00106         tikrinti(a.getVardas() == "Jonas", "Saves priskyrimo vardas nepakito");
00107
00108         //perkelimo priskyrimo operatorius
00109
00110         Studentas c("Ona", "Onaite", { 5, 6 }, 7);
00111         Studentas d;
00112         d = std::move(c);
00113         tikrinti(d.getVardas() == "Ona", "Perkelimo priskyrimo vardas");
00114         tikrinti(d.getPazSkaicius() == 2, "Perkelimo priskyrimo paz skaicius");
00115         tikrinti(c.getVardas() == "", "Perkelimo priskyrimo originalo vardas tuscias");
00116
00117         //Grandininis priskyrimas
00118         Studentas x("A", "B", { 1 }, 2);
00119         Studentas y, z;
00120         z = y = x;
00121         tikrinti(y.getVardas() == "A" && z.getVardas() == "A", "Grandininis priskyrimas y = z = x");
00122     }

```

5.19.1.7 testuoti_srautu_operatorius()

```
void testuoti_srautu_operatorius () [static]
```

Apibrąžimas failo `testavimas_klases.cpp` eilutėje 158.

```

00158     {
00159         sekcija("Srautu operatoriai");
00160
00161         //operator> su istringstream
00162         Studentas a;
00163         std::istringstream iss("Jonas Jonaitis 8 9 7 10");

```

```

00164     iss >> a;
00165     tikrinti(a.getVardas() == "Jonas", "operator» vardas");
00166     tikrinti(a.getPavarde() == "Jonaitis", "operator» pavarde");
00167     tikrinti(a.getPazSkaicius() == 3, "operator» paz skaicius");
00168     tikrinti(a.getPaz()[0] == 8, "operator» pazymiai paz[0] == 8");
00169     tikrinti(a.getPaz()[2] == 7, "operator» pazymiai paz[2] == 7");
00170
00171     //operator» su vienu skaiciumi (tik egzaminas, 0 namu darbu)
00172     Studentas b;
00173     std::istringstream iss2("Ona Onaite 5");
00174     iss2 >> b;
00175     tikrinti(b.getEgz() == 5, "operator» egzaminas");
00176     tikrinti(b.getPazSkaicius() == 0, "operator» paz skaicius 0");
00177
00178     //operator« su ostringstream
00179     Studentas c("Petras", "Petraitis", { 6, 8 }, 9);
00180     c.skaiciuotiRez(1);
00181     std::ostringstream oss;
00182     oss << c;
00183     std::string isvestis = oss.str();
00184
00185     tikrinti(isvestis.find("Petras") != std::string::npos, "operator«: yra vardas");
00186     tikrinti(isvestis.find("Petraitis") != std::string::npos, "operator«: yra pavarde");
00187     tikrinti(isvestis.find("Egz") != std::string::npos, "operator«: yra Egz.");
00188     tikrinti(isvestis.find("Rez") != std::string::npos, "operator«: yra Rez.");
00189
00190     // Apvalus testas: » tada «
00191     Studentas d;
00192     std::istringstream iss3("Inga Ingaite 7 8 9 6");
00193     iss3 >> d;
00194     std::ostringstream oss2;
00195     oss2 << d;
00196     tikrinti(oss2.str().find("Inga") != std::string::npos,
00197             "operator« po »: vardas isvestyje");
00198
00199 }

```

5.19.1.8 testuoti_su_stl()

```
void testuoti_su_stl () [static]
```

Apibrąšimas failo [testavimas_klases.cpp](#) eilutėje 259.

```

00259     {
00260         sekcija("Naudojimas su STL konteineriais ir algoritmais");
00261
00262         std::vector<Studentas> studentai = {
00263             Studentas("C", "C", {4,4}, 4),
00264             Studentas("A", "A", {9,9}, 9),
00265             Studentas("B", "B", {6,6}, 6)
00266         };
00267         for (auto& s : studentai) s.skaiciuotiRez(1);
00268
00269         // std::sort su operator<
00270         std::sort(studentai.begin(), studentai.end());
00271         tikrinti(studentai[0].getVardas() == "C" &&
00272             studentai[1].getVardas() == "B" &&
00273             studentai[2].getVardas() == "A",
00274             "std::sort su operator<: rusiuoja pagal rez_");
00275
00276         // std::find_if su islaike
00277         auto pirmas_islaikes = std::find_if(studentai.begin(), studentai.end(),
00278             [](const Studentas& s) { return s.islaike(); });
00279         tikrinti(pirmas_islaikes != studentai.end() &&
00280             pirmas_islaikes->getVardas() == "B",
00281             "std::find_if su islaike(): randa pirma islaikusi");
00282         // std::partition su operator>=
00283         std::vector<Studentas> v = studentai;
00284         auto pivot = std::partition(v.begin(), v.end(),
00285             [](const Studentas& s) { return s.islaike(); });
00286         int islaike = (int)std::distance(v.begin(), pivot);
00287         int neislaike = (int)std::distance(pivot, v.end());
00288         tikrinti(islaike == 2 && neislaike == 1,
00289             "std::partition + islaike(): 2 islaike, 1 neislaike");
00290     }

```

5.19.1.9 testuoti_zmogus_abstrakti()

```
void testuoti_zmogus_abstrakti () [static]
```

Apibrąžimas failo [testavimas_klases.cpp](#) eilutėje 293.

```

00293     {
00294         sekcijs("Zmogus abstrakti klase (v1.5)");
00295
00296         // Zmogus objekto sukurti NEGALIMA klase abstrakti.
00297         // Toliau pateiktas kodas NEKOMPILIUOJAS (tai yra tinkamas elgesys):
00298         // Zmogus z; // KLAIDA: cannot instantiate abstract class
00299         // Zmogus z("A", "B"); // KLAIDA: cannot instantiate abstract class
00300         // Tai irodo kad abstraktumas veikia teisingai.
00301
00302         // Galima sukurti tik Studentas objekta (isvestine klase):
00303         Studentas s("Jonas", "Jonaitis", { 8, 9 }, 10);
00304         tikrinti(true, "Zmogus abstrakti: Studentas objektas sukurtas sekmingai");
00305
00306         // Studentas yra Zmogus tipo (polimorfizmas)
00307         Zmogus* ptr = new Studentas("Ona", "Onaite", { 7, 8 }, 9);
00308         tikrinti(ptr != nullptr, "Zmogus abstrakti: Zmogus* rodo i Studentas objekta");
00309         tikrinti(ptr->getVardas() == "Ona", "Zmogus abstrakti: getVardas() per bazine klase");
00310         tikrinti(ptr->getPavarde() == "Onaite", "Zmogus abstrakti: getPavarde() per bazine klase");
00311
00312         // operator<< per bazine klase naudoja virtual print()
00313         std::ostringstream oss;
00314         oss << *ptr;
00315         tikrinti(oss.str().find("Ona") != std::string::npos,
00316                 "Zmogus abstrakti: operator<< per Zmogus& naudoja virtual print()");
00317
00318         // Virtualus destruktorius teisingai iskvieciamas Studentas destruktorius
00319         delete ptr;
00320         tikrinti(true, "Zmogus abstrakti: virtualus destruktorius iskviestas teisingai");
00321
00322         // Bazines klases getteriai ir setteriai veikia per isvestine klase
00323         Studentas t;
00324         t.setVardas("Petras");
00325         t.setPavarde("Petraitis");
00326         tikrinti(t.getVardas() == "Petras", "Zmogus abstrakti: setVardas/getVardas is bazines");
00327         tikrinti(t.getPavarde() == "Petraitis", "Zmogus abstrakti: setPavarde/getPavarde is bazines");
00328     }

```

5.19.1.10 tikrinti()

```

void tikrinti (
    bool salyga,
    const std::string & pavadinimas) [static]

```

Apibrąžimas failo [testavimas_klases.cpp](#) eilutėje 15.

```

00015     {
00016         ++testai_is_viso;
00017         if (salyga) {
00018             ++testai_pavyke;
00019             std::cout << "Testas pavykes: " << pavadinimas << "\n";
00020         }
00021         else {
00022             std::cout << "Testas nepavykes: " << pavadinimas << "\n";
00023         }
00024     }

```

5.19.2 Kintamojo Dokumentacija

5.19.2.1 testai_is_viso

```
int testai_is_viso = 0 [static]
```

Apibrąžimas failo [testavimas_klases.cpp](#) eilutėje 12.

5.19.2.2 testai_pavyke

```
int testai_pavyke = 0 [static]
```

Apibrąžimas failo [testavimas_klases.cpp](#) eilutėje 13.

5.20 testavimas_klases.cpp

Eiti [Å](#) Åjø failo dokumentacijÅ.

```

00001 #include "studentas.h"
00002 #include "zmogus.h"
00003 #include <iostream>
00004 #include <sstream>
00005 #include <cassert>
00006 #include <vector>
00007 #include <algorithm>
00008 #include <iomanip>
00009
00010 //Pagalbines funkcijos testavimui
00011
00012 static int testai_is_viso = 0;
00013 static int testai_pavyke = 0;
00014
00015 static void tikrinti(bool salyga, const std::string& pavadinimas) {
00016     ++testai_is_viso;
00017     if (salyga) {
00018         ++testai_pavyke;
00019         std::cout << "Testas pavykes: " << pavadinimas << "\n";
00020     }
00021     else {
00022         std::cout << "Testas nepavykes: " << pavadinimas << "\n";
00023     }
00024 }
00025
00026 static void sekcija (const std::string& pavadinimas) {
00027     std::cout << "\n===== " << pavadinimas << " =====\n";
00028 }
00029
00030 // Testavimas
00031
00032 static void testuoti_konstruktorius() {
00033     sekcija("Konstruktoriai (Rules of five)");
00034
00035     //Numatyti konstruktoriai
00036
00037     Studentas a;
00038     tikrinti(a.getVardas() == "", "Numatytasis vardas tuscias");
00039     tikrinti(a.getPavarde() == "", "Numatytasis pavarde tuscias");
00040     tikrinti(a.getEgz() == 0, "Numatytasis egzaminas 0");
00041     tikrinti(a.getRez() == 0.0, "Numatytasis rezultatas 0.0");
00042     tikrinti(a.getPazSkaicius() == 0, "Numatytasis paz saras tuscias");
00043
00044     //Parametriniai konstruktoriai
00045
00046     Studentas b("Jonas", "Jonaitis", { 8, 9, 7 }, 10);
00047     tikrinti(b.getVardas() == "Jonas", "Parametrinis vardas");
00048     tikrinti(b.getPavarde() == "Jonaitis", "Parametrinis pavarde");
00049     tikrinti(b.getEgz() == 10, "Parametrinis egzaminas");
00050     tikrinti(b.getRez() == 0.0, "Parametrinis rezultatas (pries skaiciavima)");
00051     tikrinti(b.getPazSkaicius() == 3, "Parametrinis paz skaicius");
00052     tikrinti(b.getPaz()[0] == 8, "Parametrinis pazymiai paz[0] = 8");
00053
00054     //Kopijavimo konstruktorius
00055
00056     Studentas c(b);
00057
00058     tikrinti(c.getVardas() == b.getVardas(), "Kopijavimo vardas sutampa");
00059     tikrinti(c.getPavarde() == b.getPavarde(), "Kopijavimo pavarde sutampa");
00060     tikrinti(c.getEgz() == b.getEgz(), "Kopijavimo egzaminas sutampa");
00061     tikrinti(c.getPazSkaicius() == b.getPazSkaicius(), "Kopijavimo paz skaicius sutampa");
00062     tikrinti(c.getPaz() == b.getPaz(), "Kopijavimo pazymiai sutampa");
00063     c.setVardas("Petras");
00064     tikrinti(b.getVardas() == "Jonas", "Kopijavimo gili kopija (b) nepakito");
00065
00066     //Perkelimo konstruktorius
00067
00068     Studentas d_orig("Ona", "Onaite", {5, 6}, 7);
00069     Studentas d(std::move(d_orig));
00070     tikrinti(d.getVardas() == "Ona", "Perkelimo vardas");
00071     tikrinti(d.getPavarde() == "Onaite", "Perkelimo pavarde");
00072     tikrinti(d.getEgz() == 7, "Perkelimo egzaminas");
00073     tikrinti(d.getPazSkaicius() == 2, "Perkelimo paz skaicius");
00074     tikrinti(d_orig.getPazSkaicius() == 0, "Perkelimo orginalas tuscias");
00075     tikrinti(d_orig.getVardas() == "", "Perkelimo orginalo vardas tuscias");
00076
00077
00078     //Destruktorius.
00079
00080     {
00081         Studentas laikinas("Laikinas", "Laikinaitis", { 1, 2 }, 3);
00082         (void)laikinas; // Kad destruktoriaus nebutu pazymetas kaip nenaudojamas

```



```

00083     }
00084     tikrinti(true, "Destruktorius iskvieciamas (neuzstringa)");
00085 }
00086
00087
00088 static void testuoti_priskyrima() {
00089     sekcija("Priskyrimo operatoriai (Rule of Five)");
00090
00091     Studentas a("Jonas", "Jonaitis", { 8, 9 }, 10);
00092     Studentas b;
00093
00094     //kopiavimo priskyrimo operatorius
00095     b = a;
00096     tikrinti(b.getVardas() == "Jonas", "Kopiavimo priskyrimo vardas");
00097     tikrinti(b.getPavarde() == "Jonaitis", "Kopiavimo priskyrimo pavarde");
00098     tikrinti(b.getEgz() == 10, "Kopiavimas priskyrimas egz");
00099     tikrinti(b.getPaz() == a.getPaz(), "Kopiavimo priskyrimo pazymiai sutampa");
00100     //Giliuos kopijos tikrinimas
00101     b.setVardas("kitas");
00102     tikrinti(a.getVardas() == "Jonas", "Kopiavimo priskyrimo gili kopija (a) nepakito");
00103
00104     //saves priskyrimo operatorius
00105     a = a;
00106     tikrinti(a.getVardas() == "Jonas", "Saves priskyrimo vardas nepakito");
00107
00108     //perkelimo priskyrimo operatorius
00109
00110     Studentas c("Ona", "Onaite", { 5, 6 }, 7);
00111     Studentas d;
00112     d = std::move(c);
00113     tikrinti(d.getVardas() == "Ona", "Perkelimo priskyrimo vardas");
00114     tikrinti(d.getPazSkaicius() == 2, "Perkelimo priskyrimo paz skaicius");
00115     tikrinti(c.getVardas() == "", "Perkelimo priskyrimo originalo vardas tuscias");
00116
00117     //Grandininis priskyrimas
00118     Studentas x("A", "B", { 1 }, 2);
00119     Studentas y, z;
00120     z = y = x;
00121     tikrinti(y.getVardas() == "A" && z.getVardas() == "A", "Grandininis priskyrimas y = z = x");
00122 }
00123
00124 static void testuoti_palyginimo_operatorius() {
00125     sekcija("Palyginimo operatoriai");
00126
00127     Studentas a("Jonas", "Jonaitis", { 8, 9, 7 }, 10);
00128     Studentas b("Petras", "Petraitis", { 4, 3, 5 }, 4);
00129     a.skaiciuotiRez(1); // rez_ = 8.0*0.4 + 10*0.6 = 3.2+6 = 9.2 (vidurkis=(8+9+7)/3=8)
00130     b.skaiciuotiRez(1); // rez_ = 4.0*0.4 + 4*0.6 = 1.6+2.4 = 4.0
00131
00132     tikrinti(a > b, "operator> : a.rez > b.rez");
00133     tikrinti(b < a, "operator< : b.rez < a.rez");
00134     tikrinti(a >= b, "operator>= : a.rez >= b.rez");
00135     tikrinti(b <= a, "operator<= : b.rez <= a.rez");
00136
00137     //Lygybe - pagal varda ir pavarde
00138
00139     Studentas c("Jonas", "Jonaitis", { 1 }, 1);
00140     tikrinti(a == c, "operator== : tas pats vardas ir pavarde");
00141     tikrinti(a != b, "operator!= : skirtingas vardas ir pavarde");
00142
00143     //lygus sau
00144     tikrinti(a == a, "operator== : lygus sau");
00145     tikrinti(!(a != a), "operator!= : nelygus sau");
00146
00147     // <= ir >= su lygiais rezultatais
00148
00149     Studentas d("Dalia", "Dalaite", { 8, 9, 7 }, 10);
00150     d.skaiciuotiRez(1); //tas patas rezultatas kaip a, bet skirtingas vardas ir pavarde
00151     tikrinti(a <= d, "operator<= lygus rez (a <= d)");
00152     tikrinti(a >= d, "operator>= lygus rez (a >= d)");
00153
00154 }
00155
00156
00157
00158 static void testuoti_srautu_operatorius() {
00159     sekcija("Srautu operatoriai");
00160
00161     //operator» su istringstream
00162     Studentas a;
00163     std::istringstream iss("Jonas Jonaitis 8 9 7 10");
00164     iss >> a;
00165     tikrinti(a.getVardas() == "Jonas", "operator» vardas");
00166     tikrinti(a.getPavarde() == "Jonaitis", "operator» pavarde");
00167     tikrinti(a.getPazSkaicius() == 3, "operator» paz skaicius");
00168     tikrinti(a.getPaz()[0] == 8, "operator» pazymiai paz[0] == 8");
00169     tikrinti(a.getPaz()[2] == 7, "operator» pazymiai paz[2] == 7");

```

```

00170
00171 //operator» su vienu skaiciumi (tik egzaminas,0 namu darbu)
00172 Studentas b;
00173 std::istringstream iss2("Ona Onaite 5");
00174 iss2 » b;
00175 tikrinti(b.getEgz() == 5, "operator» egzaminas");
00176 tikrinti(b.getPazSkaicius() == 0, "operator» paz skaicius 0");
00177
00178 //operator« su ostringstream
00179 Studentas c("Petras", "Petraitis", { 6,8 }, 9);
00180 c.skaiciuotiRez(1);
00181 std::ostringstream oss;
00182 oss « c;
00183 std::string isvestis = oss.str();
00184
00185 tikrinti(isvestis.find("Petras") != std::string::npos, "operator«: yra vardas");
00186 tikrinti(isvestis.find("Petraitis") != std::string::npos, "operator«: yra pavarde");
00187 tikrinti(isvestis.find("Egz") != std::string::npos, "operator«: yra Egz.");
00188 tikrinti(isvestis.find("Rez") != std::string::npos, "operator«: yra Rez.");
00189
00190 // Apvalus testas: » tada «
00191 Studentas d;
00192 std::istringstream iss3("Inga Ingaite 7 8 9 6");
00193 iss3 » d;
00194 std::ostringstream oss2;
00195 oss2 « d;
00196 tikrinti(oss2.str().find("Inga") != std::string::npos,
00197 "operator« po »: vardas isvestyje");
00198
00199 }
00200
00201 static void testuoti_metodus() {
00202     sekciija("Metodai: vidurkis, mediana, skaiciuotiRez, islaike");
00203
00204     // vidurkis
00205     Studentas a("X", "Y", { 6, 8, 10 }, 0);
00206     tikrinti(std::abs(a.vidurkis() - 8.0) < 1e-9, "vidurkis(): (6+8+10)/3 = 8.0");
00207
00208     // vidurkis su tusciu sarasu
00209     Studentas b;
00210     tikrinti(b.vidurkis() == 0.0, "vidurkis(): tuscias sarasas = 0.0");
00211
00212     // mediana nelyginis skaicius
00213     Studentas c("X", "Y", { 3, 9, 5 }, 0);
00214     tikrinti(std::abs(c.mediana() - 5.0) < 1e-9, "mediana(): {3,9,5} -> 5.0");
00215
00216     // mediana lyginis skaicius
00217     Studentas d("X", "Y", { 2, 4, 6, 8 }, 0);
00218     tikrinti(std::abs(d.mediana() - 5.0) < 1e-9, "mediana(): {2,4,6,8} -> 5.0");
00219
00220     // skaiciuotiRez su vidurkiu
00221     Studentas e("X", "Y", { 8, 10, 6 }, 8);
00222     e.skaiciuotiRez(1); // vidurkis = 8, rez = 8*0.4 + 8*0.6 = 8.0
00223     tikrinti(std::abs(e.getRez() - 8.0) < 1e-9, "skaiciuotiRez(1): rez = 8.0");
00224
00225     // skaiciuotiRez su mediana
00226     Studentas f("X", "Y", { 4, 6, 8 }, 10);
00227     f.skaiciuotiRez(2); // mediana = 6, rez = 6*0.4 + 10*0.6 = 2.4+6.0 = 8.4
00228     tikrinti(std::abs(f.getRez() - 8.4) < 1e-9, "skaiciuotiRez(2): rez = 8.4");
00229
00230     // islaike
00231     Studentas g("X", "Y", {}, 0);
00232     g.setRez(4.9);
00233     tikrinti(!g.islaike(), "islaike(): 4.9 < 5.0 -> false");
00234     g.setRez(5.0);
00235     tikrinti(g.islaike(), "islaike(): 5.0 >= 5.0 -> true");
00236     g.setRez(8.5);
00237     tikrinti(g.islaike(), "islaike(): 8.5 >= 5.0 -> true");
00238
00239     // addPazymys ir clearPazymiai
00240     Studentas h;
00241     h.addPazymys(7);
00242     h.addPazymys(9);
00243     tikrinti(h.getPazSkaicius() == 2, "addPazymys(): 2 pazymiai");
00244     h.clearPazymiai();
00245     tikrinti(h.getPazSkaicius() == 0, "clearPazymiai(): sarasas tuscias");
00246
00247     // getteriai ir setteriai
00248     Studentas i;
00249     i.setVardas("Inga");
00250     i.setPavarde("Ingaite");
00251     i.setEgz(7);
00252     i.setRez(6.5);
00253     tikrinti(i.getVardas() == "Inga", "setter/getter: vardas");
00254     tikrinti(i.getPavarde() == "Ingaite", "setter/getter: pavarde");
00255     tikrinti(i.getEgz() == 7, "setter/getter: egz");
00256     tikrinti(std::abs(i.getRez() - 6.5) < 1e-9, "setter/getter: rez");

```

```

00257 }
00258
00259 static void testuoti_su_stl() {
00260     sekcija("Naudojimas su STL konteineriais ir algoritmais");
00261
00262     std::vector<Studentas> studentai = {
00263         Studentas("C", "C", {4,4}, 4),
00264         Studentas("A", "A", {9,9}, 9),
00265         Studentas("B", "B", {6,6}, 6)
00266     };
00267     for (auto& s : studentai) s.skaiciuotiRez(1);
00268
00269     // std::sort su operator<
00270     std::sort(studentai.begin(), studentai.end());
00271     tikrinti(studentai[0].getVardas() == "C" &&
00272         studentai[1].getVardas() == "B" &&
00273         studentai[2].getVardas() == "A",
00274         "std::sort su operator<: rusiuoja pagal rez_");
00275
00276     // std::find_if su islaike
00277     auto pirmas_islaikes = std::find_if(studentai.begin(), studentai.end(),
00278         [](const Studentas& s) { return s.islaike(); });
00279     tikrinti(pirmas_islaikes != studentai.end() &&
00280         pirmas_islaikes->getVardas() == "B",
00281         "std::find_if su islaike(): randa pirma islaikusi");
00282     // std::partition su operator>=
00283     std::vector<Studentas> v = studentai;
00284     auto pivot = std::partition(v.begin(), v.end(),
00285         [](const Studentas& s) { return s.islaike(); });
00286     int islaike = (int)std::distance(v.begin(), pivot);
00287     int neislaike = (int)std::distance(pivot, v.end());
00288     tikrinti(islaike == 2 && neislaike == 1,
00289         "std::partition + islaike(): 2 islaike, 1 neislaike");
00290 }
00291
00292 // PAGRINDINIS TESTO KVIETIMAS
00293 static void testuoti_zmogus_abstrakti() {
00294     sekcija("Zmogus abstrakti klase (v1.5)");
00295
00296     // Zmogus objekto sukurti NEGALIMA klase abstrakti.
00297     // Toliau pateiktas kodas NEKOMPILIUOJAS (tai yra tinkamas elgesys):
00298     //     Zmogus z; // KLAIDA: cannot instantiate abstract class
00299     //     Zmogus z("A", "B"); // KLAIDA: cannot instantiate abstract class
00300     // Tai irodo kad abstraktumas veikia teisingai.
00301
00302     // Galima sukurti tik Studentas objekta (isvestine klase):
00303     Studentas s("Jonas", "Jonaitis", { 8, 9 }, 10);
00304     tikrinti(true, "Zmogus abstrakti: Studentas objektas sukurtas sekmingai");
00305
00306     // Studentas yra Zmogus tipo (polimorfizmas)
00307     Zmogus* ptr = new Studentas("Ona", "Onaite", { 7, 8 }, 9);
00308     tikrinti(ptr != nullptr, "Zmogus abstrakti: Zmogus* rodo i Studentas objekta");
00309     tikrinti(ptr->getVardas() == "Ona", "Zmogus abstrakti: getVardas() per bazine klase");
00310     tikrinti(ptr->getPavarde() == "Onaite", "Zmogus abstrakti: getPavarde() per bazine klase");
00311
00312     // operator< per bazine klase naudoja virtual print()
00313     std::ostringstream oss;
00314     oss << *ptr;
00315     tikrinti(oss.str().find("Ona") != std::string::npos,
00316         "Zmogus abstrakti: operator< per Zmogus& naudoja virtual print()");
00317
00318     // Virtualus destruktorius teisingai iskvieciamas Studentas destruktorius
00319     delete ptr;
00320     tikrinti(true, "Zmogus abstrakti: virtualus destruktorius iskviestas teisingai");
00321
00322     // Bazines klases getteriai ir setteriai veikia per isvestine klase
00323     Studentas t;
00324     t.setVardas("Petras");
00325     t.setPavarde("Petraitis");
00326     tikrinti(t.getVardas() == "Petras", "Zmogus abstrakti: setVardas/getVardas is bazines");
00327     tikrinti(t.getPavarde() == "Petraitis", "Zmogus abstrakti: setPavarde/getPavarde is bazines");
00328 }
00329
00330 void atlikti_klases_testus() {
00331     std::cout << "\n";
00332     std::cout << std::string(55, '=') << "\n";
00333     std::cout << " STUDENTAS KLASES TESTAVIMAS (v1.2)\n";
00334     std::cout << std::string(55, '=') << "\n";
00335
00336     testuoti_konstruktorius();
00337     testuoti_priskyrima();
00338     testuoti_palyginimo_operatorius();
00339     testuoti_srautu_operatorius();
00340     testuoti_metodus();
00341     testuoti_su_stl();
00342     testuoti_zmogus_abstrakti();
00343 }

```

```

00344     std::cout << "\n" << std::string(55, '=') << "\n";
00345     std::cout << " Rezultatas: " << testai_pavyke << " / "
00346         << testai_is_viso << " testu pavyko\n";
00347     if (testai_pavyke == testai_is_viso)
00348         std::cout << " Visi testai pavyko!\n";
00349     else
00350         std::cout << " DEMESIO: "
00351             << (testai_is_viso - testai_pavyke)
00352             << " testai nepavyko!\n";
00353     std::cout << std::string(55, '=') << "\n";
00354 }

```

5.21 unit_tests.cpp Failo Nuoroda

```

#include <gtest/gtest.h>
#include "studentas.h"
#include "zmogus.h"
#include <vector>

```

Funkcijos

- [TEST](#) (RuleOfFive, DefaultConstructor)
- [TEST](#) (RuleOfFive, ParameterConstructor)
- [TEST](#) (RuleOfFive, CopyConstructor)
- [TEST](#) (RuleOfFive, MoveConstructor)
- [TEST](#) (RuleOfFive, CopyAssignment)
- [TEST](#) (RuleOfFive, MoveAssignment)
- [TEST](#) (RuleOfFive, SelfAssignment)
- [TEST](#) (Functional, AverageAndMedian)
- [TEST](#) (Functional, CalculateResultMean)
- [TEST](#) (Functional, CalculateResultMedian)
- [TEST](#) (Functional, IsPassed)
- [TEST](#) (AbstractZmogus, Polymorphism)
- [TEST](#) (AbstractZmogus, OutputOperatorUsesVirtualPrint)
- [int main](#) (int argc, char **argv)

5.21.1 Funkcijos Dokumentacija

5.21.1.1 main()

```

int main (
    int argc,
    char ** argv)

```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 114.

```

00114     {
00115         ::testing::InitGoogleTest(&argc, argv);
00116         return RUN_ALL_TESTS();
00117     }

```

5.21.1.2 TEST() [1/13]

```
TEST (
    AbstractZmogus ,
    OutputOperatorUsesVirtualPrint )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 104.

```
00104 {
00105     Studentas s("Vardas", "Pavarde", { 5,5 }, 5);
00106     s.skaiciuotiRez(1);
00107     std::ostringstream oss;
00108     oss << s;
00109     std::string out = oss.str();
00110     EXPECT_NE(out.find("Vardas"), std::string::npos);
00111     EXPECT_NE(out.find("Rez"), std::string::npos);
00112 }
```

5.21.1.3 TEST() [2/13]

```
TEST (
    AbstractZmogus ,
    Polymorphism )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 98.

```
00098 {
00099     Zmogus* ptr = new Studentas("Polimorfas", "Test", { 1,2 }, 3);
00100     EXPECT_EQ(ptr->getVardas(), "Polimorfas");
00101     delete ptr; // virtualus destruktorius
00102 }
```

5.21.1.4 TEST() [3/13]

```
TEST (
    Functional ,
    AverageAndMedian )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 68.

```
00068 {
00069     Studentas s("T", "T", { 4,6,8,10 }, 0);
00070     EXPECT_DOUBLE_EQ(s.vidurkis(), 7.0);
00071     EXPECT_DOUBLE_EQ(s.mediana(), 7.0); // (6+8)/2 = 7
00072 }
```

5.21.1.5 TEST() [4/13]

```
TEST (
    Functional ,
    CalculateResultMean )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 74.

```
00074 {
00075     Studentas s("X", "X", { 8,10,6 }, 8); // vidurkis = 8
00076     s.skaiciuotiRez(1);
00077     // 8*0.4 + 8*0.6 = 8.0
00078     EXPECT_DOUBLE_EQ(s.getRez(), 8.0);
00079 }
```

5.21.1.6 TEST() [5/13]

```
TEST (
    Functional ,
    CalculateResultMedian )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 81.

```
00081 {
00082     Studentas s("Y", "Y", { 4,6,8 }, 10); // mediana = 6
00083     s.skaiciuotiRez(2);
00084     // 6*0.4 + 10*0.6 = 2.4+6 = 8.4
00085     EXPECT_DOUBLE_EQ(s.getRez(), 8.4);
00086 }
```

5.21.1.7 TEST() [6/13]

```
TEST (
    Functional ,
    IsPassed )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 88.

```
00088 {
00089     Studentas s;
00090     s.setRez(4.9);
00091     EXPECT_FALSE(s.islaike());
00092     s.setRez(5.0);
00093     EXPECT_TRUE(s.islaike());
00094 }
```

5.21.1.8 TEST() [7/13]

```
TEST (
    RuleOfFive ,
    CopyAssignment )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 43.

```
00043 {
00044     Studentas a("Kazys", "Kazlauskas", { 7,8 }, 9);
00045     Studentas b;
00046     b = a;
00047     EXPECT_EQ(b.getVardas(), "Kazys");
00048     a.setVardas("Laimis");
00049     EXPECT_EQ(b.getVardas(), "Kazys");
00050 }
```

5.21.1.9 TEST() [8/13]

```
TEST (
    RuleOfFive ,
    CopyConstructor )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 26.

```
00026 {
00027     Studentas a("Petras", "Petraitis", { 5,6 }, 7);
00028     Studentas b(a);
00029     EXPECT_EQ(b.getVardas(), "Petras");
00030     EXPECT_EQ(b.getPavarde(), "Petraitis");
00031     b.setVardas("Antanas");
00032     EXPECT_EQ(a.getVardas(), "Petras"); // gili kopija
00033 }
```

5.21.1.10 TEST() [9/13]

```
TEST (
    RuleOfFive ,
    DefaultConstructor )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 8.

```
00008                                     {
00009     Studentas s;
00010     EXPECT_EQ(s.getVardas(), "");
00011     EXPECT_EQ(s.getPavarde(), "");
00012     EXPECT_EQ(s.getEgz(), 0);
00013     EXPECT_DOUBLE_EQ(s.getRez(), 0.0);
00014     EXPECT_EQ(s.getPazSkaicius(), 0u);
00015 }
```

5.21.1.11 TEST() [10/13]

```
TEST (
    RuleOfFive ,
    MoveAssignment )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 52.

```
00052                                     {
00053     Studentas a("Rita", "Ritaite", { 6,7 }, 8);
00054     Studentas b;
00055     b = std::move(a);
00056     EXPECT_EQ(b.getVardas(), "Rita");
00057     EXPECT_EQ(a.getVardas(), "");
00058 }
```

5.21.1.12 TEST() [11/13]

```
TEST (
    RuleOfFive ,
    MoveConstructor )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 35.

```
00035                                     {
00036     Studentas a("Ona", "Onaite", { 9,10 }, 8);
00037     Studentas b(std::move(a));
00038     EXPECT_EQ(b.getVardas(), "Ona");
00039     EXPECT_EQ(a.getVardas(), ""); // originalas tuscias
00040     EXPECT_EQ(a.getPazSkaicius(), 0u);
00041 }
```

5.21.1.13 TEST() [12/13]

```
TEST (
    RuleOfFive ,
    ParameterConstructor )
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 17.

```
00017                                     {
00018     Studentas s("Jonas", "Jonaitis", { 8,9,7 }, 10);
00019     EXPECT_EQ(s.getVardas(), "Jonas");
00020     EXPECT_EQ(s.getPavarde(), "Jonaitis");
00021     EXPECT_EQ(s.getEgz(), 10);
00022     EXPECT_EQ(s.getPazSkaicius(), 3u);
00023     EXPECT_EQ(s.getPaz()[0], 8);
00024 }
```

5.21.1.14 TEST() [13/13]

```
TEST (
    RuleOfFive ,
    SelfAssignment )
```

Apibūrinimas failo `unit_tests.cpp` eilutėje 60.

```
00060 {
00061     Studentas a("A", "A", { 1 }, 2);
00062     a = a;
00063     EXPECT_EQ(a.getVardas(), "A");
00064 }
```

5.22 unit_tests.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include <gtest/gtest.h>
00002 #include "studentas.h"
00003 #include "zmogus.h"
00004 #include <vector>
00005
00006 // 1. Rule of Five testai
00007
00008 TEST(RuleOfFive, DefaultConstructor) {
00009     Studentas s;
00010     EXPECT_EQ(s.getVardas(), "");
00011     EXPECT_EQ(s.getPavarde(), "");
00012     EXPECT_EQ(s.getEgz(), 0);
00013     EXPECT_DOUBLE_EQ(s.getRez(), 0.0);
00014     EXPECT_EQ(s.getPazSkaicius(), 0u);
00015 }
00016
00017 TEST(RuleOfFive, ParameterConstructor) {
00018     Studentas s("Jonas", "Jonaitis", { 8,9,7 }, 10);
00019     EXPECT_EQ(s.getVardas(), "Jonas");
00020     EXPECT_EQ(s.getPavarde(), "Jonaitis");
00021     EXPECT_EQ(s.getEgz(), 10);
00022     EXPECT_EQ(s.getPazSkaicius(), 3u);
00023     EXPECT_EQ(s.getPaz()[0], 8);
00024 }
00025
00026 TEST(RuleOfFive, CopyConstructor) {
00027     Studentas a("Petras", "Petraitis", { 5,6 }, 7);
00028     Studentas b(a);
00029     EXPECT_EQ(b.getVardas(), "Petras");
00030     EXPECT_EQ(b.getPavarde(), "Petraitis");
00031     b.setVardas("Antanas");
00032     EXPECT_EQ(a.getVardas(), "Petras"); // gili kopija
00033 }
00034
00035 TEST(RuleOfFive, MoveConstructor) {
00036     Studentas a("Ona", "Onaite", { 9,10 }, 8);
00037     Studentas b(std::move(a));
00038     EXPECT_EQ(b.getVardas(), "Ona");
00039     EXPECT_EQ(a.getVardas(), ""); // originalas tuscias
00040     EXPECT_EQ(a.getPazSkaicius(), 0u);
00041 }
00042
00043 TEST(RuleOfFive, CopyAssignment) {
00044     Studentas a("Kazys", "Kazlauskas", { 7,8 }, 9);
00045     Studentas b;
00046     b = a;
00047     EXPECT_EQ(b.getVardas(), "Kazys");
00048     a.setVardas("Laimis");
00049     EXPECT_EQ(b.getVardas(), "Kazys");
00050 }
00051
00052 TEST(RuleOfFive, MoveAssignment) {
00053     Studentas a("Rita", "Ritaite", { 6,7 }, 8);
00054     Studentas b;
00055     b = std::move(a);
00056     EXPECT_EQ(b.getVardas(), "Rita");
00057     EXPECT_EQ(a.getVardas(), "");
00058 }
00059
00060 TEST(RuleOfFive, SelfAssignment) {
```



```

00061     Studentas a("A", "A", { 1 }, 2);
00062     a = a;
00063     EXPECT_EQ(a.getVardas(), "A");
00064 }
00065
00066 // 2. Kiti funkciniai testai
00067
00068 TEST(Functional, AverageAndMedian) {
00069     Studentas s("T", "T", { 4,6,8,10 }, 0);
00070     EXPECT_DOUBLE_EQ(s.vidurkis(), 7.0);
00071     EXPECT_DOUBLE_EQ(s.mediana(), 7.0);    // (6+8)/2 = 7
00072 }
00073
00074 TEST(Functional, CalculateResultMean) {
00075     Studentas s("X", "X", { 8,10,6 }, 8); // vidurkis = 8
00076     s.skaiciuotiRez(1);
00077     // 8*0.4 + 8*0.6 = 8.0
00078     EXPECT_DOUBLE_EQ(s.getRez(), 8.0);
00079 }
00080
00081 TEST(Functional, CalculateResultMedian) {
00082     Studentas s("Y", "Y", { 4,6,8 }, 10); // mediana = 6
00083     s.skaiciuotiRez(2);
00084     // 6*0.4 + 10*0.6 = 2.4+6 = 8.4
00085     EXPECT_DOUBLE_EQ(s.getRez(), 8.4);
00086 }
00087
00088 TEST(Functional, IsPassed) {
00089     Studentas s;
00090     s.setRez(4.9);
00091     EXPECT_FALSE(s.islaike());
00092     s.setRez(5.0);
00093     EXPECT_TRUE(s.islaike());
00094 }
00095
00096 // 3. Abstraktumo ir polimorfizmo testai
00097
00098 TEST(AbstractZmogus, Polymorphism) {
00099     Zmogus* ptr = new Studentas("Polimorfas", "Test", { 1,2 }, 3);
00100     EXPECT_EQ(ptr->getVardas(), "Polimorfas");
00101     delete ptr;    // virtualus destruktorius
00102 }
00103
00104 TEST(AbstractZmogus, OutputOperatorUsesVirtualPrint) {
00105     Studentas s("Vardas", "Pavarde", { 5,5 }, 5);
00106     s.skaiciuotiRez(1);
00107     std::ostringstream oss;
00108     oss << s;
00109     std::string out = oss.str();
00110     EXPECT_NE(out.find("Vardas"), std::string::npos);
00111     EXPECT_NE(out.find("Rez"), std::string::npos);
00112 }
00113
00114 int main(int argc, char** argv) {
00115     ::testing::InitGoogleTest(&argc, argv);
00116     return RUN_ALL_TESTS();
00117 }

```

5.23 utils.cpp Failo Nuoroda

```

#include "utils.h"
#include <iostream>
#include <limits>

```

Funkcijos

- std::string [investi_varda_ar_pavarde](#) (const std::string &zinute)
- int [investies_tikrinimas](#) (const std::string &zinute)
- int [investies_skaicius](#) (const std::string &zinute)
- int [skaiciavimo_metodas](#) ()
- double [galutinis_vid](#) (double x, int egz)

5.23.1 Funkcijos Dokumentacija

5.23.1.1 galutinis_vid()

```
double galutinis_vid (  
    double x,  
    int egz)
```

Apibrėžimas failo [utils.cpp](#) eilutėje 90.

```
00091 {  
00092     return x * 0.4 + egz * 0.6;  
00093 }
```

5.23.1.2 ivesti_varda_ar_pavarde()

```
std::string ivesti_varda_ar_pavarde (  
    const std::string & zinute)
```

Apibrėžimas failo [utils.cpp](#) eilutėje 5.

```
00006 {  
00007     std::string tekstas;  
00008     while (true) {  
00009         std::cout << zinute;  
00010         std::cin >> tekstas;  
00011         if (!tekstas.empty()) {  
00012             return tekstas;  
00013         }  
00014         std::cout << "Ivestis negali buti tuscia\n";  
00015     }  
00016 }
```

5.23.1.3 ivesties_skaicius()

```
int ivesties_skaicius (  
    const std::string & zinute)
```

Apibrėžimas failo [utils.cpp](#) eilutėje 44.

```
00045 {  
00046     int value;  
00047     while (true)  
00048     {  
00049         std::cout << zinute;  
00050         std::cin >> value;  
00051         if (std::cin.fail())  
00052         {  
00053             std::cout << "Ivestis turi buti skaicius, bandykite dar karta\n";  
00054             std::cin.clear();  
00055             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
00056             continue;  
00057         }  
00058         if (value >= 1)  
00059         {  
00060             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
00061             return value;  
00062         }  
00063     }  
00064     std::cout << "Ivestis turi buti teigiamas skaicius\n";  
00065     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
00066 }  
00067 }  
00068 }
```

5.23.1.4 ivesties_tikrinimas()

```
int ivesties_tikrinimas (
    const std::string & zinute)
```

Apibrėžimas failo [utils.cpp](#) eilutėje 18.

```
00019 {
00020     int value;
00021     while (true)
00022     {
00023         std::cout << zinute;
00024         std::cin >> value;
00025         if (std::cin.fail())
00026         {
00027             std::cout << "Ivestis turi buti skaicius, bandykite dar karta\n";
00028             std::cin.clear();
00029             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00030             continue;
00031         }
00032
00033         if (value == -1 || (value >= 0 && value <= 10))
00034         {
00035             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00036             return value;
00037         }
00038
00039         std::cout << "Ivestis turi buti tarp 0 ir 10 (arba -1 baigti)\n";
00040         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00041     }
00042 }
```

5.23.1.5 skaiciavimo_metodas()

```
int skaiciavimo_metodas ()
```

Apibrėžimas failo [utils.cpp](#) eilutėje 70.

```
00071 {
00072     int metodas;
00073     while (true) {
00074         std::cout << "Pasirinkite skaiciavimo metoda (1 - vidurkis, 2 - mediana): ";
00075         if (!(std::cin >> metodas)) {
00076             std::cout << "Ivestis turi buti skaicius 1 arba 2, bandykite dar karta\n";
00077             std::cin.clear();
00078             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00079             continue;
00080         }
00081         if (metodas == 1 || metodas == 2) {
00082             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00083             return metodas;
00084         }
00085         std::cout << "Neteisingas pasirinkimas. Pasirinkite 1 arba 2.\n";
00086         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00087     }
00088 }
```

5.24 utils.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "utils.h"
00002 #include <iostream>
00003 #include <limits>
00004
00005 std::string ivesti_varda_ar_pavarde(const std::string& zinute)
00006 {
00007     std::string tekstas;
00008     while (true) {
00009         std::cout << zinute;
00010         std::cin >> tekstas;
00011         if (!tekstas.empty()) {
00012             return tekstas;
00013         }
00014     }
```

```

00014         std::cout << "Ivestis negali buti tuscia\n";
00015     }
00016 }
00017
00018 int ivesties_tikrinimas(const std::string& zinute)
00019 {
00020     int value;
00021     while (true)
00022     {
00023         std::cout << zinute;
00024         std::cin >> value;
00025         if (std::cin.fail())
00026         {
00027             std::cout << "Ivestis turi buti skaicius, bandykite dar karta\n";
00028             std::cin.clear();
00029             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00030             continue;
00031         }
00032
00033         if (value == -1 || (value >= 0 && value <= 10))
00034         {
00035             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00036             return value;
00037         }
00038
00039         std::cout << "Ivestis turi buti tarp 0 ir 10 (arba -1 baigti)\n";
00040         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00041     }
00042 }
00043
00044 int ivesties_skaicius(const std::string& zinute)
00045 {
00046     int value;
00047     while (true)
00048     {
00049         std::cout << zinute;
00050         std::cin >> value;
00051         if (std::cin.fail())
00052         {
00053             std::cout << "Ivestis turi buti skaicius, bandykite dar karta\n";
00054             std::cin.clear();
00055             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00056             continue;
00057         }
00058
00059         if (value >= 1)
00060         {
00061             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00062             return value;
00063         }
00064
00065         std::cout << "Ivestis turi buti teigiamas skaicius\n";
00066         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00067     }
00068 }
00069
00070 int skaiciavimo_metodas()
00071 {
00072     int metodas;
00073     while (true) {
00074         std::cout << "Pasirinkite skaiciavimo metoda (1 - vidurkis, 2 - mediana): ";
00075         if (!(std::cin >> metodas)) {
00076             std::cout << "Ivestis turi buti skaicius 1 arba 2, bandykite dar karta\n";
00077             std::cin.clear();
00078             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00079             continue;
00080         }
00081         if (metodas == 1 || metodas == 2) {
00082             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00083             return metodas;
00084         }
00085         std::cout << "Neteisingas pasirinkimas. Pasirinkite 1 arba 2.\n";
00086         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00087     }
00088 }
00089
00090 double galutinis_vid(double x, int egz)
00091 {
00092     return x * 0.4 + egz * 0.6;
00093 }

```

5.25 utils.h Failo Nuoroda

```
#include <string>
```

Funkcijos

- `std::string ivesiti_varda_ar_pavarde` (`const std::string &zinute`)
- `int ivesties_tikrinimas` (`const std::string &zinute`)
- `int ivesties_skaicius` (`const std::string &zinute`)
- `int skaiciavimo_metodas` ()

5.25.1 Funkcijos Dokumentacija

5.25.1.1 ivesiti_varda_ar_pavarde()

```
std::string ivesiti_varda_ar_pavarde (  
    const std::string & zinute)
```

Apibrėžimas failo `utils.cpp` eilutėje 5.

```
00006 {  
00007     std::string tekstas;  
00008     while (true) {  
00009         std::cout << zinute;  
00010         std::cin >> tekstas;  
00011         if (!tekstas.empty()) {  
00012             return tekstas;  
00013         }  
00014         std::cout << "Ivestis negali buti tuscia\n";  
00015     }  
00016 }
```

5.25.1.2 ivesties_skaicius()

```
int ivesties_skaicius (  
    const std::string & zinute)
```

Apibrėžimas failo `utils.cpp` eilutėje 44.

```
00045 {  
00046     int value;  
00047     while (true)  
00048     {  
00049         std::cout << zinute;  
00050         std::cin >> value;  
00051         if (std::cin.fail())  
00052         {  
00053             std::cout << "Ivestis turi buti skaicius, bandykite dar karta\n";  
00054             std::cin.clear();  
00055             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
00056             continue;  
00057         }  
00058  
00059         if (value >= 1)  
00060         {  
00061             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
00062             return value;  
00063         }  
00064  
00065         std::cout << "Ivestis turi buti teigiamas skaicius\n";  
00066         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
00067     }  
00068 }
```

5.25.1.3 ivesties_tikrinimas()

```
int ivesties_tikrinimas (
    const std::string & zinate)
```

Apibrėžimas failo [utils.cpp](#) eilutėje 18.

```
00019 {
00020     int value;
00021     while (true)
00022     {
00023         std::cout << zinate;
00024         std::cin >> value;
00025         if (std::cin.fail())
00026         {
00027             std::cout << "Ivestis turi buti skaicius, bandykite dar karta\n";
00028             std::cin.clear();
00029             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00030             continue;
00031         }
00032
00033         if (value == -1 || (value >= 0 && value <= 10))
00034         {
00035             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00036             return value;
00037         }
00038
00039         std::cout << "Ivestis turi buti tarp 0 ir 10 (arba -1 baigti)\n";
00040         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00041     }
00042 }
```

5.25.1.4 skaiciavimo_metodas()

```
int skaiciavimo_metodas ()
```

Apibrėžimas failo [utils.cpp](#) eilutėje 70.

```
00071 {
00072     int metodas;
00073     while (true) {
00074         std::cout << "Pasirinkite skaiciavimo metoda (1 - vidurkis, 2 - mediana): ";
00075         if (!(std::cin >> metodas)) {
00076             std::cout << "Ivestis turi buti skaicius 1 arba 2, bandykite dar karta\n";
00077             std::cin.clear();
00078             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00079             continue;
00080         }
00081         if (metodas == 1 || metodas == 2) {
00082             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00083             return metodas;
00084         }
00085         std::cout << "Neteisingas pasirinkimas. Pasirinkite 1 arba 2.\n";
00086         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00087     }
00088 }
```

5.26 utils.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef UTILS_H
00002 #define UTILS_H
00003
00004 #include <string>
00005
00006 std::string ivesti_varda_ar_pavarde(const std::string& zinate);
00007 int ivesties_tikrinimas(const std::string& zinate);
00008 int ivesties_skaicius(const std::string& zinate);
00009 int skaiciavimo_metodas();
00010
00011 #endif
```

5.27 vector.cpp Failo Nuoroda

5.28 vector.cpp

[Eiti į šio failo dokumentaciją.](#)

00001

5.29 vector.h Failo Nuoroda

Nuosavos [Vector](#) konteinerio klasės aprašas.

```
#include <memory>
#include <stdexcept>
#include <algorithm>
#include <iterator>
#include <initializer_list>
#include <limits>
#include <cassert>
```

Klasės

- class [Vector](#)< T, Allocator >
Nuosava [Vector](#) konteinerio klasė.

Funkcijos

- template<typename T, typename A>
bool [operator==](#) (const [Vector](#)< T, A > &lhs, const [Vector](#)< T, A > &rhs)
Lygybės operatorius.
- template<typename T, typename A>
bool [operator!=](#) (const [Vector](#)< T, A > &lhs, const [Vector](#)< T, A > &rhs)
Nelygybės operatorius.
- template<typename T, typename A>
bool [operator<](#) (const [Vector](#)< T, A > &lhs, const [Vector](#)< T, A > &rhs)
Leksikografinis < palyginimas.
- template<typename T, typename A>
bool [operator<=](#) (const [Vector](#)< T, A > &lhs, const [Vector](#)< T, A > &rhs)
Leksikografinis <= palyginimas.
- template<typename T, typename A>
bool [operator>](#) (const [Vector](#)< T, A > &lhs, const [Vector](#)< T, A > &rhs)
Leksikografinis > palyginimas.
- template<typename T, typename A>
bool [operator>=](#) (const [Vector](#)< T, A > &lhs, const [Vector](#)< T, A > &rhs)
Leksikografinis >= palyginimas.
- template<typename T, typename A>
void [swap](#) ([Vector](#)< T, A > &lhs, [Vector](#)< T, A > &rhs) noexcept
Globalus swap (ADL).
- template<typename T, typename A, typename U>
[Vector](#)< T, A >::size_type [erase](#) ([Vector](#)< T, A > &c, const U &value)
Ištrina elementus pagal reikšmę (C++20 stilius).
- template<typename T, typename A, typename Pred>
[Vector](#)< T, A >::size_type [erase_if](#) ([Vector](#)< T, A > &c, Pred pred)
Ištrina elementus pagal predikato sąlygą (C++20 stilius).

5.29.1 Smulkus aprašymas

Nuosavos [Vector](#) konteinerio klasės aprašas.

Pilnavertė `std::vector` alternatyva, realizuojanti visus Member types, Member functions ir Non-member functions pagal: <https://en.cppreference.com/w/cpp/container/vector>

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [vector.h](#).

5.29.2 Funkcijos Dokumentacija

5.29.2.1 erase()

```
template<typename T, typename A, typename U>
Vector< T, A >::size_type erase (
    Vector< T, A > & c,
    const U & value)
```

Ištrina elementus pagal reikšmę (C++20 stilius).

Apibrėžimas failo [vector.h](#) eilutėje 622.

```
00622 {
00623     auto it = std::remove(c.begin(), c.end(), value);
00624     auto n = c.end() - it;
00625     c.erase(it, c.end());
00626     return static_cast<typename Vector<T, A>::size_type>(n);
00627 }
```

5.29.2.2 erase_if()

```
template<typename T, typename A, typename Pred>
Vector< T, A >::size_type erase_if (
    Vector< T, A > & c,
    Pred pred)
```

Ištrina elementus pagal predikato sąlygą (C++20 stilius).

Apibrėžimas failo [vector.h](#) eilutėje 631.

```
00631 {
00632     auto it = std::remove_if(c.begin(), c.end(), pred);
00633     auto n = c.end() - it;
00634     c.erase(it, c.end());
00635     return static_cast<typename Vector<T, A>::size_type>(n);
00636 }
```


5.29.2.3 operator!=(())

```
template<typename T, typename A>
bool operator!=(
    const Vector< T, A > & lhs,
    const Vector< T, A > & rhs)
```

Nelygybės operatorius.

Apibrėžimas failo [vector.h](#) eilutėje 585.

```
00585                                     {
00586     return !(lhs == rhs);
00587 }
```

5.29.2.4 operator<()

```
template<typename T, typename A>
bool operator< (
    const Vector< T, A > & lhs,
    const Vector< T, A > & rhs)
```

Leksikografinis < palyginimas.

Apibrėžimas failo [vector.h](#) eilutėje 591.

```
00591                                     {
00592     return std::lexicographical_compare(lhs.begin(), lhs.end(),
00593         rhs.begin(), rhs.end());
00594 }
```

5.29.2.5 operator<=()

```
template<typename T, typename A>
bool operator<= (
    const Vector< T, A > & lhs,
    const Vector< T, A > & rhs)
```

Leksikografinis <= palyginimas.

Apibrėžimas failo [vector.h](#) eilutėje 598.

```
00598                                     {
00599     return !(rhs < lhs);
00600 }
```

5.29.2.6 operator==(())

```
template<typename T, typename A>
bool operator==(
    const Vector< T, A > & lhs,
    const Vector< T, A > & rhs)
```

Lygybės operatorius.

Apibrėžimas failo [vector.h](#) eilutėje 578.

```
00578                                     {
00579     if (lhs.size() != rhs.size()) return false;
00580     return std::equal(lhs.begin(), lhs.end(), rhs.begin());
00581 }
```

5.29.2.7 operator>()

```
template<typename T, typename A>
bool operator> (
    const Vector< T, A > & lhs,
    const Vector< T, A > & rhs)
```

Leksikografinis > palyginimas.

Apibrėžimas failo [vector.h](#) eilutėje 604.

```
00604                                     {
00605     return rhs < lhs;
00606 }
```

5.29.2.8 operator>=()

```
template<typename T, typename A>
bool operator>= (
    const Vector< T, A > & lhs,
    const Vector< T, A > & rhs)
```

Leksikografinis >= palyginimas.

Apibrėžimas failo [vector.h](#) eilutėje 610.

```
00610                                     {
00611     return !(lhs < rhs);
00612 }
```

5.29.2.9 swap()

```
template<typename T, typename A>
void swap (
    Vector< T, A > & lhs,
    Vector< T, A > & rhs) [noexcept]
```

Globalus swap (ADL).

Apibrėžimas failo [vector.h](#) eilutėje 616.

```
00616                                     {
00617     lhs.swap(rhs);
00618 }
```

5.30 vector.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 /**
00005  * @file Vector.h
00006  * @brief Nuosavos Vector konteinerio klasės aprašas.
00007  *
00008  * Pilnavertė std::vector alternatyva, realizuojanti visus
00009  * Member types, Member functions ir Non-member functions pagal:
00010  * https://en.cppreference.com/w/cpp/container/vector
00011  *
00012  * @author Studentas
00013  * @version 3.0
```

```

00014  */
00015
00016 #include <memory>
00017 #include <stdexcept>
00018 #include <algorithm>
00019 #include <iterator>
00020 #include <initializer_list>
00021 #include <limits>
00022 #include <cassert>
00023
00024 /**
00025  * @brief Nuosava Vector konteinerio klasė.
00026  *
00027  * Dinaminės atminties masyvas su automatine atminties valdymu.
00028  * Veikia kaip pilnavertis std::vector pakaitalas.
00029  *
00030  * @tparam T Elementų tipas
00031  * @tparam Allocator Atminties paskirstytojas (pagal nutylėjamą std::allocator<T>)
00032  */
00033 template<typename T, typename Allocator = std::allocator<T>
00034 class Vector {
00035 public:
00036     // Member types
00037     using value_type = T; //< Elementų tipas
00038     using allocator_type = Allocator; //< Atminties paskirstytojo tipas
00039     using size_type = std::size_t; //< Dydžio tipas
00040     using difference_type = std::ptrdiff_t; //< Skirtumo tipas
00041     using reference = T&; //< Elementų nuorodų tipas
00042     using const_reference = const T&; //< Konstantiškų elementų nuorodų tipas
00043     using pointer = typename std::allocator_traits<Allocator>::pointer; //< Elementų rodyklės tipas
00044     using const_pointer = typename std::allocator_traits<Allocator>::const_pointer; //< Konstantiškų
    elementų rodyklės tipas
00045     using iterator = T*; //< Elementų iteratoriaus tipas
00046     using const_iterator = const T*; //< Konstantiškų elementų iteratoriaus tipas
00047     using reverse_iterator = std::reverse_iterator<iterator>; //< Atvirkštinių elementų iteratoriaus
    tipas
00048     using const_reverse_iterator = std::reverse_iterator<const_iterator>; //< Atvirkštinių
    konstantiškų elementų iteratoriaus tipas
00049
00050 private:
00051     pointer data_; //< Rodyklė į duomenų masyvą
00052     size_type size_; //< Dabartinis elementų skaičius
00053     size_type capacity_; //< Maksimalus galimas elementų skaičius be perskirstymo
00054     Allocator alloc_; //< Atminties paskirstytojas
00055
00056     using AllocTraits = std::allocator_traits<Allocator>;
00057
00058     /**
00059      * @brief Atlaisvina visą naudojamą atmintį.
00060      */
00061     void deallocate_all() {
00062         if (data_) {
00063             for (size_type i = 0; i < size_; ++i)
00064                 AllocTraits::destroy(alloc_, data_ + i);
00065             AllocTraits::deallocate(alloc_, data_, capacity_);
00066             data_ = nullptr;
00067         }
00068         size_ = 0;
00069         capacity_ = 0;
00070     }
00071
00072     /**
00073      * @brief Perskirsto atmintį naujam dydžiui.
00074      * @param new_cap Naujas capacity dydis
00075      */
00076     void reallocate(size_type new_cap) {
00077         pointer new_data = AllocTraits::allocate(alloc_, new_cap);
00078         size_type i = 0;
00079         try {
00080             for (; i < size_; ++i)
00081                 AllocTraits::construct(alloc_, new_data + i, std::move_if_noexcept(data_[i]));
00082         } catch (...) {
00083             for (size_type j = 0; j < i; ++j)
00084                 AllocTraits::destroy(alloc_, new_data + j);
00085             AllocTraits::deallocate(alloc_, new_data, new_cap);
00086             throw;
00087         }
00088         deallocate_all();
00089         data_ = new_data;
00090         size_ = i;
00091         capacity_ = new_cap;
00092     }
00093 public:
00094     // Constructors
00095
00096     /**
00097      * @brief Numatytasis konstruktorius. Sukuria tuščią vektorių.

```

```

00098     */
00099 Vector() noexcept : data_(nullptr), size_(0), capacity_(0), alloc_() {}
00100
00101 /**
00102  * @brief Konstruktorius su paskirstytoju.
00103  * @param alloc Naudojamas paskirstytojas
00104  */
00105
00106 explicit Vector(const Allocator& alloc) noexcept
00107     : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {}
00108
00109
00110 /**
00111  * @brief Konstruktorius su dydžiu.
00112  * @param count Elementų skaičius
00113  * @param value Pradinė reikšmė (pagal nutylėjimą - numatytoji)
00114  * @param alloc Paskirstytojas
00115  */
00116 explicit Vector(size_type count, const T& value = T(),
00117     const Allocator& alloc = Allocator())
00118     : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00119     assign(count, value);
00120 }
00121
00122 /**
00123  * @brief Konstruktorius iš iteratorių diapazono.
00124  * @param first Pradžios iteratorius
00125  * @param last Pabaigos iteratorius
00126  * @param alloc Paskirstytojas
00127  */
00128
00129 template<typename InputIt,
00130     typename = std::enable_if_t<std::is_base_of_v<
00131         std::input_iterator_tag,
00132         typename std::iterator_traits<InputIt>::iterator_category>>
00133 Vector(InputIt first, InputIt last, const Allocator& alloc = Allocator())
00134     : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00135     assign(first, last);
00136 }
00137
00138 /**
00139  * @brief Kopijos konstruktorius.
00140  * @param other Kopijuojamas vektorius
00141  */
00142 Vector(const Vector& other)
00143     : data_(nullptr), size_(0), capacity_(0),
00144     alloc_(AllocTraits::select_on_container_copy_construction(other.alloc_)) {
00145     assign(other.begin(), other.end());
00146 }
00147
00148 /**
00149  * @brief Perkėlimo konstruktorius.
00150  * @param other Perkeliamas vektorius
00151  */
00152
00153 Vector(Vector&& other) noexcept
00154     : data_(other.data_), size_(other.size_), capacity_(other.capacity_),
00155     alloc_(std::move(other.alloc_)) {
00156     other.data_ = nullptr;
00157     other.size_ = 0;
00158     other.capacity_ = 0;
00159 }
00160
00161 /**
00162  * @brief Konstruktorius iš std::initializer_list.
00163  * @param il Pradinių reikšmių sąrašas
00164  * @param alloc Paskirstytojas
00165  */
00166 Vector(std::initializer_list<T> il, const Allocator& alloc = Allocator())
00167     : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00168     assign(il.begin(), il.end());
00169 }
00170
00171 /**
00172  * @brief Destruktorius. Atlaisvina visą atmintį.
00173  */
00174 ~Vector() { deallocate_all(); }
00175
00176 // Priskyrimo operatoriai
00177
00178 /**
00179  * @brief Kopijos priskyrimo operatorius.
00180  */
00181 Vector& operator=(const Vector& other) {
00182     if (this == &other) return *this;
00183     if constexpr (AllocTraits::propagate_on_container_copy_assignment::value)
00184         alloc_ = other.alloc_; // <- Čia ištaisyta klaida (buvo alloc = ...)

```

```

00185         assign(other.begin(), other.end());
00186         return *this;
00187     }
00188
00189     /**
00190     * @brief Perkélimo priskyrimo operatorius.
00191     */
00192
00193     Vector& operator=(Vector&& other) noexcept {
00194         if (this == &other) return *this;
00195         deallocate_all();
00196         if constexpr (AllocTraits::propagate_on_container_move_assignment::value)
00197             alloc_ = std::move(other.alloc_);
00198         data_ = other.data_;
00199         size_ = other.size_;
00200         capacity_ = other.capacity_;
00201         other.data_ = nullptr;
00202         other.size_ = 0;
00203         other.capacity_ = 0;
00204         return *this;
00205     }
00206
00207     /**
00208     * @brief Priskyrimo operatorius iš initializer_list.
00209     */
00210     Vector& operator=(std::initializer_list<T> il) {
00211         assign(il.begin(), il.end());
00212         return *this;
00213     }
00214
00215     // priskyrimo funkcijos
00216
00217     /**
00218     * @brief Priskiria count kopijų reikšmės value.
00219     * @param count Elementų skaičius
00220     * @param value Reikšmė
00221     */
00222     void assign(size_type count, const T& value) {
00223         clear();
00224         reserve(count);
00225         for (size_type i = 0; i < count; ++i)
00226             AllocTraits::construct(alloc_, data_ + i, value);
00227         size_ = count;
00228     }
00229
00230     /**
00231     * @brief Priskiria elementus iš iteratorių diapazono.
00232     */
00233     template<typename InputIt,
00234             typename = std::enable_if_t<std::is_base_of_v<
00235                 std::input_iterator_tag,
00236                 typename std::iterator_traits<InputIt>::iterator_category>>
00237     void assign(InputIt first, InputIt last) {
00238         clear();
00239         for (auto it = first; it != last; ++it)
00240             push_back(*it);
00241     }
00242
00243     /**
00244     * @brief Priskiria elementus iš initializer_list.
00245     */
00246     void assign(std::initializer_list<T> il) {
00247         assign(il.begin(), il.end());
00248     }
00249
00250     /**
00251     * @brief Gražina paskirstytojo kopiją.
00252     */
00253     allocator_type get_allocator() const noexcept { return alloc_; }
00254
00255     // element access
00256
00257     /**
00258     * @brief Prieiga prie elemento su ribų tikrinimu.
00259     * @throws std::out_of_range jei pos >= size()
00260     */
00261     reference at(size_type pos) {
00262         if (pos >= size_) throw std::out_of_range("Vector::at - indeksas uz ribu");
00263         return data_[pos];
00264     }
00265
00266     /**
00267     * @brief Prieiga prie elemento su ribų tikrinimu (const).
00268     */
00269     const_reference at(size_type pos) const {
00270         if (pos >= size_) throw std::out_of_range("Vector::at - indeksas uz ribu");
00271         return data_[pos];

```

```

00272     }
00273
00274     /**
00275      * @brief Prieiga prie elemento pagal indeksą (be tikrinimo).
00276      */
00277     reference operator[](size_type pos) { return data_[pos]; }
00278
00279     /**
00280      * @brief Prieiga prie elemento pagal indeksą (be tikrinimo, const).
00281      */
00282     const_reference operator[](size_type pos) const { return data_[pos]; }
00283
00284     /**
00285      * @brief Gražina pirmą elementą.
00286      */
00287     reference front() { return data_[0]; }
00288     const_reference front() const { return data_[0]; }
00289
00290     /**
00291      * @brief Gražina paskutinį elementą.
00292      */
00293     reference back() { return data_[size_ - 1]; }
00294     const_reference back() const { return data_[size_ - 1]; }
00295
00296     /**
00297      * @brief Gražina rodyklę į vidinį duomenų masyvą.
00298      */
00299     T* data() noexcept { return data_; }
00300     const T* data() const noexcept { return data_; }
00301
00302     // Iteratoriai
00303
00304     /** @brief Gražina iteratorių į pradžią. */
00305     iterator begin() noexcept { return data_; }
00306     const_iterator begin() const noexcept { return data_; }
00307     const_iterator cbegin() const noexcept { return data_; }
00308
00309     /** @brief Gražina iteratorių į pabaigą (past-the-end). */
00310     iterator end() noexcept { return data_ + size_; }
00311     const_iterator end() const noexcept { return data_ + size_; }
00312     const_iterator cend() const noexcept { return data_ + size_; }
00313
00314     /** @brief Gražina atvirkštinį iteratorių. */
00315     reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00316     const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00317     const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00318
00319     /** @brief Gražina atvirkštinį past-the-end iteratorių. */
00320     reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00321     const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00322     const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00323
00324     // Talpa ir dydis
00325
00326     /** @brief Tikrina ar vektorius tuščias. */
00327     [[nodiscard]] bool empty() const noexcept { return size_ == 0; }
00328
00329     /** @brief Gražina elementų skaičių. */
00330     size_type size() const noexcept { return size_; }
00331
00332     /** @brief Gražina maksimalų galimą elementų skaičių. */
00333     size_type max_size() const noexcept {
00334         return AllocTraits::max_size(alloc_);
00335     }
00336
00337     /**
00338      * @brief Rezervuoja atmintį new_cap elementams.
00339      * @param new_cap Naujas capacity
00340      * @throws std::length_error jei new_cap > max_size()
00341      */
00342     void reserve(size_type new_cap) {
00343         if (new_cap <= capacity_) return;
00344         if (new_cap > max_size())
00345             throw std::length_error("Vector::reserve - per didelis dydis");
00346         reallocate(new_cap);
00347     }
00348
00349     /** @brief Gražina dabartiną capacity. */
00350     size_type capacity() const noexcept { return capacity_; }
00351
00352     /**
00353      * @brief Sumažina capacity iki size (shrink to fit).
00354      */
00355     void shrink_to_fit() {
00356         if (size_ < capacity_)
00357             reallocate(size_);
00358     }

```

```

00359
00360 // Modifikacija
00361
00362 /** @brief Ištrina visus elementus (size = 0, capacity nesikeičia). */
00363 void clear() noexcept {
00364     for (size_type i = 0; i < size_; ++i)
00365         AllocTraits::destroy(alloc_, data_ + i);
00366     size_ = 0;
00367 }
00368
00369 /**
00370  * @brief Įterpia elementą prieš pos.
00371  * @param pos    Iteratorius į vietą
00372  * @param value  Įterpiama reikšmė
00373  * @return Iteratorius į įterptą elementą
00374  */
00375 iterator insert(const_iterator pos, const T& value) {
00376     return emplace(pos, value);
00377 }
00378
00379 iterator insert(const_iterator pos, T&& value) {
00380     return emplace(pos, std::move(value));
00381 }
00382
00383 /**
00384  * @brief Įterpia count kopijų reikšmės prieš pos.
00385  */
00386 iterator insert(const_iterator pos, size_type count, const T& value) {
00387     size_type idx = pos - cbegin();
00388     if (count == 0) return begin() + idx;
00389     if (size_ + count > capacity_) {
00390         size_type new_cap = std::max(size_ + count, capacity_ * 2);
00391         reallocate(new_cap);
00392     }
00393
00394     for (size_type i = size_; i > idx; --i) {
00395         size_type old_idx = i - 1;
00396         size_type new_idx = old_idx + count;
00397         if (new_idx >= size_) {
00398             AllocTraits::construct(alloc_, data_ + new_idx, std::move(data_[old_idx]));
00399         }
00400         else {
00401             data_[new_idx] = std::move(data_[old_idx]);
00402         }
00403     }
00404
00405     size_type affected_constructed = std::min(size_, idx + count);
00406     for (size_type i = idx; i < affected_constructed; ++i) {
00407         AllocTraits::destroy(alloc_, data_ + i);
00408     }
00409
00410     for (size_type i = 0; i < count; ++i) {
00411         AllocTraits::construct(alloc_, data_ + idx + i, value);
00412     }
00413     size_ += count;
00414     return begin() + idx;
00415 }
00416
00417 /**
00418  * @brief Įterpia elementus iš diapazono prieš pos.
00419  */
00420 template<typename InputIt,
00421          typename = std::enable_if_t<std::is_base_of_v<
00422              std::input_iterator_tag,
00423              typename>::iterator_traits<InputIt>::iterator_category>>
00424 iterator insert(const_iterator pos, InputIt first, InputIt last) {
00425     size_type idx = pos - cbegin();
00426     Vector tmp(first, last);
00427     size_type count = tmp.size();
00428     if (count == 0) return begin() + idx;
00429     if (size_ + count > capacity_)
00430         reallocate(std::max(size_ + count, capacity_ * 2));
00431
00432     for (size_type i = size_; i > idx; --i) {
00433         size_type old_idx = i - 1;
00434         size_type new_idx = old_idx + count;
00435         if (new_idx >= size_) {
00436             AllocTraits::construct(alloc_, data_ + new_idx, std::move(data_[old_idx]));
00437         }
00438         else {
00439             data_[new_idx] = std::move(data_[old_idx]);
00440         }
00441     }
00442
00443     size_type affected_constructed = std::min(size_, idx + count);
00444     for (size_type i = idx; i < affected_constructed; ++i) {
00445         AllocTraits::destroy(alloc_, data_ + i);

```

```

00446     }
00447
00448     for (size_type i = 0; i < count; ++i)
00449         AllocTraits::construct(alloc_, data_ + idx + i, std::move(tmp[i]));
00450     size_ += count;
00451     return begin() + idx;
00452 }
00453
00454 iterator insert(const_iterator pos, std::initializer_list<T> il) {
00455     return insert(pos, il.begin(), il.end());
00456 }
00457
00458 /**
00459  * @brief Sukonstruoja elementą vietoje prieš pos.
00460  */
00461 template<typename... Args>
00462 iterator emplace(const_iterator pos, Args&&... args) {
00463     size_type idx = pos - cbegin();
00464     if (size_ == capacity_)
00465         reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00466
00467     for (size_type i = size_; i > idx; --i) {
00468         size_type old_idx = i - 1;
00469         size_type new_idx = old_idx + 1;
00470         if (new_idx >= size_) {
00471             AllocTraits::construct(alloc_, data_ + new_idx, std::move(data_[old_idx]));
00472         }
00473         else {
00474             data_[new_idx] = std::move(data_[old_idx]);
00475         }
00476     }
00477
00478     if (idx < size_) {
00479         AllocTraits::destroy(alloc_, data_ + idx);
00480     }
00481
00482     AllocTraits::construct(alloc_, data_ + idx, std::forward<Args>(args)...);
00483     ++size_;
00484     return begin() + idx;
00485 }
00486
00487 /**
00488  * @brief Ištrina elementą ties pos.
00489  * @return Iteratorius į kitą elementą po ištrintojo
00490  */
00491 iterator erase(const_iterator pos) {
00492     return erase(pos, pos + 1);
00493 }
00494
00495 /**
00496  * @brief Ištrina elementus diapazone [first, last).
00497  */
00498 iterator erase(const_iterator first, const_iterator last) {
00499     iterator f = begin() + (first - cbegin());
00500     iterator l = begin() + (last - cbegin());
00501     size_type count = l - f;
00502     std::move(l, end(), f);
00503     for (size_type i = 0; i < count; ++i)
00504         AllocTraits::destroy(alloc_, end() - 1 - i);
00505     size_ -= count;
00506     return f;
00507 }
00508
00509 /**
00510  * @brief Prideda elementą į galą (kopija).
00511  * @param value Priedama reikšmė
00512  */
00513 void push_back(const T& value) {
00514     emplace_back(value);
00515 }
00516
00517 /**
00518  * @brief Prideda elementą į galą (perkėlimas).
00519  */
00519 void push_back(T&& value) {
00520     emplace_back(std::move(value));
00521 }
00522
00523 /**
00524  * @brief Sukonstruoja elementą gale vietoje.
00525  * @return Nuoroda į sukonstruotą elementą
00526  */
00527 template<typename... Args>
00528 reference emplace_back(Args&&... args) {
00529     if (size_ == capacity_)
00530         reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00531     AllocTraits::construct(alloc_, data_ + size_, std::forward<Args>(args)...);
00532     return data_[size_++];

```



```

00533     }
00534
00535     /**
00536      * @brief Pašalina paskutinį elementą.
00537      */
00538     void pop_back() {
00539         AllocTraits::destroy(alloc_, data_ + --size_);
00540     }
00541
00542     /**
00543      * @brief Pakeičia vektorių dydį.
00544      * @param count Naujas dydis
00545      * @param value Reikšmė naujiems elementams (jei didėja)
00546      */
00547     void resize(size_type count, const T& value = T()) {
00548         if (count < size_) {
00549             for (size_type i = count; i < size_; ++i)
00550                 AllocTraits::destroy(alloc_, data_ + i);
00551             size_ = count;
00552         }
00553         else if (count > size_) {
00554             reserve(count);
00555             for (size_type i = size_; i < count; ++i)
00556                 AllocTraits::construct(alloc_, data_ + i, value);
00557             size_ = count;
00558         }
00559     }
00560
00561     /**
00562      * @brief Sukeičia du vektorius vietomis (O(1)).
00563      * @param other Kitas vektorius
00564      */
00565     void swap(Vector& other) noexcept {
00566         std::swap(data_, other.data_);
00567         std::swap(size_, other.size_);
00568         std::swap(capacity_, other.capacity_);
00569         if constexpr (AllocTraits::propagate_on_container_swap::value)
00570             std::swap(alloc_, other.alloc_);
00571     }
00572 };
00573
00574 // Non-member functions
00575
00576 /** @brief Lygybės operatorius. */
00577 template<typename T, typename A>
00578 bool operator==(const Vector<T, A>& lhs, const Vector<T, A>& rhs) {
00579     if (lhs.size() != rhs.size()) return false;
00580     return std::equal(lhs.begin(), lhs.end(), rhs.begin());
00581 }
00582
00583 /** @brief Nelygybės operatorius. */
00584 template<typename T, typename A>
00585 bool operator!=(const Vector<T, A>& lhs, const Vector<T, A>& rhs) {
00586     return !(lhs == rhs);
00587 }
00588
00589 /** @brief Leksikografinis < palyginimas. */
00590 template<typename T, typename A>
00591 bool operator<(const Vector<T, A>& lhs, const Vector<T, A>& rhs) {
00592     return std::lexicographical_compare(lhs.begin(), lhs.end(),
00593         rhs.begin(), rhs.end());
00594 }
00595
00596 /** @brief Leksikografinis <= palyginimas. */
00597 template<typename T, typename A>
00598 bool operator<=(const Vector<T, A>& lhs, const Vector<T, A>& rhs) {
00599     return !(rhs < lhs);
00600 }
00601
00602 /** @brief Leksikografinis > palyginimas. */
00603 template<typename T, typename A>
00604 bool operator>(const Vector<T, A>& lhs, const Vector<T, A>& rhs) {
00605     return rhs < lhs;
00606 }
00607
00608 /** @brief Leksikografinis >= palyginimas. */
00609 template<typename T, typename A>
00610 bool operator>=(const Vector<T, A>& lhs, const Vector<T, A>& rhs) {
00611     return !(lhs < rhs);
00612 }
00613
00614 /** @brief Globalus swap (ADL). */
00615 template<typename T, typename A>
00616 void swap(Vector<T, A>& lhs, Vector<T, A>& rhs) noexcept {
00617     lhs.swap(rhs);
00618 }
00619

```

```

00620 /** @brief Ištrina elementus pagal reikšmę (C++20 stilius). */
00621 template<typename T, typename A, typename U>
00622 typename Vector<T, A>::size_type erase(Vector<T, A>& c, const U& value) {
00623     auto it = std::remove(c.begin(), c.end(), value);
00624     auto n = c.end() - it;
00625     c.erase(it, c.end());
00626     return static_cast<typename Vector<T, A>::size_type>(n);
00627 }
00628
00629 /** @brief Ištrina elementus pagal predikato sąlygą (C++20 stilius). */
00630 template<typename T, typename A, typename Pred>
00631 typename Vector<T, A>::size_type erase_if(Vector<T, A>& c, Pred pred) {
00632     auto it = std::remove_if(c.begin(), c.end(), pred);
00633     auto n = c.end() - it;
00634     c.erase(it, c.end());
00635     return static_cast<typename Vector<T, A>::size_type>(n);
00636 }
00637
00638 #endif // VECTOR_H

```

5.31 vector_ops.cpp Failo Nuoroda

Vektoriaus operacijų realizacija naudojant nuosavą [Vector](#) konteinerį.

```

#include "vector_ops.h"
#include "utils.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <iomanip>
#include <stdexcept>
#include <chrono>

```

Funkcijos

- void [skaitymas_is_failo](#) (const std::string &filename, [Vector< Studentas >](#) &studentai)
- void [isvedimas](#) (const [Vector< Studentas >](#) &studentai, int metodas)
Išveda studentų sąrašą į konsolę.
- void [isvedimas_i_faile](#) (const [Vector< Studentas >](#) &studentai, const std::string &filename, const std::string &kategorija)
Išveda studentų sąrašą į failą.
- void [pasirinkimo_metodas](#) (int tipas, [Vector< Studentas >](#) &studentai)
Apskaičiuoja galutinį rezultatą kiekvienam studentui.
- void [rusiavimas](#) ([Vector< Studentas >](#) &studentai, int budas)
Surūšiuoja studentų vektorius pagal pasirinktą kriterijų.
- int [pasirinkimas_rusiavimo_budo](#) ()
Paprašo vartotojo pasirinkti rūšiavimo būdą.
- void [skirstymas_i_grupes](#) (const [Vector< Studentas >](#) &visi, [Vector< Studentas >](#) &kieti, [Vector< Studentas >](#) &vargsai)
Paprastas skirstymas į dvi grupes (originalas lieka nepakitęs).
- double [skirstymas_s1](#) (const [Vector< Studentas >](#) &studentai, [Vector< Studentas >](#) &kieti, [Vector< Studentas >](#) &vargsai)
Strategija S1: du nauji konteineriai, kopijavimas su copy_if.
- double [skirstymas_s2](#) ([Vector< Studentas >](#) &studentai, [Vector< Studentas >](#) &vargsai)
Strategija S2: vienas naujas + erase/remove_if.
- double [skirstymas_s3](#) ([Vector< Studentas >](#) &studentai, [Vector< Studentas >](#) &vargsai)
Strategija S3: std::partition.

5.31.1 Smulkus aprašymas

Vektoriaus operacijų realizacija naudojant nuosavą `Vector` konteinerį.

v3.0: `std::vector` pakeistas nuosavu `Vector<T>` konteineriu.

Apibrėžimas faile `vector_ops.cpp`.

5.31.2 Funkcijos Dokumentacija

5.31.2.1 isvedimas()

```
void isvedimas (
    const Vector< Studentas > & studentai,
    int metodas)
```

Išveda studentų sąrašą į konsolę.

Parametrai

<i>studentai</i>	Studentų vektorius.
<i>metodas</i>	1 – vidurkis, 2 – mediana (nurodoma antraštėje).

Apibrėžimas failo `vector_ops.cpp` eilutėje 47.

```
00047
00048     std::string rez = (metodas == 1) ? "Galutinis (vidurkis)" : "Galutinis (mediana)";
00049     std::cout << std::left << std::setw(15) << "Vardas"
00050         << std::setw(15) << "Pavarde"
00051         << std::right << std::setw(20) << rez << "\n";
00052     for (const auto& s : studentai)
00053         std::cout << s << "\n";
00054 }
```

5.31.2.2 isvedimas_i_faila()

```
void isvedimas_i_faila (
    const Vector< Studentas > & studentai,
    const std::string & filename,
    const std::string & kategorija)
```

Išveda studentų sąrašą į failą.

Parametrai

<i>studentai</i>	Studentų vektorius.
<i>filename</i>	Išvesties failo pavadinimas.
<i>kategorija</i>	"Kietiakai" arba "Vargsiukai" – naudojama pranešimui.

Išimtys

<i>std::runtime_error</i>	Jei failo nepavyksta sukurti.
---------------------------	-------------------------------

Apibrėžimas failo `vector_ops.cpp` eilutėje 56.

```
00058 {
00059     std::ofstream out(filename);
00060     if (!out.is_open())
00061         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00062
00063     out << std::left << std::setw(20) << "Vardas"
00064         << std::setw(20) << "Pavarde"
00065         << std::right << std::setw(20) << "Galutinis\n"
00066         << std::string(60, '-') << "\n";
00067
00068     for (const auto& s : studentai)
00069         out << s << "\n";
00070
00071     std::cout << " [" << kategorija << "]" << studentai.size()
00072         << " studentu -> " << filename << "\n";
00073 }
```

5.31.2.3 pasirinkimas_rusiavimo_budo()

```
int pasirinkimas_rusiavimo_budo ()
```

Paprašo vartotojo pasirinkti rūšiavimo būdą.

Gražina

Pasirinkimo kodas (1..6).

Apibrėžimas failo `vector_ops.cpp` eilutėje 98.

```
00098 {
00099     std::cout << "Pasirinkite rusiavimo buda:\n"
00100         << "1 - pagal varda didejanciai\n"
00101         << "2 - pagal varda mazejanciai\n"
00102         << "3 - pagal pavarde didejanciai\n"
00103         << "4 - pagal pavarde mazejanciai\n"
00104         << "5 - pagal galutini rezultata didejanciai\n"
00105         << "6 - pagal galutini rezultata mazejanciai\n";
00106     int p; std::cin >> p;
00107     if (std::cin.fail() || p < 1 || p > 6) {
00108         std::cerr << "Neteisingas pasirinkimas.\n";
00109         return 5;
00110     }
00111     return p;
00112 }
```

5.31.2.4 pasirinkimo_metodas()

```
void pasirinkimo_metodas (
    int tipas,
    Vector< Studentas > & studentai)
```

Apskaičiuoja galutinį rezultatą kiekvienam studentui.

Parametrai

<i>tipas</i>	1 – pagal vidurkį, 2 – pagal medianą.
<i>studentai</i>	Studentų vektorius (modifikuojamas).

Apibrėžimas failo `vector_ops.cpp` eilutėje 75.

```
00075 {
00076     for (auto& s : studentai)
00077         s.skaiciuotiRez(tipas);
00078 }
```

5.31.2.5 rusiavimas()

```
void rusiavimas (
    Vector< Studentas > & studentai,
    int budas)
```

Surūšiuoja studentų vektorių pagal pasirinktą kriterijų.

Parametrai

<i>studentai</i>	Vektorius (modifikuojamas).
<i>budas</i>	Rūšiavimo būdas: 1 – pagal vardą didėjant, 2 – pagal vardą mažėjant, 3 – pagal pavardę didėjant, 4 – pagal pavardę mažėjant, 5 – pagal galutinį rezultatą didėjant, 6 – pagal galutinį rezultatą mažėjant.

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 80.

```
00080                                     {
00081     switch (budas) {
00082     case 1: std::sort(studentai.begin(), studentai.end(),
00083         [](const Studentas& a, const Studentas& b) { return a.getVardas() < b.getVardas(); }); break;
00084     case 2: std::sort(studentai.begin(), studentai.end(),
00085         [](const Studentas& a, const Studentas& b) { return a.getVardas() > b.getVardas(); }); break;
00086     case 3: std::sort(studentai.begin(), studentai.end(),
00087         [](const Studentas& a, const Studentas& b) { return a.getPavarde() < b.getPavarde(); });
00088         break;
00089     case 4: std::sort(studentai.begin(), studentai.end(),
00090         [](const Studentas& a, const Studentas& b) { return a.getPavarde() > b.getPavarde(); });
00091         break;
00092     case 5: std::sort(studentai.begin(), studentai.end(),
00093         [](const Studentas& a, const Studentas& b) { return a.getRez() < b.getRez(); }); break;
00094     case 6: std::sort(studentai.begin(), studentai.end(),
00095         [](const Studentas& a, const Studentas& b) { return a.getRez() > b.getRez(); }); break;
00096     default: std::cerr << "Neteisingas rusiavimo budas.\n"; break;
00097     }
```

5.31.2.6 skaitymas_is_failo()

```
void skaitymas_is_failo (
    const std::string & filename,
    Vector< Studentas > & studentai)
```

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 19.

```
00019                                     {
00020     std::ifstream file(filename);
00021     if (!file.is_open())
00022         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00023
00024     studentai.clear();
00025     std::string header, line;
00026     std::getline(file, header);
00027
00028     int nr = 0;
00029     while (std::getline(file, line)) {
00030         ++nr;
00031         std::stringstream ss(line);
00032         std::string vardas, pavarde;
00033         ss >> vardas >> pavarde;
00034         Vector<int> paz;
00035         int v;
00036         while (ss >> v) paz.push_back(v);
00037         if (paz.empty()) continue;
00038         int egz = paz.back();
00039         paz.pop_back();
00040         studentai.emplace_back(vardas, pavarde, paz, egz);
00041         if (nr % 1000000 == 0)
00042             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00043     }
00044     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00045 }
```

5.31.2.7 skirstymas_i_grupes()

```
void skirstymas_i_grupes (
    const Vector< Studentas > & visi,
    Vector< Studentas > & kieti,
    Vector< Studentas > & vargsai)
```

Paprastas skirstymas į dvi grupes (originalas lieka nepakitęs).

Parametrai

<i>visi</i>	Visi studentai (const).
<i>kieti</i>	[out] Studentai su rez ≥ 5.0 .
<i>vargsai</i>	[out] Studentai su rez < 5.0 .

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 114.

```
00116         {
00117     for (const auto& s : visi) {
00118         if (s.getRez() >= 5.0) kieti.push_back(s);
00119         else vargsai.push_back(s);
00120     }
00121 }
```

5.31.2.8 skirstymas_s1()

```
double skirstymas_s1 (
    const Vector< Studentas > & studentai,
    Vector< Studentas > & kieti,
    Vector< Studentas > & vargsai)
```

Strategija S1: du nauji konteineriai, kopijavimas su copy_if.

Gražina

Sugaištas laikas sekundėmis.

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 123.

```
00125         {
00126     auto t0 = std::chrono::high_resolution_clock::now();
00127     kieti.clear(); vargsai.clear();
00128     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(kieti),
00129         [](const Studentas& s) { return s.getRez() >= 5.0; });
00130     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00131         [](const Studentas& s) { return s.getRez() < 5.0; });
00132     return std::chrono::duration<double>{
00133         std::chrono::high_resolution_clock::now() - t0}.count();
00134 }
```

5.31.2.9 skirstymas_s2()

```
double skirstymas_s2 (
    Vector< Studentas > & studentai,
    Vector< Studentas > & vargsai)
```

Strategija S2: vienas naujas + erase/remove_if.

Gražina

Sugaištas laikas sekundėmis.

Apibrėžimas failo `vector_ops.cpp` eilutėje 136.

```
00137     {
00138         auto t0 = std::chrono::high_resolution_clock::now();
00139         vargsai.clear();
00140         std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00141             [](const Studentas& s) { return s.getRez() < 5.0; });
00142         studentai.erase(
00143             std::remove_if(studentai.begin(), studentai.end(),
00144                 [](const Studentas& s) { return s.getRez() < 5.0; }),
00145             studentai.end());
00146         return std::chrono::duration<double>(
00147             std::chrono::high_resolution_clock::now() - t0).count();
00148     }
```

5.31.2.10 skirstymas_s3()

```
double skirstymas_s3 (
    Vector< Studentas > & studentai,
    Vector< Studentas > & vargsai)
```

Strategija S3: `std::partition`.

Gražina

Sugaištas laikas sekundėmis.

Apibrėžimas failo `vector_ops.cpp` eilutėje 150.

```
00151     {
00152         auto t0 = std::chrono::high_resolution_clock::now();
00153         vargsai.clear();
00154         auto pivot = std::partition(studentai.begin(), studentai.end(),
00155             [](const Studentas& s) { return s.getRez() >= 5.0; });
00156         vargsai.assign(pivot, studentai.end());
00157         studentai.erase(pivot, studentai.end());
00158         return std::chrono::duration<double>(
00159             std::chrono::high_resolution_clock::now() - t0).count();
00160     }
```

5.32 vector_ops.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file vector_ops.cpp
00003  * @brief Vektoriaus operacijų realizacija naudojant nuosavą Vector konteinerį.
00004  *
00005  * v3.0: std::vector pakeistas nuosavu Vector<T> konteineriu.
00006  */
00007
00008 #include "vector_ops.h"
00009 #include "utils.h"
00010
00011 #include <iostream>
00012 #include <fstream>
00013 #include <sstream>
00014 #include <algorithm>
00015 #include <iomanip>
00016 #include <stdexcept>
00017 #include <chrono>
00018
00019 void skaitymas_is_failo(const std::string& filename, Vector<Studentas>& studentai) {
00020     std::ifstream file(filename);
00021     if (!file.is_open())
00022         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
```

```

00023
00024     studentai.clear();
00025     std::string header, line;
00026     std::getline(file, header);
00027
00028     int nr = 0;
00029     while (std::getline(file, line)) {
00030         ++nr;
00031         std::stringstream ss(line);
00032         std::string vardas, pavarde;
00033         ss » vardas » pavarde;
00034         Vector<int> paz;
00035         int v;
00036         while (ss » v) paz.push_back(v);
00037         if (paz.empty()) continue;
00038         int egz = paz.back();
00039         paz.pop_back();
00040         studentai.emplace_back(vardas, pavarde, paz, egz);
00041         if (nr % 1000000 == 0)
00042             std::cout << " Nuskaityta: " << nr << " studentu...\n";
00043     }
00044     std::cout << " Is viso nuskaityta: " << studentai.size() << " studentu\n";
00045 }
00046
00047 void isvedimas(const Vector<Studentas>& studentai, int metodos) {
00048     std::string rez = (metodos == 1) ? "Galutinis (vidurkis)" : "Galutinis (mediana)";
00049     std::cout << std::left << std::setw(15) << "Vardas"
00050         << std::setw(15) << "Pavarde"
00051         << std::right << std::setw(20) << rez << "\n";
00052     for (const auto& s : studentai)
00053         std::cout << s << "\n";
00054 }
00055
00056 void isvedimas_i_faila(const Vector<Studentas>& studentai,
00057     const std::string& filename,
00058     const std::string& kategorija) {
00059     std::ofstream out(filename);
00060     if (!out.is_open())
00061         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00062
00063     out << std::left << std::setw(20) << "Vardas"
00064         << std::setw(20) << "Pavarde"
00065         << std::right << std::setw(20) << "Galutinis\n"
00066         << std::string(60, '-') << "\n";
00067
00068     for (const auto& s : studentai)
00069         out << s << "\n";
00070
00071     std::cout << " [" << kategorija << "]" << " << studentai.size()
00072         << " studentu -> " << filename << "\n";
00073 }
00074
00075 void pasirinkimo_metodas(int tipas, Vector<Studentas>& studentai) {
00076     for (auto& s : studentai)
00077         s.skaiciuotiRez(tipas);
00078 }
00079
00080 void rusiavimas(Vector<Studentas>& studentai, int budas) {
00081     switch (budas) {
00082     case 1: std::sort(studentai.begin(), studentai.end(),
00083         [](const Studentas& a, const Studentas& b) { return a.getVardas() < b.getVardas(); }); break;
00084     case 2: std::sort(studentai.begin(), studentai.end(),
00085         [](const Studentas& a, const Studentas& b) { return a.getVardas() > b.getVardas(); }); break;
00086     case 3: std::sort(studentai.begin(), studentai.end(),
00087         [](const Studentas& a, const Studentas& b) { return a.getPavarde() < b.getPavarde(); });
00088         break;
00089     case 4: std::sort(studentai.begin(), studentai.end(),
00090         [](const Studentas& a, const Studentas& b) { return a.getPavarde() > b.getPavarde(); });
00091         break;
00092     case 5: std::sort(studentai.begin(), studentai.end(),
00093         [](const Studentas& a, const Studentas& b) { return a.getRez() < b.getRez(); }); break;
00094     case 6: std::sort(studentai.begin(), studentai.end(),
00095         [](const Studentas& a, const Studentas& b) { return a.getRez() > b.getRez(); }); break;
00096     default: std::cerr << "Neteisingas rusiavimo budas.\n"; break;
00097     }
00098 }
00099
00098 int pasirinkimas_rusiavimo_budo() {
00099     std::cout << "Pasirinkite rusiavimo buda:\n"
00100         << "1 - pagal varda didejanciai\n"
00101         << "2 - pagal varda mazejanciai\n"
00102         << "3 - pagal pavarde didejanciai\n"
00103         << "4 - pagal pavarde mazejanciai\n"
00104         << "5 - pagal galutini rezultata didejanciai\n"
00105         << "6 - pagal galutini rezultata mazejanciai\n";
00106     int p; std::cin » p;
00107     if (std::cin.fail() || p < 1 || p > 6) {

```



```

00108         std::cerr << "Neteisingas pasirinkimas.\n";
00109         return 5;
00110     }
00111     return p;
00112 }
00113
00114 void skirstymas_i_grupes(const Vector<Studentas>& visi,
00115     Vector<Studentas>& kieti,
00116     Vector<Studentas>& vargsai) {
00117     for (const auto& s : visi) {
00118         if (s.getRez() >= 5.0) kieti.push_back(s);
00119         else vargsai.push_back(s);
00120     }
00121 }
00122
00123 double skirstymas_s1(const Vector<Studentas>& studentai,
00124     Vector<Studentas>& kieti,
00125     Vector<Studentas>& vargsai) {
00126     auto t0 = std::chrono::high_resolution_clock::now();
00127     kieti.clear(); vargsai.clear();
00128     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(kieti),
00129         [](const Studentas& s) { return s.getRez() >= 5.0; });
00130     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00131         [](const Studentas& s) { return s.getRez() < 5.0; });
00132     return std::chrono::duration<double>(
00133         std::chrono::high_resolution_clock::now() - t0).count();
00134 }
00135
00136 double skirstymas_s2(Vector<Studentas>& studentai,
00137     Vector<Studentas>& vargsai) {
00138     auto t0 = std::chrono::high_resolution_clock::now();
00139     vargsai.clear();
00140     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00141         [](const Studentas& s) { return s.getRez() < 5.0; });
00142     studentai.erase(
00143         std::remove_if(studentai.begin(), studentai.end(),
00144             [](const Studentas& s) { return s.getRez() < 5.0; }),
00145         studentai.end());
00146     return std::chrono::duration<double>(
00147         std::chrono::high_resolution_clock::now() - t0).count();
00148 }
00149
00150 double skirstymas_s3(Vector<Studentas>& studentai,
00151     Vector<Studentas>& vargsai) {
00152     auto t0 = std::chrono::high_resolution_clock::now();
00153     vargsai.clear();
00154     auto pivot = std::partition(studentai.begin(), studentai.end(),
00155         [](const Studentas& s) { return s.getRez() >= 5.0; });
00156     vargsai.assign(pivot, studentai.end());
00157     studentai.erase(pivot, studentai.end());
00158     return std::chrono::duration<double>(
00159         std::chrono::high_resolution_clock::now() - t0).count();
00160 }

```

5.33 vector_ops.h Failo Nuoroda

Vektoriaus operacijos naudojant nuosavą [Vector](#) konteinerį.

```

#include "studentas.h"
#include "Vector.h"
#include <string>

```

Funkcijos

- void [skaitymas_is_failo](#) (const std::string &filename, [Vector< Studentas >](#) &studentai)
- void [isvedimas](#) (const [Vector< Studentas >](#) &studentai, int metodas)
Išveda studentų sąrašą į konsolę.
- void [isvedimas_i_faila](#) (const [Vector< Studentas >](#) &studentai, const std::string &filename, const std::string &kategorija)
Išveda studentų sąrašą į failą.

- void `pasirinkimo_metodas` (int tipas, `Vector< Studentas >` &studentai)
Apskaičiuoja galutinį rezultatą kiekvienam studentui.
- void `rusiavimas` (`Vector< Studentas >` &studentai, int budas)
Surūšiuoja studentų vektorių pagal pasirinktą kriterijų.
- int `pasirinkimas_rusiavimo_budo` ()
Paprašo vartotojo pasirinkti rūšiavimo būdą.
- void `skirstymas_i_grupes` (const `Vector< Studentas >` &visi, `Vector< Studentas >` &kieti, `Vector< Studentas >` &vargsai)
Paprastas skirstymas į dvi grupes (originalas lieka nepakitęs).
- double `skirstymas_s1` (const `Vector< Studentas >` &studentai, `Vector< Studentas >` &kieti, `Vector< Studentas >` &vargsai)
Strategija S1: du nauji konteineriai, kopijavimas su copy_if.
- double `skirstymas_s2` (`Vector< Studentas >` &studentai, `Vector< Studentas >` &vargsai)
Strategija S2: vienas naujas + erase/remove_if.
- double `skirstymas_s3` (`Vector< Studentas >` &studentai, `Vector< Studentas >` &vargsai)
Strategija S3: std::partition.

5.33.1 Smulkus aprašymas

Vektoriaus operacijos naudojant nuosavą `Vector` konteinerį.

v3.0: `std::vector` pakeistas nuosavu `Vector<T>` konteineriu.

Apibrėžimas faile `vector_ops.h`.

5.33.2 Funkcijos Dokumentacija

5.33.2.1 isvedimas()

```
void isvedimas (
    const Vector< Studentas > & studentai,
    int metodas)
```

Išveda studentų sąrašą į konsolę.

Parametrai

<i>studentai</i>	Studentų vektorius.
<i>metodas</i>	1 – vidurkis, 2 – mediana (nurodoma antraštėje).

Apibrėžimas failo `vector_ops.cpp` eilutėje 47.

```
00047 {
00048     std::string rez = (metodas == 1) ? "Galutinis (vidurkis)" : "Galutinis (mediana)";
00049     std::cout << std::left << std::setw(15) << "Vardas"
00050         << std::setw(15) << "Pavarde"
00051         << std::right << std::setw(20) << rez << "\n";
00052     for (const auto& s : studentai)
00053         std::cout << s << "\n";
00054 }
```

5.33.2.2 isvedimas_i_faila()

```
void isvedimas_i_faila (  
    const Vector< Studentas > & studentai,  
    const std::string & filename,  
    const std::string & kategorija)
```

Išveda studentų sąrašą į failą.

Parametrai

<i>studentai</i>	Studentų vektorius.
------------------	---------------------

<i>filename</i>	Išvesties failo pavadinimas.
<i>kategorija</i>	"Kietiakai" arba "Vargsiukai" – naudojama pranešimui.

Išimtyys

<i>std::runtime_error</i>	Jei failo nepavyksta sukurti.
---------------------------	-------------------------------

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 56.

```

00058     {
00059         std::ofstream out(filename);
00060         if (!out.is_open())
00061             throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00062
00063         out << std::left << std::setw(20) << "Vardas"
00064             << std::setw(20) << "Pavarde"
00065             << std::right << std::setw(20) << "Galutinis\n"
00066             << std::string(60, '-') << "\n";
00067
00068         for (const auto& s : studentai)
00069             out << s << "\n";
00070
00071         std::cout << " [" << kategorija << "]" << studentai.size()
00072             << " studentu -> " << filename << "\n";
00073     }

```

5.33.2.3 pasirinkimas_rusiavimo_budo()

```
int pasirinkimas_rusiavimo_budo ()
```

Paprašo vartotojo pasirinkti rūšiavimo būdą.

Gražina

Pasirinkimo kodas (1..6).

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 98.

```

00098     {
00099         std::cout << "Pasirinkite rusiavimo buda:\n"
00100             << "1 - pagal varda didejanciai\n"
00101             << "2 - pagal varda mazejanciai\n"
00102             << "3 - pagal pavarde didejanciai\n"
00103             << "4 - pagal pavarde mazejanciai\n"
00104             << "5 - pagal galutini rezultata didejanciai\n"
00105             << "6 - pagal galutini rezultata mazejanciai\n";
00106         int p; std::cin >> p;
00107         if (std::cin.fail() || p < 1 || p > 6) {
00108             std::cerr << "Neteisingas pasirinkimas.\n";
00109             return 5;
00110         }
00111         return p;
00112     }

```

5.33.2.4 pasirinkimo_metodas()

```

void pasirinkimo_metodas (
    int tipas,
    Vector< Studentas > & studentai)

```

Apskaičiuoja galutinį rezultatą kiekvienam studentui.

Parametrai

<i>tipas</i>	1 – pagal vidurkį, 2 – pagal medianą.
--------------	---------------------------------------

<i>studentai</i>	Studentų vektorius (modifikuojamas).
------------------	--------------------------------------

Apibrėžimas failo `vector_ops.cpp` eilutėje 75.

```
00075                                     {
00076     for (auto& s : studentai)
00077         s.skaiciuotiRez(tipas);
00078 }
```

5.33.2.5 rusiavimas()

```
void rusiavimas (
    Vector< Studentas > & studentai,
    int budas)
```

Surūšiuoja studentų vektorių pagal pasirinktą kriterijų.

Parametrai

<i>studentai</i>	Vektorius (modifikuojamas).
<i>budas</i>	Rūšiavimo būdas: 1 – pagal vardą didėjant, 2 – pagal vardą mažėjant, 3 – pagal pavardę didėjant, 4 – pagal pavardę mažėjant, 5 – pagal galutinį rezultatą didėjant, 6 – pagal galutinį rezultatą mažėjant.

Apibrėžimas failo `vector_ops.cpp` eilutėje 80.

```
00080                                     {
00081     switch (budas) {
00082     case 1: std::sort(studentai.begin(), studentai.end(),
00083         [](const Studentas& a, const Studentas& b) { return a.getVardas() < b.getVardas(); }); break;
00084     case 2: std::sort(studentai.begin(), studentai.end(),
00085         [](const Studentas& a, const Studentas& b) { return a.getVardas() > b.getVardas(); }); break;
00086     case 3: std::sort(studentai.begin(), studentai.end(),
00087         [](const Studentas& a, const Studentas& b) { return a.getPavarde() < b.getPavarde(); });
00088         break;
00089     case 4: std::sort(studentai.begin(), studentai.end(),
00090         [](const Studentas& a, const Studentas& b) { return a.getPavarde() > b.getPavarde(); });
00091         break;
00092     case 5: std::sort(studentai.begin(), studentai.end(),
00093         [](const Studentas& a, const Studentas& b) { return a.getRez() < b.getRez(); }); break;
00094     case 6: std::sort(studentai.begin(), studentai.end(),
00095         [](const Studentas& a, const Studentas& b) { return a.getRez() > b.getRez(); }); break;
00096     default: std::cerr << "Neteisingas rusiavimo budas.\n"; break;
00097     }
00098 }
```

5.33.2.6 skaitymas_is_failo()

```
void skaitymas_is_failo (
    const std::string & filename,
    Vector< Studentas > & studentai)
```

Apibrėžimas failo `vector_ops.cpp` eilutėje 19.

```
00019                                     {
00020     std::ifstream file(filename);
00021     if (!file.is_open())
00022         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00023
00024     studentai.clear();
00025     std::string header, line;
00026     std::getline(file, header);
00027
00028     int nr = 0;
00029     while (std::getline(file, line)) {
00030         ++nr;
```

```

00031         std::stringstream ss(line);
00032         std::string vardas, pavarde;
00033         ss » vardas » pavarde;
00034         Vector<int> paz;
00035         int v;
00036         while (ss » v) paz.push_back(v);
00037         if (paz.empty()) continue;
00038         int egz = paz.back();
00039         paz.pop_back();
00040         studentai.emplace_back(vardas, pavarde, paz, egz);
00041         if (nr % 1000000 == 0)
00042             std::cout « " Nuskaityta: " « nr « " studentu...\n";
00043     }
00044     std::cout « " Is viso nuskaityta: " « studentai.size() « " studentu\n";
00045 }

```

5.33.2.7 skirstymas_i_grupes()

```

void skirstymas_i_grupes (
    const Vector< Studentas > & visi,
    Vector< Studentas > & kieti,
    Vector< Studentas > & vargsai)

```

Paprastas skirstymas į dvi grupes (originalas lieka nepakitęs).

Parametrai

<i>visi</i>	Visi studentai (const).
<i>kieti</i>	[out] Studentai su rez >= 5.0.
<i>vargsai</i>	[out] Studentai su rez < 5.0.

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 114.

```

00116         {
00117             for (const auto& s : visi) {
00118                 if (s.getRez() >= 5.0) kieti.push_back(s);
00119                 else vargsai.push_back(s);
00120             }
00121 }

```

5.33.2.8 skirstymas_s1()

```

double skirstymas_s1 (
    const Vector< Studentas > & studentai,
    Vector< Studentas > & kieti,
    Vector< Studentas > & vargsai)

```

Strategija S1: du nauji konteineriai, kopijavimas su copy_if.

Gražina

Sugaištas laikas sekundėmis.

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 123.

```

00125         {
00126             auto t0 = std::chrono::high_resolution_clock::now();
00127             kieti.clear(); vargsai.clear();
00128             std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(kieti),
00129                 [](const Studentas& s) { return s.getRez() >= 5.0; });
00130             std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00131                 [](const Studentas& s) { return s.getRez() < 5.0; });
00132             return std::chrono::duration<double>(
00133                 std::chrono::high_resolution_clock::now() - t0).count();
00134 }

```

5.33.2.9 skirstymas_s2()

```
double skirstymas_s2 (
    Vector< Studentas > & studentai,
    Vector< Studentas > & vargsai)
```

Strategija S2: vienas naujas + erase/remove_if.

Gražina

Sugaištas laikas sekundėmis.

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 136.

```
00137         {
00138     auto t0 = std::chrono::high_resolution_clock::now();
00139     vargsai.clear();
00140     std::copy_if(studentai.begin(), studentai.end(), std::back_inserter(vargsai),
00141         [](const Studentas& s) { return s.getRez() < 5.0; });
00142     studentai.erase(
00143         std::remove_if(studentai.begin(), studentai.end(),
00144             [](const Studentas& s) { return s.getRez() < 5.0; }),
00145         studentai.end());
00146     return std::chrono::duration<double>{
00147         std::chrono::high_resolution_clock::now() - t0}.count();
00148 }
```

5.33.2.10 skirstymas_s3()

```
double skirstymas_s3 (
    Vector< Studentas > & studentai,
    Vector< Studentas > & vargsai)
```

Strategija S3: std::partition.

Gražina

Sugaištas laikas sekundėmis.

Apibrėžimas failo [vector_ops.cpp](#) eilutėje 150.

```
00151         {
00152     auto t0 = std::chrono::high_resolution_clock::now();
00153     vargsai.clear();
00154     auto pivot = std::partition(studentai.begin(), studentai.end(),
00155         [](const Studentas& s) { return s.getRez() >= 5.0; });
00156     vargsai.assign(pivot, studentai.end());
00157     studentai.erase(pivot, studentai.end());
00158     return std::chrono::duration<double>{
00159         std::chrono::high_resolution_clock::now() - t0}.count();
00160 }
```

5.34 vector_ops.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef VECTOR_OPS_H
00002 #define VECTOR_OPS_H
00003
00004 #include "studentas.h"
00005 #include "Vector.h"
00006 #include <string>
00007
00008 /**
00009  * @file vector_ops.h
00010  * @brief Vektoriaus operacijos naudojant nuosavą Vector konteinerį.
00011  *
00012  * v3.0: std::vector pakeistas nuosavu Vector<T> konteineriu.
00013  */
00014
00015 // Skaitymas ir išvedimas
00016 void skaitymas_is_failo(const std::string& filename, Vector<Studentas>& studentai);
00017
00018 /**
00019  * @brief Išveda studentų sąrašą į konsolę.
00020  * @param studentai Studentų vektorius.
00021  * @param metodas 1 - vidurkis, 2 - mediana (nurodoma antraštėje).
00022  */
00023 void isvedimas(const Vector<Studentas>& studentai, int metodas);
00024
00025 /**
00026  * @brief Išveda studentų sąrašą į failą.
00027  * @param studentai Studentų vektorius.
00028  * @param filename Išvesties failo pavadinimas.
00029  * @param kategorija "Kietiakai" arba "Vargsiukai" - naudojama pranešimui.
00030  * @throws std::runtime_error Jei failo nepavyksta sukurti.
00031  */
00032 void isvedimas_i_faila(const Vector<Studentas>& studentai,
00033     const std::string& filename,
00034     const std::string& kategorija);
00035
00036 /**
00037  * @brief Apskaičiuoja galutinį rezultatą kiekvienam studentui.
00038  * @param tipas 1 - pagal vidurkį, 2 - pagal medianą.
00039  * @param studentai Studentų vektorius (modifikuojamas).
00040  */
00041 void pasirinkimo_metodas(int tipas, Vector<Studentas>& studentai);
00042
00043 /**
00044  * @brief Surūšiuoja studentų vektorių pagal pasirinktą kriterijų.
00045  * @param studentai Vektorius (modifikuojamas).
00046  * @param budas Rūšiavimo būdas:
00047  *   1 - pagal vardą didėjant,
00048  *   2 - pagal vardą mažėjant,
00049  *   3 - pagal pavardę didėjant,
00050  *   4 - pagal pavardę mažėjant,
00051  *   5 - pagal galutinį rezultatą didėjant,
00052  *   6 - pagal galutinį rezultatą mažėjant.
00053  */
00054 void rusiavimas(Vector<Studentas>& studentai, int budas);
00055
00056 /**
00057  * @brief Paprašo vartotojo pasirinkti rūšiavimo būdą.
00058  * @return Pasirinkimo kodas (1..6).
00059  */
00060 int pasirinkimas_rusiavimo_budo();
00061
00062 /**
00063  * @brief Paprastas skirstymas į dvi grupes (originalas lieka nepakitęs).
00064  * @param visi Visi studentai (const).
00065  * @param kieti [out] Studentai su rez >= 5.0.
00066  * @param vargsai [out] Studentai su rez < 5.0.
00067  */
00068 void skirstymas_i_grupes(const Vector<Studentas>& visi,
00069     Vector<Studentas>& kieti,
00070     Vector<Studentas>& vargsai);
00071
00072 /**
00073  * @brief Strategija S1: du nauji konteineriai, kopijavimas su copy_if.
00074  * @return Sugaištas laikas sekundėmis.
00075  */
00076 double skirstymas_sl(const Vector<Studentas>& studentai,
00077     Vector<Studentas>& kieti,
00078     Vector<Studentas>& vargsai);

```



```

00083
00084 /**
00085  * @brief Strategija S2: vienas naujas + erase/remove_if.
00086  * @return Sugaištas laikas sekundėmis.
00087  */
00088 double skirstymas_s2(Vector<Studentas>& studentai,
00089     Vector<Studentas>& vargsai);
00090 /**
00091  * @brief Strategija S3: std::partition.
00092  * @return Sugaištas laikas sekundėmis.
00093  */
00094
00095 double skirstymas_s3(Vector<Studentas>& studentai,
00096     Vector<Studentas>& vargsai);
00097
00098 #endif

```

5.35 vector_unit_tests.cpp Failo Nuoroda

Google Test testai nuosavai [Vector](#) klasei.

```

#include <gtest/gtest.h>
#include "Vector.h"
#include <vector>
#include <string>
#include <sstream>
#include <numeric>
#include <chrono>

```

Funkcijos

- [TEST](#) (VectorConstructors, DefaultConstructor)
- [TEST](#) (VectorConstructors, CountValueConstructor)
- [TEST](#) (VectorConstructors, CountDefaultConstructor)
- [TEST](#) (VectorConstructors, RangeConstructor)
- [TEST](#) (VectorConstructors, InitializerListConstructor)
- [TEST](#) (VectorConstructors, CopyConstructor)
- [TEST](#) (VectorConstructors, MoveConstructor)
- [TEST](#) (VectorAssignment, CopyAssignment)
- [TEST](#) (VectorAssignment, MoveAssignment)
- [TEST](#) (VectorAssignment, InitializerListAssignment)
- [TEST](#) (VectorAssignment, SelfAssignment)
- [TEST](#) (VectorAssignment, AssignCountValue)
- [TEST](#) (VectorAssignment, AssignRange)
- [TEST](#) (VectorAssignment, AssignInitializerList)
- [TEST](#) (VectorElementAccess, OperatorBrackets)
- [TEST](#) (VectorElementAccess, AtInBounds)
- [TEST](#) (VectorElementAccess, AtOutOfBounds)
- [TEST](#) (VectorElementAccess, FrontBack)
- [TEST](#) (VectorElementAccess, DataPointer)
- [TEST](#) (VectorIterators, BeginEnd)
- [TEST](#) (VectorIterators, RangeFor)
- [TEST](#) (VectorIterators, ConstIterator)
- [TEST](#) (VectorIterators, ReverselIterator)
- [TEST](#) (VectorIterators, StdAlgorithms)
- [TEST](#) (VectorCapacity, Empty)
- [TEST](#) (VectorCapacity, Size)

- [TEST](#) (VectorCapacity, Reserve)
- [TEST](#) (VectorCapacity, ReserveNoShrink)
- [TEST](#) (VectorCapacity, ShrinkToFit)
- [TEST](#) (VectorCapacity, CapacityGrowth)
- [TEST](#) (VectorModifiers, Clear)
- [TEST](#) (VectorModifiers, PushBack)
- [TEST](#) (VectorModifiers, PushBackMove)
- [TEST](#) (VectorModifiers, PopBack)
- [TEST](#) (VectorModifiers, InsertSingle)
- [TEST](#) (VectorModifiers, InsertCount)
- [TEST](#) (VectorModifiers, InsertRange)
- [TEST](#) (VectorModifiers, InsertInitializerList)
- [TEST](#) (VectorModifiers, EraseSingle)
- [TEST](#) (VectorModifiers, EraseRange)
- [TEST](#) (VectorModifiers, EmplaceBack)
- [TEST](#) (VectorModifiers, Emplace)
- [TEST](#) (VectorModifiers, ResizeSmaller)
- [TEST](#) (VectorModifiers, ResizeLarger)
- [TEST](#) (VectorModifiers, Swap)
- [TEST](#) (VectorComparison, Equality)
- [TEST](#) (VectorComparison, LexicographicOrder)
- [TEST](#) (VectorNonMember, GlobalSwap)
- [TEST](#) (VectorNonMember, EraseByValue)
- [TEST](#) (VectorNonMember, EraseIf)
- [TEST](#) (VectorReallocation, CountReallocations)
- [TEST](#) (VectorReallocation, CompareWithStdVector)
- [TEST](#) (VectorPerformance, PushBack10k)
- [TEST](#) (VectorPerformance, PushBack100k)
- [TEST](#) (VectorPerformance, PushBack1M)
- [TEST](#) (VectorPerformance, StdVectorPushBack1M)
- [TEST](#) (VectorEdgeCases, EmptyVectorIterators)
- [TEST](#) (VectorEdgeCases, MoveAndPushBack)
- [TEST](#) (VectorEdgeCases, StringElements)
- [TEST](#) (VectorEdgeCases, NestedVector)
- [TEST](#) (VectorEdgeCases, ReserveBeforePush)

5.35.1 Smulkus aprašymas

Google Test testai nuosavai [Vector](#) klasei.

Patikrina visą [Vector](#) funkcionalumą pagal std::vector specifikaciją. Kiekviena testuojama grupė atitinka std::vector Member functions sekciją.

Apibrėžimas faile [vector_unit_tests.cpp](#).

5.35.2 Funkcijos Dokumentacija

5.35.2.1 TEST() [1/61]

```
TEST (
    VectorAssignment ,
    AssignCountValue )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 110.

```
00110                                     {
00111     Vector<int> v = {1, 2, 3};
00112     v.assign(5, 7);
00113     EXPECT_EQ(v.size(), 5u);
00114     for (size_t i = 0; i < 5; ++i)
00115         EXPECT_EQ(v[i], 7);
00116 }
```

5.35.2.2 TEST() [2/61]

```
TEST (
    VectorAssignment ,
    AssignInitializerList )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 126.

```
00126                                     {
00127     Vector<int> v;
00128     v.assign({100, 200, 300});
00129     EXPECT_EQ(v.size(), 3u);
00130     EXPECT_EQ(v[2], 300);
00131 }
```

5.35.2.3 TEST() [3/61]

```
TEST (
    VectorAssignment ,
    AssignRange )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 118.

```
00118                                     {
00119     std::vector<int> src = {10, 20, 30};
00120     Vector<int> v;
00121     v.assign(src.begin(), src.end());
00122     EXPECT_EQ(v.size(), 3u);
00123     EXPECT_EQ(v[1], 20);
00124 }
```

5.35.2.4 TEST() [4/61]

```
TEST (
    VectorAssignment ,
    CopyAssignment )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 78.

```
00078                                     {
00079     Vector<int> a = {1, 2, 3};
00080     Vector<int> b;
00081     b = a;
00082     EXPECT_EQ(b.size(), 3u);
00083     a[0] = 99;
00084     EXPECT_EQ(b[0], 1); // gili kopija
00085 }
```

5.35.2.5 TEST() [5/61]

```
TEST (
    VectorAssignment ,
    InitializerListAssignment )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 96.

```
00096                                     {
00097     Vector<int> v;
00098     v = {7, 8, 9};
00099     EXPECT_EQ(v.size(), 3u);
00100     EXPECT_EQ(v[2], 9);
00101 }
```

5.35.2.6 TEST() [6/61]

```
TEST (
    VectorAssignment ,
    MoveAssignment )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 87.

```
00087                                     {
00088     Vector<int> a = {4, 5, 6};
00089     Vector<int> b;
00090     b = std::move(a);
00091     EXPECT_EQ(b.size(), 3u);
00092     EXPECT_EQ(b[0], 4);
00093     EXPECT_EQ(a.size(), 0u);
00094 }
```

5.35.2.7 TEST() [7/61]

```
TEST (
    VectorAssignment ,
    SelfAssignment )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 103.

```
00103                                     {
00104     Vector<int> v = {1, 2, 3};
00105     v = v; // neturi sukelti klaidos
00106     EXPECT_EQ(v.size(), 3u);
00107     EXPECT_EQ(v[0], 1);
00108 }
```

5.35.2.8 TEST() [8/61]

```
TEST (
    VectorCapacity ,
    CapacityGrowth )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 251.

```
00251                                     {
00252     Vector<int> v;
00253     size_t reallocs = 0;
00254     size_t prev_cap = 0;
00255     for (int i = 0; i < 1000; ++i) {
00256         if (v.capacity() == v.size()) ++reallocs;
00257         v.push_back(i);
00258         prev_cap = v.capacity();
00259     }
00260     // Turėtų būti logaritminis persikirstymų skaičius
00261     EXPECT_LT(reallocs, 25u); // <= log2(1000) + 1
00262 }
```

5.35.2.9 TEST() [9/61]

```
TEST (
    VectorCapacity ,
    Empty )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 212.

```
00212                                     {
00213     Vector<int> v;
00214     EXPECT_TRUE(v.empty());
00215     v.push_back(1);
00216     EXPECT_FALSE(v.empty());
00217 }
```

5.35.2.10 TEST() [10/61]

```
TEST (
    VectorCapacity ,
    Reserve )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 227.

```
00227 {
00228     Vector<int> v;
00229     v.reserve(100);
00230     EXPECT_GE(v.capacity(), 100u);
00231     EXPECT_EQ(v.size(), 0u); // dydis nepakito
00232 }
```

5.35.2.11 TEST() [11/61]

```
TEST (
    VectorCapacity ,
    ReserveNoShrink )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 234.

```
00234 {
00235     Vector<int> v;
00236     v.reserve(100);
00237     size_t cap = v.capacity();
00238     v.reserve(10); // mažesnis - neturėtų sumažėti
00239     EXPECT_EQ(v.capacity(), cap);
00240 }
```

5.35.2.12 TEST() [12/61]

```
TEST (
    VectorCapacity ,
    ShrinkToFit )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 242.

```
00242 {
00243     Vector<int> v;
00244     v.reserve(100);
00245     v.push_back(1);
00246     v.push_back(2);
00247     v.shrink_to_fit();
00248     EXPECT_EQ(v.capacity(), v.size());
00249 }
```

5.35.2.13 TEST() [13/61]

```
TEST (
    VectorCapacity ,
    Size )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 219.

```
00219 {
00220     Vector<int> v;
00221     EXPECT_EQ(v.size(), 0u);
00222     v.push_back(1);
00223     v.push_back(2);
00224     EXPECT_EQ(v.size(), 2u);
00225 }
```

5.35.2.14 TEST() [14/61]

```
TEST (
    VectorComparison ,
    Equality )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 389.

```
00389                                     {
00390     Vector<int> a = {1, 2, 3};
00391     Vector<int> b = {1, 2, 3};
00392     Vector<int> c = {1, 2, 4};
00393     EXPECT_TRUE(a == b);
00394     EXPECT_FALSE(a == c);
00395     EXPECT_TRUE(a != c);
00396 }
```

5.35.2.15 TEST() [15/61]

```
TEST (
    VectorComparison ,
    LexicographicOrder )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 398.

```
00398                                     {
00399     Vector<int> a = {1, 2, 3};
00400     Vector<int> b = {1, 2, 4};
00401     Vector<int> c = {1, 2, 3, 0};
00402     EXPECT_TRUE(a < b);
00403     EXPECT_TRUE(a < c);
00404     EXPECT_TRUE(b > a);
00405     EXPECT_TRUE(a <= a);
00406     EXPECT_TRUE(a >= a);
00407 }
```

5.35.2.16 TEST() [16/61]

```
TEST (
    VectorConstructors ,
    CopyConstructor )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 57.

```
00057                                     {
00058     Vector<int> a = {1, 2, 3};
00059     Vector<int> b(a);
00060     EXPECT_EQ(b.size(), 3u);
00061     EXPECT_EQ(b[0], 1);
00062     // Gili kopija - pakeitimas a neturi itakos b
00063     a[0] = 99;
00064     EXPECT_EQ(b[0], 1);
00065 }
```

5.35.2.17 TEST() [17/61]

```
TEST (
    VectorConstructors ,
    CountDefaultConstructor )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 34.

```
00034                                     {
00035     Vector<int> v(4);
00036     EXPECT_EQ(v.size(), 4u);
00037     for (size_t i = 0; i < 4; ++i)
00038         EXPECT_EQ(v[i], 0);
00039 }
```

5.35.2.18 TEST() [18/61]

```
TEST (
    VectorConstructors ,
    CountValueConstructor )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 27.

```
00027                                     {
00028     Vector<int> v(5, 42);
00029     EXPECT_EQ(v.size(), 5u);
00030     for (size_t i = 0; i < 5; ++i)
00031         EXPECT_EQ(v[i], 42);
00032 }
```

5.35.2.19 TEST() [19/61]

```
TEST (
    VectorConstructors ,
    DefaultConstructor )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 20.

```
00020                                     {
00021     Vector<int> v;
00022     EXPECT_EQ(v.size(), 0u);
00023     EXPECT_EQ(v.capacity(), 0u);
00024     EXPECT_TRUE(v.empty());
00025 }
```

5.35.2.20 TEST() [20/61]

```
TEST (
    VectorConstructors ,
   _INITIALIZER_LIST_CONSTRUCTOR )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 49.

```
00049                                     {
00050     Vector<int> v = {10, 20, 30};
00051     EXPECT_EQ(v.size(), 3u);
00052     EXPECT_EQ(v[0], 10);
00053     EXPECT_EQ(v[1], 20);
00054     EXPECT_EQ(v[2], 30);
00055 }
```

5.35.2.21 TEST() [21/61]

```
TEST (
    VectorConstructors ,
    MoveConstructor )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 67.

```
00067                                     {
00068     Vector<int> a = {1, 2, 3};
00069     Vector<int> b(std::move(a));
00070     EXPECT_EQ(b.size(), 3u);
00071     EXPECT_EQ(b[0], 1);
00072     EXPECT_EQ(a.size(), 0u); // originalas tuscias po perkélimo
00073     EXPECT_EQ(a.data(), nullptr);
00074 }
```

5.35.2.22 TEST() [22/61]

```
TEST (
    VectorConstructors ,
    RangeConstructor )
```

Apibrēžimas failo [vector_unit_tests.cpp](#) eilutėje 41.

```
00041 {
00042     std::vector<int> src = {1, 2, 3, 4, 5};
00043     Vector<int> v(src.begin(), src.end());
00044     EXPECT_EQ(v.size(), 5u);
00045     for (size_t i = 0; i < 5; ++i)
00046         EXPECT_EQ(v[i], (int)(i + 1));
00047 }
```

5.35.2.23 TEST() [23/61]

```
TEST (
    VectorEdgeCases ,
    EmptyVectorIterators )
```

Apibrēžimas failo [vector_unit_tests.cpp](#) eilutėje 520.

```
00520 {
00521     Vector<int> v;
00522     EXPECT_EQ(v.begin(), v.end());
00523     EXPECT_EQ(v.rbegin(), v.rend());
00524 }
```

5.35.2.24 TEST() [24/61]

```
TEST (
    VectorEdgeCases ,
    MoveAndPushBack )
```

Apibrēžimas failo [vector_unit_tests.cpp](#) eilutėje 526.

```
00526 {
00527     Vector<int> a = {1, 2, 3};
00528     Vector<int> b = std::move(a);
00529     b.push_back(4);
00530     EXPECT_EQ(b.size(), 4u);
00531     EXPECT_EQ(b[3], 4);
00532 }
```

5.35.2.25 TEST() [25/61]

```
TEST (
    VectorEdgeCases ,
    NestedVector )
```

Apibrēžimas failo [vector_unit_tests.cpp](#) eilutėje 545.

```
00545 {
00546     Vector<Vector<int>> v;
00547     v.push_back({1, 2, 3});
00548     v.push_back({4, 5});
00549     EXPECT_EQ(v.size(), 2u);
00550     EXPECT_EQ(v[0].size(), 3u);
00551     EXPECT_EQ(v[1][0], 4);
00552 }
```


5.35.2.26 TEST() [26/61]

```
TEST (
    VectorEdgeCases ,
    ReserveBeforePush )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 554.

```
00554 {
00555     Vector<int> v;
00556     v.reserve(1000);
00557     size_t cap = v.capacity();
00558     for (int i = 0; i < 1000; ++i) v.push_back(i);
00559     // Neturėtų būti perskirstymo
00560     EXPECT_EQ(v.capacity(), cap);
00561     EXPECT_EQ(v.size(), 1000u);
00562 }
```

5.35.2.27 TEST() [27/61]

```
TEST (
    VectorEdgeCases ,
    StringElements )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 534.

```
00534 {
00535     Vector<std::string> v;
00536     v.push_back("labas");
00537     v.push_back("pasauli");
00538     EXPECT_EQ(v.size(), 2u);
00539     EXPECT_EQ(v[0], "labas");
00540     v.insert(v.begin() + 1, "grazus");
00541     EXPECT_EQ(v[1], "grazus");
00542     EXPECT_EQ(v[2], "pasauli");
00543 }
```

5.35.2.28 TEST() [28/61]

```
TEST (
    VectorElementAccess ,
    AtInBounds )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 142.

```
00142 {
00143     Vector<int> v = {1, 2, 3};
00144     EXPECT_EQ(v.at(0), 1);
00145     EXPECT_EQ(v.at(2), 3);
00146 }
```

5.35.2.29 TEST() [29/61]

```
TEST (
    VectorElementAccess ,
    AtOutOfBounds )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 148.

```
00148 {
00149     Vector<int> v = {1, 2, 3};
00150     EXPECT_THROW(v.at(3), std::out_of_range);
00151     EXPECT_THROW(v.at(100), std::out_of_range);
00152 }
```

5.35.2.30 TEST() [30/61]

```
TEST (
    VectorElementAccess ,
    DataPointer )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 162.

```
00162                                     {
00163     Vector<int> v = {1, 2, 3};
00164     int* ptr = v.data();
00165     EXPECT_EQ(ptr[0], 1);
00166     ptr[0] = 42;
00167     EXPECT_EQ(v[0], 42);
00168 }
```

5.35.2.31 TEST() [31/61]

```
TEST (
    VectorElementAccess ,
    FrontBack )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 154.

```
00154                                     {
00155     Vector<int> v = {5, 6, 7};
00156     EXPECT_EQ(v.front(), 5);
00157     EXPECT_EQ(v.back(), 7);
00158     v.front() = 99;
00159     EXPECT_EQ(v[0], 99);
00160 }
```

5.35.2.32 TEST() [32/61]

```
TEST (
    VectorElementAccess ,
    OperatorBrackets )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 135.

```
00135                                     {
00136     Vector<int> v = {10, 20, 30};
00137     EXPECT_EQ(v[0], 10);
00138     v[1] = 99;
00139     EXPECT_EQ(v[1], 99);
00140 }
```

5.35.2.33 TEST() [33/61]

```
TEST (
    VectorIterators ,
    BeginEnd )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 172.

```
00172                                     {
00173     Vector<int> v = {1, 2, 3, 4, 5};
00174     int sum = 0;
00175     for (auto it = v.begin(); it != v.end(); ++it)
00176         sum += *it;
00177     EXPECT_EQ(sum, 15);
00178 }
```

5.35.2.34 TEST() [34/61]

```
TEST (
    VectorIterators ,
    ConstIterator )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 187.

```
00187 {
00188     const Vector<int> v = {10, 20, 30};
00189     int sum = 0;
00190     for (auto it = v.cbegin(); it != v.cend(); ++it)
00191         sum += *it;
00192     EXPECT_EQ(sum, 60);
00193 }
```

5.35.2.35 TEST() [35/61]

```
TEST (
    VectorIterators ,
    RangeFor )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 180.

```
00180 {
00181     Vector<int> v = {2, 4, 6};
00182     int sum = 0;
00183     for (const auto& x : v) sum += x;
00184     EXPECT_EQ(sum, 12);
00185 }
```

5.35.2.36 TEST() [36/61]

```
TEST (
    VectorIterators ,
    ReverseIterator )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 195.

```
00195 {
00196     Vector<int> v = {1, 2, 3};
00197     std::vector<int> rev(v.rbegin(), v.rend());
00198     EXPECT_EQ(rev[0], 3);
00199     EXPECT_EQ(rev[1], 2);
00200     EXPECT_EQ(rev[2], 1);
00201 }
```

5.35.2.37 TEST() [37/61]

```
TEST (
    VectorIterators ,
    StdAlgorithms )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 203.

```
00203 {
00204     Vector<int> v = {5, 3, 1, 4, 2};
00205     std::sort(v.begin(), v.end());
00206     EXPECT_EQ(v[0], 1);
00207     EXPECT_EQ(v[4], 5);
00208 }
```

5.35.2.38 TEST() [38/61]

```
TEST (
    VectorModifiers ,
    Clear )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 266.

```
00266     {
00267         Vector<int> v = {1, 2, 3};
00268         v.clear();
00269         EXPECT_EQ(v.size(), 0u);
00270         EXPECT_TRUE(v.empty());
00271         // Capacity išsaugoma
00272         EXPECT_GE(v.capacity(), 3u);
00273     }
```

5.35.2.39 TEST() [39/61]

```
TEST (
    VectorModifiers ,
    Emerge )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 355.

```
00355     {
00356         Vector<std::pair<int,int>> v = {{1,2}, {5,6}};
00357         v.emplace(v.begin() + 1, 3, 4);
00358         EXPECT_EQ(v.size(), 3u);
00359         EXPECT_EQ(v[1].first, 3);
00360     }
```

5.35.2.40 TEST() [40/61]

```
TEST (
    VectorModifiers ,
    EmergeBack )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 346.

```
00346     {
00347         Vector<std::pair<int,int>> v;
00348         v.emplace_back(1, 2);
00349         v.emplace_back(3, 4);
00350         EXPECT_EQ(v.size(), 2u);
00351         EXPECT_EQ(v[0].first, 1);
00352         EXPECT_EQ(v[1].second, 4);
00353     }
```

5.35.2.41 TEST() [41/61]

```
TEST (
    VectorModifiers ,
    EraseRange )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 338.

```
00338     {
00339         Vector<int> v = {1, 2, 3, 4, 5};
00340         v.erase(v.begin() + 1, v.begin() + 4); // istrina 2,3,4
00341         EXPECT_EQ(v.size(), 2u);
00342         EXPECT_EQ(v[0], 1);
00343         EXPECT_EQ(v[1], 5);
00344     }
```

5.35.2.42 TEST() [42/61]

```
TEST (
    VectorModifiers ,
    EraseSingle )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 330.

```
00330 {
00331     Vector<int> v = {1, 2, 3, 4};
00332     auto it = v.erase(v.begin() + 1);
00333     EXPECT_EQ(v.size(), 3u);
00334     EXPECT_EQ(v[1], 3);
00335     EXPECT_EQ(*it, 3);
00336 }
```

5.35.2.43 TEST() [43/61]

```
TEST (
    VectorModifiers ,
    InsertCount )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 305.

```
00305 {
00306     Vector<int> v = {1, 5};
00307     v.insert(v.begin() + 1, 3, 2); // idesime 3 x "2"
00308     EXPECT_EQ(v.size(), 5u);
00309     EXPECT_EQ(v[1], 2);
00310     EXPECT_EQ(v[2], 2);
00311     EXPECT_EQ(v[3], 2);
00312     EXPECT_EQ(v[4], 5);
00313 }
```

5.35.2.44 TEST() [44/61]

```
TEST (
    VectorModifiers ,
    InsertInitializerList )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 323.

```
00323 {
00324     Vector<int> v = {1, 5};
00325     v.insert(v.begin() + 1, {2, 3, 4});
00326     EXPECT_EQ(v.size(), 5u);
00327     for (int i = 0; i < 5; ++i) EXPECT_EQ(v[i], i + 1);
00328 }
```

5.35.2.45 TEST() [45/61]

```
TEST (
    VectorModifiers ,
    InsertRange )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 315.

```
00315 {
00316     Vector<int> v = {1, 5};
00317     std::vector<int> src = {2, 3, 4};
00318     v.insert(v.begin() + 1, src.begin(), src.end());
00319     EXPECT_EQ(v.size(), 5u);
00320     for (int i = 0; i < 5; ++i) EXPECT_EQ(v[i], i + 1);
00321 }
```

5.35.2.46 TEST() [46/61]

```
TEST (
    VectorModifiers ,
    InsertSingle )
```

Apibrazimas failo [vector_unit_tests.cpp](#) eilutėje 297.

```
00297 {
00298     Vector<int> v = {1, 2, 4, 5};
00299     auto it = v.insert(v.begin() + 2, 3);
00300     EXPECT_EQ(v.size(), 5u);
00301     EXPECT_EQ(v[2], 3);
00302     EXPECT_EQ(*it, 3);
00303 }
```

5.35.2.47 TEST() [47/61]

```
TEST (
    VectorModifiers ,
    PopBack )
```

Apibrazimas failo [vector_unit_tests.cpp](#) eilutėje 290.

```
00290 {
00291     Vector<int> v = {1, 2, 3};
00292     v.pop_back();
00293     EXPECT_EQ(v.size(), 2u);
00294     EXPECT_EQ(v.back(), 2);
00295 }
```

5.35.2.48 TEST() [48/61]

```
TEST (
    VectorModifiers ,
    PushBack )
```

Apibrazimas failo [vector_unit_tests.cpp](#) eilutėje 275.

```
00275 {
00276     Vector<int> v;
00277     for (int i = 0; i < 10; ++i) v.push_back(i);
00278     EXPECT_EQ(v.size(), 10u);
00279     EXPECT_EQ(v[9], 9);
00280 }
```

5.35.2.49 TEST() [49/61]

```
TEST (
    VectorModifiers ,
    PushBackMove )
```

Apibrazimas failo [vector_unit_tests.cpp](#) eilutėje 282.

```
00282 {
00283     Vector<std::string> v;
00284     std::string s = "testas";
00285     v.push_back(std::move(s));
00286     EXPECT_EQ(v[0], "testas");
00287     EXPECT_TRUE(s.empty() || s == ""); // po perkėlimo
00288 }
```

5.35.2.50 TEST() [50/61]

```
TEST (
    VectorModifiers ,
    ResizeLarger )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 369.

```
00369 {
00370     Vector<int> v = {1, 2};
00371     v.resize(5, 99);
00372     EXPECT_EQ(v.size(), 5u);
00373     EXPECT_EQ(v[2], 99);
00374     EXPECT_EQ(v[4], 99);
00375 }
```

5.35.2.51 TEST() [51/61]

```
TEST (
    VectorModifiers ,
    ResizeSmaller )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 362.

```
00362 {
00363     Vector<int> v = {1, 2, 3, 4, 5};
00364     v.resize(3);
00365     EXPECT_EQ(v.size(), 3u);
00366     EXPECT_EQ(v[2], 3);
00367 }
```

5.35.2.52 TEST() [52/61]

```
TEST (
    VectorModifiers ,
    Swap )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 377.

```
00377 {
00378     Vector<int> a = {1, 2, 3};
00379     Vector<int> b = {4, 5};
00380     a.swap(b);
00381     EXPECT_EQ(a.size(), 2u);
00382     EXPECT_EQ(b.size(), 3u);
00383     EXPECT_EQ(a[0], 4);
00384     EXPECT_EQ(b[0], 1);
00385 }
```

5.35.2.53 TEST() [53/61]

```
TEST (
    VectorNonMember ,
    EraseByValue )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 419.

```
00419 {
00420     Vector<int> v = {1, 2, 3, 2, 4, 2};
00421     auto removed = erase(v, 2);
00422     EXPECT_EQ(removed, 3u);
00423     EXPECT_EQ(v.size(), 3u);
00424     for (const auto& x : v) EXPECT_NE(x, 2);
00425 }
```

5.35.2.54 TEST() [54/61]

```
TEST (
    VectorNonMember ,
    EraseIf )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 427.

```
00427 {
00428     Vector<int> v = {1, 2, 3, 4, 5, 6};
00429     auto removed = erase_if(v, [](int x) { return x % 2 == 0; });
00430     EXPECT_EQ(removed, 3u);
00431     EXPECT_EQ(v.size(), 3u);
00432     for (const auto& x : v) EXPECT_EQ(x % 2, 1);
00433 }
```

5.35.2.55 TEST() [55/61]

```
TEST (
    VectorNonMember ,
    GlobalSwap )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 411.

```
00411 {
00412     Vector<int> a = {1, 2};
00413     Vector<int> b = {3, 4, 5};
00414     swap(a, b);
00415     EXPECT_EQ(a.size(), 3u);
00416     EXPECT_EQ(b.size(), 2u);
00417 }
```

5.35.2.56 TEST() [56/61]

```
TEST (
    VectorPerformance ,
    PushBack100k )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 488.

```
00488 {
00489     auto t0 = std::chrono::high_resolution_clock::now();
00490     Vector<int> v;
00491     for (int i = 1; i <= 100000; ++i) v.push_back(i);
00492     auto dt = std::chrono::duration<double>{
00493         std::chrono::high_resolution_clock::now() - t0}.count();
00494     EXPECT_EQ(v.size(), 100000u);
00495     std::cout << "Vector push_back 100k: " << dt << " s\n";
00496 }
```

5.35.2.57 TEST() [57/61]

```
TEST (
    VectorPerformance ,
    PushBack10k )
```

Apibrēzimas failo [vector_unit_tests.cpp](#) eilutėje 477.

```
00477 {
00478     auto t0 = std::chrono::high_resolution_clock::now();
00479     Vector<int> v;
00480     for (int i = 1; i <= 10000; ++i) v.push_back(i);
00481     auto dt = std::chrono::duration<double>{
00482         std::chrono::high_resolution_clock::now() - t0}.count();
00483     EXPECT_EQ(v.size(), 10000u);
00484     EXPECT_EQ(v.back(), 10000);
00485     std::cout << "Vector push_back 10k: " << dt << " s\n";
00486 }
```


5.35.2.58 TEST() [58/61]

```
TEST (
    VectorPerformance ,
    PushBack1M )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 498.

```
00498      {
00499      auto t0 = std::chrono::high_resolution_clock::now();
00500      Vector<int> v;
00501      for (int i = 1; i <= 1000000; ++i) v.push_back(i);
00502      auto dt = std::chrono::duration<double>{
00503          std::chrono::high_resolution_clock::now() - t0}.count();
00504      EXPECT_EQ(v.size(), 1000000u);
00505      std::cout << "Vector push_back 1M: " << dt << " s\n";
00506  }
```

5.35.2.59 TEST() [59/61]

```
TEST (
    VectorPerformance ,
    StdVectorPushBack1M )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 508.

```
00508      {
00509      auto t0 = std::chrono::high_resolution_clock::now();
00510      std::vector<int> v;
00511      for (int i = 1; i <= 1000000; ++i) v.push_back(i);
00512      auto dt = std::chrono::duration<double>{
00513          std::chrono::high_resolution_clock::now() - t0}.count();
00514      EXPECT_EQ(v.size(), 1000000u);
00515      std::cout << "std::vector push_back 1M: " << dt << " s\n";
00516  }
```

5.35.2.60 TEST() [60/61]

```
TEST (
    VectorReallocation ,
    CompareWithStdVector )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 454.

```
00454      {
00455      // Lyginti perskirstymų skaičių su std::vector
00456      Vector<int> myV;
00457      std::vector<int> stdV;
00458      int myReallocs = 0, stdReallocs = 0;
00459      size_t myCap = 0, stdCap = 0;
00460
00461      for (int i = 1; i <= 1000000; ++i) {
00462          if (myV.capacity() == myV.size()) ++myReallocs;
00463          if (stdV.capacity() == stdV.size()) ++stdReallocs;
00464          myV.push_back(i);
00465          stdV.push_back(i);
00466      }
00467
00468      std::cout << "Vector perskirstymai (1M el.):      mano=" << myReallocs
00469          << ", std=" << stdReallocs << "\n";
00470      // Abu turėtų turėti panašų skaičių
00471      EXPECT_LT(myReallocs, 30);
00472      EXPECT_LT(stdReallocs, 30);
00473  }
```

5.35.2.61 TEST() [61/61]

```
TEST (
    VectorReallocation ,
    CountReallocations )
```

Apibrėžimas failo [vector_unit_tests.cpp](#) eilutėje 437.

```
00437                                     {
00438     Vector<int> v;
00439     int reallocs = 0;
00440     size_t prev_cap = v.capacity();
00441
00442     for (int i = 1; i <= 100000; ++i) {
00443         v.push_back(i);
00444         if (v.capacity() != prev_cap) {
00445             ++reallocs;
00446             prev_cap = v.capacity();
00447         }
00448     }
00449     // std::vector paprastai daro ~17 perskirstymų 100000 elementams
00450     EXPECT_LT(reallocs, 25);
00451     std::cout << "Vector perskirstymu skaicus (100000 el.): " << reallocs << "\n";
00452 }
```

5.36 vector_unit_tests.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file vector_unit_tests.cpp
00003  * @brief Google Test testai nuosavai Vector klasei.
00004  *
00005  * Patikrina visą Vector funkcionalumą pagal std::vector specifikaciją.
00006  * Kiekviena testuojama grupė atitinka std::vector Member functions sekciją.
00007  */
00008
00009 #include <gtest/gtest.h>
00010 #include "Vector.h"
00011 #include <vector>
00012 #include <string>
00013 #include <sstream>
00014 #include <numeric>
00015 #include <chrono>
00016
00017 // 1. konstruktoriai
00018
00019
00020 TEST(VectorConstructors, DefaultConstructor) {
00021     Vector<int> v;
00022     EXPECT_EQ(v.size(), 0u);
00023     EXPECT_EQ(v.capacity(), 0u);
00024     EXPECT_TRUE(v.empty());
00025 }
00026
00027 TEST(VectorConstructors, CountValueConstructor) {
00028     Vector<int> v(5, 42);
00029     EXPECT_EQ(v.size(), 5u);
00030     for (size_t i = 0; i < 5; ++i)
00031         EXPECT_EQ(v[i], 42);
00032 }
00033
00034 TEST(VectorConstructors, CountDefaultConstructor) {
00035     Vector<int> v(4);
00036     EXPECT_EQ(v.size(), 4u);
00037     for (size_t i = 0; i < 4; ++i)
00038         EXPECT_EQ(v[i], 0);
00039 }
00040
00041 TEST(VectorConstructors, RangeConstructor) {
00042     std::vector<int> src = {1, 2, 3, 4, 5};
00043     Vector<int> v(src.begin(), src.end());
00044     EXPECT_EQ(v.size(), 5u);
00045     for (size_t i = 0; i < 5; ++i)
00046         EXPECT_EQ(v[i], (int)(i + 1));
00047 }
00048
00049 TEST(VectorConstructors, InitializerListConstructor) {
```

```

00050     Vector<int> v = {10, 20, 30};
00051     EXPECT_EQ(v.size(), 3u);
00052     EXPECT_EQ(v[0], 10);
00053     EXPECT_EQ(v[1], 20);
00054     EXPECT_EQ(v[2], 30);
00055 }
00056
00057 TEST(VectorConstructors, CopyConstructor) {
00058     Vector<int> a = {1, 2, 3};
00059     Vector<int> b(a);
00060     EXPECT_EQ(b.size(), 3u);
00061     EXPECT_EQ(b[0], 1);
00062     // Gili kopija - pakeitimas a neturi itakos b
00063     a[0] = 99;
00064     EXPECT_EQ(b[0], 1);
00065 }
00066
00067 TEST(VectorConstructors, MoveConstructor) {
00068     Vector<int> a = {1, 2, 3};
00069     Vector<int> b(std::move(a));
00070     EXPECT_EQ(b.size(), 3u);
00071     EXPECT_EQ(b[0], 1);
00072     EXPECT_EQ(a.size(), 0u); // originalas tuscias po perkélimo
00073     EXPECT_EQ(a.data(), nullptr);
00074 }
00075
00076 // 2. priskyrimo operatoriai
00077
00078 TEST(VectorAssignment, CopyAssignment) {
00079     Vector<int> a = {1, 2, 3};
00080     Vector<int> b;
00081     b = a;
00082     EXPECT_EQ(b.size(), 3u);
00083     a[0] = 99;
00084     EXPECT_EQ(b[0], 1); // gili kopija
00085 }
00086
00087 TEST(VectorAssignment, MoveAssignment) {
00088     Vector<int> a = {4, 5, 6};
00089     Vector<int> b;
00090     b = std::move(a);
00091     EXPECT_EQ(b.size(), 3u);
00092     EXPECT_EQ(b[0], 4);
00093     EXPECT_EQ(a.size(), 0u);
00094 }
00095
00096 TEST(VectorAssignment, InitializerListAssignment) {
00097     Vector<int> v;
00098     v = {7, 8, 9};
00099     EXPECT_EQ(v.size(), 3u);
00100     EXPECT_EQ(v[2], 9);
00101 }
00102
00103 TEST(VectorAssignment, SelfAssignment) {
00104     Vector<int> v = {1, 2, 3};
00105     v = v; // neturi sukelti klaidos
00106     EXPECT_EQ(v.size(), 3u);
00107     EXPECT_EQ(v[0], 1);
00108 }
00109
00110 TEST(VectorAssignment, AssignCountValue) {
00111     Vector<int> v = {1, 2, 3};
00112     v.assign(5, 7);
00113     EXPECT_EQ(v.size(), 5u);
00114     for (size_t i = 0; i < 5; ++i)
00115         EXPECT_EQ(v[i], 7);
00116 }
00117
00118 TEST(VectorAssignment, AssignRange) {
00119     std::vector<int> src = {10, 20, 30};
00120     Vector<int> v;
00121     v.assign(src.begin(), src.end());
00122     EXPECT_EQ(v.size(), 3u);
00123     EXPECT_EQ(v[1], 20);
00124 }
00125
00126 TEST(VectorAssignment, AssignInitializerList) {
00127     Vector<int> v;
00128     v.assign({100, 200, 300});
00129     EXPECT_EQ(v.size(), 3u);
00130     EXPECT_EQ(v[2], 300);
00131 }
00132
00133 // 3. elementu prieiga
00134
00135 TEST(VectorElementAccess, OperatorBrackets) {
00136     Vector<int> v = {10, 20, 30};

```

```

00137     EXPECT_EQ(v[0], 10);
00138     v[1] = 99;
00139     EXPECT_EQ(v[1], 99);
00140 }
00141
00142 TEST(VectorElementAccess, AtInBounds) {
00143     Vector<int> v = {1, 2, 3};
00144     EXPECT_EQ(v.at(0), 1);
00145     EXPECT_EQ(v.at(2), 3);
00146 }
00147
00148 TEST(VectorElementAccess, AtOutOfBounds) {
00149     Vector<int> v = {1, 2, 3};
00150     EXPECT_THROW(v.at(3), std::out_of_range);
00151     EXPECT_THROW(v.at(100), std::out_of_range);
00152 }
00153
00154 TEST(VectorElementAccess, FrontBack) {
00155     Vector<int> v = {5, 6, 7};
00156     EXPECT_EQ(v.front(), 5);
00157     EXPECT_EQ(v.back(), 7);
00158     v.front() = 99;
00159     EXPECT_EQ(v[0], 99);
00160 }
00161
00162 TEST(VectorElementAccess, DataPointer) {
00163     Vector<int> v = {1, 2, 3};
00164     int* ptr = v.data();
00165     EXPECT_EQ(ptr[0], 1);
00166     ptr[0] = 42;
00167     EXPECT_EQ(v[0], 42);
00168 }
00169
00170 // 4. iteratoriai
00171
00172 TEST(VectorIterators, BeginEnd) {
00173     Vector<int> v = {1, 2, 3, 4, 5};
00174     int sum = 0;
00175     for (auto it = v.begin(); it != v.end(); ++it)
00176         sum += *it;
00177     EXPECT_EQ(sum, 15);
00178 }
00179
00180 TEST(VectorIterators, RangeFor) {
00181     Vector<int> v = {2, 4, 6};
00182     int sum = 0;
00183     for (const auto& x : v) sum += x;
00184     EXPECT_EQ(sum, 12);
00185 }
00186
00187 TEST(VectorIterators, ConstIterator) {
00188     const Vector<int> v = {10, 20, 30};
00189     int sum = 0;
00190     for (auto it = v.cbegin(); it != v.cend(); ++it)
00191         sum += *it;
00192     EXPECT_EQ(sum, 60);
00193 }
00194
00195 TEST(VectorIterators, ReverseIterator) {
00196     Vector<int> v = {1, 2, 3};
00197     std::vector<int> rev(v.rbegin(), v.rend());
00198     EXPECT_EQ(rev[0], 3);
00199     EXPECT_EQ(rev[1], 2);
00200     EXPECT_EQ(rev[2], 1);
00201 }
00202
00203 TEST(VectorIterators, StdAlgorithms) {
00204     Vector<int> v = {5, 3, 1, 4, 2};
00205     std::sort(v.begin(), v.end());
00206     EXPECT_EQ(v[0], 1);
00207     EXPECT_EQ(v[4], 5);
00208 }
00209
00210 // 5. talpa (Capacity)
00211
00212 TEST(VectorCapacity, Empty) {
00213     Vector<int> v;
00214     EXPECT_TRUE(v.empty());
00215     v.push_back(1);
00216     EXPECT_FALSE(v.empty());
00217 }
00218
00219 TEST(VectorCapacity, Size) {
00220     Vector<int> v;
00221     EXPECT_EQ(v.size(), 0u);
00222     v.push_back(1);
00223     v.push_back(2);

```

```

00224     EXPECT_EQ(v.size(), 2u);
00225 }
00226
00227 TEST(VectorCapacity, Reserve) {
00228     Vector<int> v;
00229     v.reserve(100);
00230     EXPECT_GE(v.capacity(), 100u);
00231     EXPECT_EQ(v.size(), 0u); // dydis nepakito
00232 }
00233
00234 TEST(VectorCapacity, ReserveNoShrink) {
00235     Vector<int> v;
00236     v.reserve(100);
00237     size_t cap = v.capacity();
00238     v.reserve(10); // mažesnis - neturėtų sumažėti
00239     EXPECT_EQ(v.capacity(), cap);
00240 }
00241
00242 TEST(VectorCapacity, ShrinkToFit) {
00243     Vector<int> v;
00244     v.reserve(100);
00245     v.push_back(1);
00246     v.push_back(2);
00247     v.shrink_to_fit();
00248     EXPECT_EQ(v.capacity(), v.size());
00249 }
00250
00251 TEST(VectorCapacity, CapacityGrowth) {
00252     Vector<int> v;
00253     size_t reallocs = 0;
00254     size_t prev_cap = 0;
00255     for (int i = 0; i < 1000; ++i) {
00256         if (v.capacity() == v.size()) ++reallocs;
00257         v.push_back(i);
00258         prev_cap = v.capacity();
00259     }
00260     // Turėtų būti logaritminis perskirstymų skaičius
00261     EXPECT_LT(reallocs, 25u); // <= log2(1000) + 1
00262 }
00263
00264 // 6. modifikatoriai (Modifiers)
00265
00266 TEST(VectorModifiers, Clear) {
00267     Vector<int> v = {1, 2, 3};
00268     v.clear();
00269     EXPECT_EQ(v.size(), 0u);
00270     EXPECT_TRUE(v.empty());
00271     // Capacity išsaugoma
00272     EXPECT_GE(v.capacity(), 3u);
00273 }
00274
00275 TEST(VectorModifiers, PushBack) {
00276     Vector<int> v;
00277     for (int i = 0; i < 10; ++i) v.push_back(i);
00278     EXPECT_EQ(v.size(), 10u);
00279     EXPECT_EQ(v[9], 9);
00280 }
00281
00282 TEST(VectorModifiers, PushBackMove) {
00283     Vector<std::string> v;
00284     std::string s = "testas";
00285     v.push_back(std::move(s));
00286     EXPECT_EQ(v[0], "testas");
00287     EXPECT_TRUE(s.empty() || s == ""); // po perkėlimo
00288 }
00289
00290 TEST(VectorModifiers, PopBack) {
00291     Vector<int> v = {1, 2, 3};
00292     v.pop_back();
00293     EXPECT_EQ(v.size(), 2u);
00294     EXPECT_EQ(v.back(), 2);
00295 }
00296
00297 TEST(VectorModifiers, InsertSingle) {
00298     Vector<int> v = {1, 2, 4, 5};
00299     auto it = v.insert(v.begin() + 2, 3);
00300     EXPECT_EQ(v.size(), 5u);
00301     EXPECT_EQ(v[2], 3);
00302     EXPECT_EQ(*it, 3);
00303 }
00304
00305 TEST(VectorModifiers, InsertCount) {
00306     Vector<int> v = {1, 5};
00307     v.insert(v.begin() + 1, 3, 2); // idesime 3 x "2"
00308     EXPECT_EQ(v.size(), 5u);
00309     EXPECT_EQ(v[1], 2);
00310     EXPECT_EQ(v[2], 2);

```

```

00311     EXPECT_EQ(v[3], 2);
00312     EXPECT_EQ(v[4], 5);
00313 }
00314
00315 TEST(VectorModifiers, InsertRange) {
00316     Vector<int> v = {1, 5};
00317     std::vector<int> src = {2, 3, 4};
00318     v.insert(v.begin() + 1, src.begin(), src.end());
00319     EXPECT_EQ(v.size(), 5u);
00320     for (int i = 0; i < 5; ++i) EXPECT_EQ(v[i], i + 1);
00321 }
00322
00323 TEST(VectorModifiers, InsertInitializerList) {
00324     Vector<int> v = {1, 5};
00325     v.insert(v.begin() + 1, {2, 3, 4});
00326     EXPECT_EQ(v.size(), 5u);
00327     for (int i = 0; i < 5; ++i) EXPECT_EQ(v[i], i + 1);
00328 }
00329
00330 TEST(VectorModifiers, EraseSingle) {
00331     Vector<int> v = {1, 2, 3, 4};
00332     auto it = v.erase(v.begin() + 1);
00333     EXPECT_EQ(v.size(), 3u);
00334     EXPECT_EQ(v[1], 3);
00335     EXPECT_EQ(*it, 3);
00336 }
00337
00338 TEST(VectorModifiers, EraseRange) {
00339     Vector<int> v = {1, 2, 3, 4, 5};
00340     v.erase(v.begin() + 1, v.begin() + 4); // istrina 2,3,4
00341     EXPECT_EQ(v.size(), 2u);
00342     EXPECT_EQ(v[0], 1);
00343     EXPECT_EQ(v[1], 5);
00344 }
00345
00346 TEST(VectorModifiers, EmplaceBack) {
00347     Vector<std::pair<int, int>> v;
00348     v.emplace_back(1, 2);
00349     v.emplace_back(3, 4);
00350     EXPECT_EQ(v.size(), 2u);
00351     EXPECT_EQ(v[0].first, 1);
00352     EXPECT_EQ(v[1].second, 4);
00353 }
00354
00355 TEST(VectorModifiers, Emplace) {
00356     Vector<std::pair<int, int>> v = {{1,2}, {5,6}};
00357     v.emplace(v.begin() + 1, 3, 4);
00358     EXPECT_EQ(v.size(), 3u);
00359     EXPECT_EQ(v[1].first, 3);
00360 }
00361
00362 TEST(VectorModifiers, ResizeSmaller) {
00363     Vector<int> v = {1, 2, 3, 4, 5};
00364     v.resize(3);
00365     EXPECT_EQ(v.size(), 3u);
00366     EXPECT_EQ(v[2], 3);
00367 }
00368
00369 TEST(VectorModifiers, ResizeLarger) {
00370     Vector<int> v = {1, 2};
00371     v.resize(5, 99);
00372     EXPECT_EQ(v.size(), 5u);
00373     EXPECT_EQ(v[2], 99);
00374     EXPECT_EQ(v[4], 99);
00375 }
00376
00377 TEST(VectorModifiers, Swap) {
00378     Vector<int> a = {1, 2, 3};
00379     Vector<int> b = {4, 5};
00380     a.swap(b);
00381     EXPECT_EQ(a.size(), 2u);
00382     EXPECT_EQ(b.size(), 3u);
00383     EXPECT_EQ(a[0], 4);
00384     EXPECT_EQ(b[0], 1);
00385 }
00386
00387 // 7. palyginimo operatoriai (Non-member comparison)
00388
00389 TEST(VectorComparison, Equality) {
00390     Vector<int> a = {1, 2, 3};
00391     Vector<int> b = {1, 2, 3};
00392     Vector<int> c = {1, 2, 4};
00393     EXPECT_TRUE(a == b);
00394     EXPECT_FALSE(a == c);
00395     EXPECT_TRUE(a != c);
00396 }
00397

```

```

00398 TEST(VectorComparison, LexicographicOrder) {
00399     Vector<int> a = {1, 2, 3};
00400     Vector<int> b = {1, 2, 4};
00401     Vector<int> c = {1, 2, 3, 0};
00402     EXPECT_TRUE(a < b);
00403     EXPECT_TRUE(a < c);
00404     EXPECT_TRUE(b > a);
00405     EXPECT_TRUE(a <= a);
00406     EXPECT_TRUE(a >= a);
00407 }
00408
00409 // 8. globalios funkcijos
00410
00411 TEST(VectorNonMember, GlobalSwap) {
00412     Vector<int> a = {1, 2};
00413     Vector<int> b = {3, 4, 5};
00414     swap(a, b);
00415     EXPECT_EQ(a.size(), 3u);
00416     EXPECT_EQ(b.size(), 2u);
00417 }
00418
00419 TEST(VectorNonMember, EraseByValue) {
00420     Vector<int> v = {1, 2, 3, 2, 4, 2};
00421     auto removed = erase(v, 2);
00422     EXPECT_EQ(removed, 3u);
00423     EXPECT_EQ(v.size(), 3u);
00424     for (const auto& x : v) EXPECT_NE(x, 2);
00425 }
00426
00427 TEST(VectorNonMember, EraseIf) {
00428     Vector<int> v = {1, 2, 3, 4, 5, 6};
00429     auto removed = erase_if(v, [](int x) { return x % 2 == 0; });
00430     EXPECT_EQ(removed, 3u);
00431     EXPECT_EQ(v.size(), 3u);
00432     for (const auto& x : v) EXPECT_EQ(x % 2, 1);
00433 }
00434
00435 // 9. atminties perskyrimas
00436
00437 TEST(VectorReallocation, CountReallocations) {
00438     Vector<int> v;
00439     int reallocs = 0;
00440     size_t prev_cap = v.capacity();
00441
00442     for (int i = 1; i <= 100000; ++i) {
00443         v.push_back(i);
00444         if (v.capacity() != prev_cap) {
00445             ++reallocs;
00446             prev_cap = v.capacity();
00447         }
00448     }
00449     // std::vector paprastai daro ~17 perskirstymų 100000 elementams
00450     EXPECT_LT(reallocs, 25);
00451     std::cout << "Vector perskirstymu skaicus (100000 el.): " << reallocs << "\n";
00452 }
00453
00454 TEST(VectorReallocation, CompareWithStdVector) {
00455     // Lyginti perskirstymų skaičių su std::vector
00456     Vector<int> myV;
00457     std::vector<int> stdV;
00458     int myReallocs = 0, stdReallocs = 0;
00459     size_t myCap = 0, stdCap = 0;
00460
00461     for (int i = 1; i <= 1000000; ++i) {
00462         if (myV.capacity() == myV.size()) ++myReallocs;
00463         if (stdV.capacity() == stdV.size()) ++stdReallocs;
00464         myV.push_back(i);
00465         stdV.push_back(i);
00466     }
00467
00468     std::cout << "Vector perskirstymai (1M el.):      mano=" << myReallocs
00469               << ", std=" << stdReallocs << "\n";
00470     // Abu turėtų turėti panašų skaičių
00471     EXPECT_LT(myReallocs, 30);
00472     EXPECT_LT(stdReallocs, 30);
00473 }
00474
00475 // 10. spartos palyginimas
00476
00477 TEST(VectorPerformance, PushBack10k) {
00478     auto t0 = std::chrono::high_resolution_clock::now();
00479     Vector<int> v;
00480     for (int i = 1; i <= 10000; ++i) v.push_back(i);
00481     auto dt = std::chrono::duration<double>{
00482         std::chrono::high_resolution_clock::now() - t0}.count();
00483     EXPECT_EQ(v.size(), 10000u);
00484     EXPECT_EQ(v.back(), 10000);

```

```

00485     std::cout << "Vector push_back 10k:  " << dt << " s\n";
00486 }
00487
00488 TEST(VectorPerformance, PushBack100k) {
00489     auto t0 = std::chrono::high_resolution_clock::now();
00490     Vector<int> v;
00491     for (int i = 1; i <= 100000; ++i) v.push_back(i);
00492     auto dt = std::chrono::duration<double>{
00493         std::chrono::high_resolution_clock::now() - t0}.count();
00494     EXPECT_EQ(v.size(), 100000u);
00495     std::cout << "Vector push_back 100k:  " << dt << " s\n";
00496 }
00497
00498 TEST(VectorPerformance, PushBack1M) {
00499     auto t0 = std::chrono::high_resolution_clock::now();
00500     Vector<int> v;
00501     for (int i = 1; i <= 1000000; ++i) v.push_back(i);
00502     auto dt = std::chrono::duration<double>{
00503         std::chrono::high_resolution_clock::now() - t0}.count();
00504     EXPECT_EQ(v.size(), 1000000u);
00505     std::cout << "Vector push_back 1M:    " << dt << " s\n";
00506 }
00507
00508 TEST(VectorPerformance, StdVectorPushBack1M) {
00509     auto t0 = std::chrono::high_resolution_clock::now();
00510     std::vector<int> v;
00511     for (int i = 1; i <= 1000000; ++i) v.push_back(i);
00512     auto dt = std::chrono::duration<double>{
00513         std::chrono::high_resolution_clock::now() - t0}.count();
00514     EXPECT_EQ(v.size(), 1000000u);
00515     std::cout << "std::vector push_back 1M: " << dt << " s\n";
00516 }
00517
00518 // 11. kraštutiniai atvejai (Edge Cases)
00519
00520 TEST(VectorEdgeCases, EmptyVectorIterators) {
00521     Vector<int> v;
00522     EXPECT_EQ(v.begin(), v.end());
00523     EXPECT_EQ(v.rbegin(), v.rend());
00524 }
00525
00526 TEST(VectorEdgeCases, MoveAndPushBack) {
00527     Vector<int> a = {1, 2, 3};
00528     Vector<int> b = std::move(a);
00529     b.push_back(4);
00530     EXPECT_EQ(b.size(), 4u);
00531     EXPECT_EQ(b[3], 4);
00532 }
00533
00534 TEST(VectorEdgeCases, StringElements) {
00535     Vector<std::string> v;
00536     v.push_back("labas");
00537     v.push_back("pasauli");
00538     EXPECT_EQ(v.size(), 2u);
00539     EXPECT_EQ(v[0], "labas");
00540     v.insert(v.begin() + 1, "grazus");
00541     EXPECT_EQ(v[1], "grazus");
00542     EXPECT_EQ(v[2], "pasauli");
00543 }
00544
00545 TEST(VectorEdgeCases, NestedVector) {
00546     Vector<Vector<int>> v;
00547     v.push_back({1, 2, 3});
00548     v.push_back({4, 5});
00549     EXPECT_EQ(v.size(), 2u);
00550     EXPECT_EQ(v[0].size(), 3u);
00551     EXPECT_EQ(v[1][0], 4);
00552 }
00553
00554 TEST(VectorEdgeCases, ReserveBeforePush) {
00555     Vector<int> v;
00556     v.reserve(1000);
00557     size_t cap = v.capacity();
00558     for (int i = 0; i < 1000; ++i) v.push_back(i);
00559     // Neturėtų būti perskirstymo
00560     EXPECT_EQ(v.capacity(), cap);
00561     EXPECT_EQ(v.size(), 1000u);
00562 }

```


5.37 zmogus.cpp Failo Nuoroda

```
#include "zmogus.h"
```

5.38 zmogus.cpp

Eiti [Ä](#) Åjø failo dokumentacijÄ.

```
00001 #include "zmogus.h"
00002
00003 // --- Numatytasis konstruktorius ---
00004 Zmogus::Zmogus()
00005     : vardas_("", pavarde_("")) {
00006 }
00007
00008 // --- Parametrinis konstruktorius ---
00009 Zmogus::Zmogus(const std::string& vardas, const std::string& pavarde)
00010     : vardas_(vardas), pavarde_(pavarde) {
00011 }
00012
00013 // --- Kopijavimo konstruktorius ---
00014 Zmogus::Zmogus(const Zmogus& other)
00015     : vardas_(other.vardas_), pavarde_(other.pavarde_) {
00016 }
00017
00018 // --- Perkelimo konstruktorius ---
00019 Zmogus::Zmogus(Zmogus&& other) noexcept
00020     : vardas_(std::move(other.vardas_)),
00021       pavarde_(std::move(other.pavarde_)) {
00022 }
00023
00024 // --- Grynas virtualus destruktorius turi tureti realizacija ---
00025 // Net ir grynas virtualus destruktorius privalo buti realizuotas,
00026 // nes isvestines klases destruktorius ji isskviecia grandinine tvarka.
00027 Zmogus::~Zmogus() {}
00028
00029 // --- Kopijavimo priskyrimo operatorius ---
00030 Zmogus& Zmogus::operator=(const Zmogus& other) {
00031     if (this != &other) {
00032         vardas_ = other.vardas_;
00033         pavarde_ = other.pavarde_;
00034     }
00035     return *this;
00036 }
00037
00038 // --- Perkelimo priskyrimo operatorius ---
00039 Zmogus& Zmogus::operator=(Zmogus&& other) noexcept {
00040     if (this != &other) {
00041         vardas_ = std::move(other.vardas_);
00042         pavarde_ = std::move(other.pavarde_);
00043     }
00044     return *this;
00045 }
```

5.39 zmogus.h Failo Nuoroda

```
#include <string>
#include <iostream>
```

Klasės

- class [Zmogus](#)

Abstrakti bazine klase, atstovaujanti zmogu.

Apibrėžimai

- `#define` [ZMOGUS_H](#)

Funkcijos

- `std::ostream & operator<< (std::ostream &os, const Zmogus &z)`
Globalus isvedimo operatorius.

5.39.1 Apibrėžimų Dokumentacija

5.39.1.1 ZMOGUS_H

```
#define ZMOGUS_H
```

Apibrėžimas failo [zmogus.h](#) eilutėje 3.

5.39.2 Funkcijos Dokumentacija

5.39.2.1 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Zmogus & z) [inline]
```

Globalus isvedimo operatorius.

Parametrai

<i>os</i>	Išvesties srautas
<i>z</i>	Nuoroda i Zmogus objekta

Gražina

Nuoroda i išvesties srauta

Naudoja virtualu `print()` metoda polimorfizmui užtikrinti

Apibrėžimas failo [zmogus.h](#) eilutėje 113.

```
00113                                     {  
00114     z.print(os);  
00115     return os;  
00116 }
```

5.40 zmogus.h

Eiti [Ä](#) Åjø failo dokumentacijÄ.

```

00001 #pragma once
00002 #ifndef ZMOGUS_H
00003 #define ZMOGUS_H
00004
00005 #include <string>
00006 #include <iostream>
00007
00008 /**
00009  * @class Zmogus
00010  * @brief Abstrakti bazine klase, atstovaujanti zmogu
00011  *
00012  * Negalima sukurti Zmogus tipo objektu tiesiogiai -
00013  * klase abstrakti del grynai virtualaus destruktoriaus.
00014  * Is jos galima kurti tik isvestines klases (pvz. Studentas).
00015  *
00016  * @author Studentu Valdymo Sistema
00017  * @version 2.0
00018  */
00019 class Zmogus {
00020 protected:
00021     std::string vardas_; // Vardas
00022     std::string pavarde_; // Pavarde
00023
00024 public:
00025     // --- Konstruktoriai (Rule of Five) ---
00026
00027     /**
00028      * @brief Numatytasis konstruktorius
00029      * Sukuria tuscia zmogu su tusciais vardu ir pavarde
00030      */
00031     Zmogus();
00032
00033     /**
00034      * @brief Parametrinis konstruktorius
00035      * @param vardas Asmens vardas
00036      * @param pavarde Asmens pavarde
00037      */
00038     Zmogus(const std::string& vardas, const std::string& pavarde);
00039
00040     /**
00041      * @brief Kopijavimo konstruktorius
00042      * @param other Zmogus objektas, is kurio kopijuojama
00043      */
00044     Zmogus(const Zmogus& other);
00045
00046     /**
00047      * @brief Perkelimo konstruktorius
00048      * @param other Zmogus objektas, is kurio perkeliama
00049      */
00050
00051     Zmogus(Zmogus&& other) noexcept;
00052
00053     /**
00054      * @brief Virtualus destruktorius
00055      *
00056      * Gynas virtualus destruktorius - daro klase abstrakcia.
00057      * Isvestines klases privalo ji realizuoti.
00058      */
00059     virtual ~Zmogus() = 0;
00060
00061     // --- Priskyrimo operatoriai (Rule of Five) ---
00062
00063     /**
00064      * @brief Kopijavimo priskyrimo operatorius
00065      * @param other Zmogus objektas, is kurio kopijuojama
00066      * @return Nuoroda i si objekta
00067      */
00068     Zmogus& operator=(const Zmogus& other);
00069
00070     /**
00071      * @brief Perkelimo priskyrimo operatorius
00072      * @param other Zmogus objektas, is kurio perkeliama
00073      * @return Nuoroda i si objekta
00074      */
00075     Zmogus& operator=(Zmogus&& other) noexcept;
00076
00077     // --- Getteriai ---
00078
00079     /** @return Asmens vardas */
00080     const std::string& getVardas() const { return vardas_; }
00081
00082     /** @return Asmens pavarde */

```

```
00083     const std::string& getPavarde() const { return pavarde_; }
00084
00085     // --- Setteriai ---
00086
00087     /** @param v Naujas vardas */
00088     void setVardas(const std::string& v) { vardas_ = v; }
00089
00090     /** @param p Nauja pavarde */
00091     void setPavarde(const std::string& p) { pavarde_ = p; }
00092
00093     // --- Gynas virtualus metodus isvestines klases privalo realizuoti ---
00094
00095     /**
00096      * @brief Gynas virtualus metodus spausdinimui
00097      * @param os Ivesties srautas
00098      *
00099      * Ivestines klases privalo realizuoti si metoda.
00100      * Naudojamas operator« globaliame operatoriuje.
00101      */
00102     virtual void print(std::ostream& os) const = 0;
00103 };
00104
00105 /**
00106  * @brief Globalus isvedimo operatorius
00107  * @param os Ivesties srautas
00108  * @param z Nuoroda i Zmogus objekta
00109  * @return Nuoroda i isvesties srauta
00110  *
00111  * @details Naudoja virtualu print() metoda polimorfizmui uztikrinti
00112  */
00113 inline std::ostream& operator«(std::ostream& os, const Zmogus& z) {
00114     z.print(os);
00115     return os;
00116 }
00117
00118 #endif // ZMOGUS_H
```

skyrius 6

Pavyzdžiai

6.1 C:/Users/GYTIS/Documents/GitHub/Objektinis-prog-3/0.1_versija/0.1_versija/studentas.h

Parametrinis konstruktorius.

Parametrinis konstruktorius

Parametrai

<i>vardas</i>	Studento vardas
<i>pavarde</i>	Studento pavarde
<i>paz</i>	Namu darbu pazymiu vektorius
<i>egz</i>	Egzamino pazymys

```
std::vector<int> pazymiai = {8, 9, 7}; Studentas s("Jonas", "Jonaitis", pazymiai, 10);
```

```
#ifndef STUDENTAS_H
#define STUDENTAS_H

#include "zmogus.h"
#include "vector.h"
#include <vector>
#include <iostream>
#include <iomanip>
#include <sstream>

/**
 * @class Studentas
 * @brief Atstovauja studenta su pazymiais ir egzamino rezultatu
 *
 * Si klase praplecia Zmogus klase ir igyvendina studentams budinga funkcionaluma,
 * iskaitant pazymiu valdyma, rezultatu skaiciavima (vidurkis/mediana) ir
 * klasifikavima i grupes.
 *
 * @author Studentu Valdymo Sistema
 * @version 2.0
 * @date 2026
 */

class Studentas : public Zmogus {
private:
    std::vector<int> paz_; // Pazymiai
    int egz_; // Egzamino rezultatas
    double rez_; // Galutinis rezultatas (vidurkis arba mediana)

public:
    // Konstruktoriai
    /**
```

```

    * @brief Numatytasis konstruktorius
    *
    * Sukuria tuscia studenta su numatytosiomis reiksmemis.
    * Visi pazymiai isvalomi, egzaminas = 0, rezultatas = 0.0
    */

Studentas();

/**
 * @brief Parametrinis konstruktorius
 * @param vardas Studento vardas
 * @param pavarde Studento pavarde
 * @param paz Namu darbu pazymiu vektorius
 * @param egz Egzamino pazymys
 *
 * @example
 * std::vector<int> pazymiai = {8, 9, 7};
 * Studentas s("Jonas", "Jonaitis", pazymiai, 10);
 */

Studentas(const std::string& vardas,
          const std::string& pavarde,
          const Vector<int>& paz,
          int egz);

/**
 * @brief Kopijavimo konstruktorius (gilioji kopija)
 * @param other Studentas objektas, is kurio kopijuojama
 * @kompleksiskumas O(n), kur n yra pazymiu skaicius
 */

// Kopijavimo konstruktorius
Studentas(const Studentas& other);

/**
 * @brief Perkelimo konstruktorius
 * @param other Studentas objektas, is kurio perkeliama
 * @kompleksiskumas O(1)
 */

Studentas(Studentas&& other) noexcept;

/**
 * @brief Virtualus destruktoriaus
 *
 * Uztikrina tinkama isvestiniu klasiu isvalyma
 */
~Studentas();

// Priskyrimo operatoriai

/**
 * @brief Kopijavimo priskyrimo operatorius
 * @param other Studentas objektas, is kurio kopijuojama
 * @return Nuoroda i si objekta
 */

Studentas& operator=(const Studentas& other);

/**
 * @brief Perkelimo priskyrimo operatorius
 * @param other Studentas objektas, is kurio perkeliama
 * @return Nuoroda i si objekta
 */
Studentas& operator=(Studentas&& other) noexcept;

// Palyginimo operatoriai

/**
 * @brief Maziau uz (pagal galutini rezultata)
 * @param other Studentas su kuriuo lyginama
 * @return true jei si.rez_ < other.rez_
 */
bool operator<(const Studentas& other) const;

/**
 * @brief Daugiau uz (pagal galutini rezultata)
 * @param other Studentas su kuriuo lyginama
 * @return true jei si.rez_ > other.rez_
 */
bool operator>(const Studentas& other) const;

/**
 * @brief Lygybe (pagal varda ir pavarde)
 * @param other Studentas su kuriuo lyginama

```

```

* @return true jei vardai ir pavardes sutampa
*/
bool operator==(const Studentas& other) const;

/**
 * @brief Maziau arba lygu
 * @return true jei si.rez_ <= other.rez_
 */
bool operator<=(const Studentas& other) const;

/**
 * @brief Daugiau arba lygu
 * @return true jei si.rez_ >= other.rez_
 */
bool operator>=(const Studentas& other) const;

/**
 * @brief Nelygybe
 * @param other Studentas su kuriuo lyginama
 * @return true jei vardai ar pavardes skiriasi
 */
bool operator!=(const Studentas& other) const;

// Pridėti pazymi: s += 8
Studentas& operator+=(int pazymys);

// Prieiga prie paz_[i]
int operator[](size_t i) const;
int& operator[](size_t i);

// SRAUTU OPERATORIAI

/**
 * @brief Išvedimo į ekrano operatorius
 * @param os Išvesties srautas
 * @param s Studentas objektas
 * @return Nuoroda į išvesties srautą
 * @detales Formatos: "Vardas Pavarde [pazymiai] Egz: E Rez: R"
 */
friend std::ostream& operator<<(std::ostream& os, const Studentas& s);

/**
 * @brief Įvedimo iš failo operatorius
 * @param is Išvesties srautas
 * @param s Studentas objektas
 * @return Nuoroda į išvesties srautą
 * @detales Skaito: "Vardas Pavarde paz1 paz2 ... pazN egzaminas"
 */
friend std::istream& operator>>(std::istream& is, Studentas& s);

//
// Virtualus metodus realizuoja bazinės klasės gryną virtualų metodą
//

/**
 * @brief Spausdina studento informaciją (pakeičia Zmogus::print)
 * @param os Išvesties srautas
 */
void print(std::ostream& os) const override;

//
// GETTERIAI
//

/** @return Pastovi nuoroda į pažymių vektorių */
const std::vector<int>& getPaz() const { return paz_; }

/** @return Egzamino pažymys */
int getEgz() const { return egz_; }

/** @return Galutinis rezultatas */
double getRez() const { return rez_; }

/** @return Namų darbų pažymių skaičius */
size_t getPazSkaicius() const { return paz_.size(); }

//
// SETTERIAI
//

/** @param e Naujas egzamino pažymys (0-10) */
void setEgz(int e) { egz_ = e; }

/** @param r Naujas galutinis rezultatas */

```

```
void setRez(double r) { rez_ = r; }

//
// METODAI
//

/**
 * @brief Prideda viena namu darbu pazymi
 * @param p Pazymys (1-10)
 */
void addPazymys(int p);

/** @brief Išvalo visus namu darbu pazymius */
void clearPazymiai();

/**
 * @brief Skaiciuoja namu darbu pazymiu vidurki
 * @return Aritmetinis vidurkis kaip double
 */
double vidurkis() const;

/**
 * @brief Skaiciuoja namu darbu pazymiu mediana
 * @return Mediana kaip double
 */
double mediana() const;

/**
 * @brief Skaiciuoja galutini rezultata
 * @param tipas 1=vidurkis, 2=mediana
 * @details Formule: 0.4 * pazymys + 0.6 * egzaminas
 */
void skaiciuotiRez(int tipas); // 1=vidurkis, 2=mediana

/**
 * @brief Tikrina ar studentas islaike
 * @return true jei rez_ >= 5.0
 */
bool islaike() const { return rez_ >= 5.0; }
};

#endif
```


Rodyklė

- ~Studentas
 - Studentas, [11](#)
- ~Vector
 - Vector< T, Allocator >, [33](#)
- ~Zmogus
 - Zmogus, [51](#)
- addPazymys
 - Studentas, [11](#)
- alloc_
 - Vector< T, Allocator >, [47](#)
- allocator_type
 - Vector< T, Allocator >, [29](#)
- AllocTraits
 - Vector< T, Allocator >, [29](#)
- assign
 - Vector< T, Allocator >, [34](#)
- at
 - Vector< T, Allocator >, [34](#), [35](#)
- atlikti_klases_testus
 - main.cpp, [73](#)
 - testavimas_klases.cpp, [94](#)
- atlikti_visus_testus
 - main.cpp, [74](#)
 - testavimas.cpp, [89](#)
- back
 - Vector< T, Allocator >, [35](#)
- begin
 - Vector< T, Allocator >, [35](#), [36](#)
- bendras
 - TestRow, [23](#)
- capacity
 - Vector< T, Allocator >, [36](#)
- capacity_
 - Vector< T, Allocator >, [47](#)
- cbegin
 - Vector< T, Allocator >, [36](#)
- cend
 - Vector< T, Allocator >, [36](#)
- clear
 - Vector< T, Allocator >, [36](#)
- clearPazymiai
 - Studentas, [11](#)
- const_iterator
 - Vector< T, Allocator >, [29](#)
- const_pointer
 - Vector< T, Allocator >, [29](#)
- const_reference
 - Vector< T, Allocator >, [29](#)
- const_reverse_iterator
 - Vector< T, Allocator >, [29](#)
- crbegin
 - Vector< T, Allocator >, [37](#)
- crend
 - Vector< T, Allocator >, [37](#)
- data
 - Vector< T, Allocator >, [37](#)
- data_
 - Vector< T, Allocator >, [48](#)
- deallocate_all
 - Vector< T, Allocator >, [37](#)
- deque_ops.cpp, [55](#)
 - isvedimas_i_faila_d, [55](#)
 - pasirinkimo_metodas_d, [56](#)
 - rusiavimas_d, [56](#)
 - skaitymas_is_failo_d, [56](#)
 - skirstymas_i_grupes_d, [56](#)
 - skirstymas_s1_d, [57](#)
 - skirstymas_s2_d, [57](#)
 - skirstymas_s3_d, [57](#)
- deque_ops.h, [59](#)
 - isvedimas_i_faila_d, [60](#)
 - pasirinkimo_metodas_d, [60](#)
 - rusiavimas_d, [60](#)
 - skaitymas_is_failo_d, [60](#)
 - skirstymas_i_grupes_d, [61](#)
 - skirstymas_s1_d, [61](#)
 - skirstymas_s2_d, [61](#)
 - skirstymas_s3_d, [62](#)
- difference_type
 - Vector< T, Allocator >, [30](#)
- egz_
 - Studentas, [22](#)
- emplace
 - Vector< T, Allocator >, [38](#)
- emplace_back
 - Vector< T, Allocator >, [38](#)
- empty
 - Vector< T, Allocator >, [39](#)
- end
 - Vector< T, Allocator >, [39](#)
- erase
 - Vector< T, Allocator >, [39](#)
 - vector.h, [116](#)
- erase_if
 - vector.h, [116](#)

- failas
 - TestRow, 23
- front
 - Vector< T, Allocator >, 40
- galutinis_vid
 - utils.cpp, 110
- generator.cpp, 63
 - generuoti_faila, 63
 - generuoti_pazymi, 63
 - generuoti_testu_failus, 64
- generuoti_faila
 - generator.cpp, 63
 - main.cpp, 74
- generuoti_pazymi
 - generator.cpp, 63
- generuoti_testu_failus
 - generator.cpp, 64
 - main.cpp, 75
- get_allocator
 - Vector< T, Allocator >, 40
- getEgz
 - Studentas, 12
- getPavarde
 - Zmogus, 51
- getPaz
 - Studentas, 12
- getPazSkaicius
 - Studentas, 12
- getRez
 - Studentas, 13
- getVardas
 - Zmogus, 51
- insert
 - Vector< T, Allocator >, 40–42
- islaike
 - Studentas, 13
- isvedimas
 - vector_ops.cpp, 127
 - vector_ops.h, 134
- isvedimas_i_faila
 - vector_ops.cpp, 127
 - vector_ops.h, 134
- isvedimas_i_faila_d
 - deque_ops.cpp, 55
 - deque_ops.h, 60
- isvedimas_i_faila_l
 - list_ops.cpp, 65
 - list_ops.h, 70
- iterator
 - Vector< T, Allocator >, 30
- ivesti_varda_ar_pavarde
 - utils.cpp, 110
 - utils.h, 113
- ivesties_skaicius
 - utils.cpp, 110
 - utils.h, 113
- ivesties_tikrinimas
 - utils.cpp, 110
 - utils.h, 113
- konteineris
 - TestRow, 23
- list_ops.cpp, 65
 - isvedimas_i_faila_l, 65
 - pasirinkimo_metodas_l, 66
 - rusiavimas_l, 66
 - skaitymas_is_failo_l, 66
 - skirstymas_i_grupes_l, 66
 - skirstymas_s1_l, 67
 - skirstymas_s2_l, 67
 - skirstymas_s3_l, 67
- list_ops.h, 69
 - isvedimas_i_faila_l, 70
 - pasirinkimo_metodas_l, 70
 - rusiavimas_l, 70
 - skaitymas_is_failo_l, 70
 - skirstymas_i_grupes_l, 71
 - skirstymas_s1_l, 71
 - skirstymas_s2_l, 71
 - skirstymas_s3_l, 72
- main
 - main.cpp, 75
 - unit_tests.cpp, 104
- main.cpp, 73
 - atlikti_klases_testus, 73
 - atlikti_visus_testus, 74
 - generuoti_faila, 74
 - generuoti_testu_failus, 75
 - main, 75
 - rankinis_ivedimas, 76
 - skaityti_deque, 76
 - skaityti_list, 77
 - skaityti_vector, 77
- max_size
 - Vector< T, Allocator >, 42
- mediana
 - Studentas, 13
- operator!=
 - Studentas, 14
 - vector.h, 116
- operator<
 - Studentas, 14
 - vector.h, 117
- operator<<
 - Studentas, 20
 - studentas.cpp, 81
 - zmogus.h, 166
- operator<=
 - Studentas, 15
 - vector.h, 117
- operator>
 - Studentas, 17
 - vector.h, 117

operator>>
 Studentas, 20
 studentas.cpp, 81

operator>=
 Studentas, 17
 vector.h, 118

operator+=
 Studentas, 14

operator=
 Studentas, 15, 16
 Vector< T, Allocator >, 42, 43
 Zmogus, 51, 52

operator==
 Studentas, 16
 vector.h, 117

operator[]
 Studentas, 18
 Vector< T, Allocator >, 43

pasirinkimas_rusiavimo_budo
 vector_ops.cpp, 128
 vector_ops.h, 136

pasirinkimo_metodas
 vector_ops.cpp, 128
 vector_ops.h, 136

pasirinkimo_metodas_d
 deque_ops.cpp, 56
 deque_ops.h, 60

pasirinkimo_metodas_l
 list_ops.cpp, 66
 list_ops.h, 70

pavarde_
 Zmogus, 53

paz_
 Studentas, 22

pointer
 Vector< T, Allocator >, 30

pop_back
 Vector< T, Allocator >, 44

print
 Studentas, 18
 Zmogus, 52

push_back
 Vector< T, Allocator >, 44

rankinis_ivedimas
 main.cpp, 76

rbegin
 Vector< T, Allocator >, 44, 45

reallocate
 Vector< T, Allocator >, 45

reference
 Vector< T, Allocator >, 30

rend
 Vector< T, Allocator >, 45

reserve
 Vector< T, Allocator >, 46

resize
 Vector< T, Allocator >, 46

reverse_iterator
 Vector< T, Allocator >, 30

rez_
 Studentas, 22

rusiavimas
 TestRow, 24
 vector_ops.cpp, 128
 vector_ops.h, 137

rusiavimas_d
 deque_ops.cpp, 56
 deque_ops.h, 60

rusiavimas_l
 list_ops.cpp, 66
 list_ops.h, 70

sekcija
 testavimas_klases.cpp, 94

setEgz
 Studentas, 18

setPavarde
 Zmogus, 52

setRez
 Studentas, 19

setVardas
 Zmogus, 53

shrink_to_fit
 Vector< T, Allocator >, 46

size
 Vector< T, Allocator >, 47

size_
 Vector< T, Allocator >, 48

size_type
 Vector< T, Allocator >, 30

skaiciavimo_metodas
 utils.cpp, 111
 utils.h, 114

skaiciuotiRez
 Studentas, 19

skaitymas
 TestRow, 24

skaitymas_is_failo
 vector_ops.cpp, 129
 vector_ops.h, 137

skaitymas_is_failo_d
 deque_ops.cpp, 56
 deque_ops.h, 60

skaitymas_is_failo_l
 list_ops.cpp, 66
 list_ops.h, 70

skaityti_deque
 main.cpp, 76

skaityti_list
 main.cpp, 77

skaityti_vector
 main.cpp, 77

skirstymas
 TestRow, 24

skirstymas_i_grupes
 vector_ops.cpp, 129

- vector_ops.h, 138
- skirstymas_i_grupes_d
 - deque_ops.cpp, 56
 - deque_ops.h, 61
- skirstymas_i_grupes_l
 - list_ops.cpp, 66
 - list_ops.h, 71
- skirstymas_s1
 - vector_ops.cpp, 130
 - vector_ops.h, 138
- skirstymas_s1_d
 - deque_ops.cpp, 57
 - deque_ops.h, 61
- skirstymas_s1_l
 - list_ops.cpp, 67
 - list_ops.h, 71
- skirstymas_s2
 - vector_ops.cpp, 130
 - vector_ops.h, 138
- skirstymas_s2_d
 - deque_ops.cpp, 57
 - deque_ops.h, 61
- skirstymas_s2_l
 - list_ops.cpp, 67
 - list_ops.h, 71
- skirstymas_s3
 - vector_ops.cpp, 131
 - vector_ops.h, 139
- skirstymas_s3_d
 - deque_ops.cpp, 57
 - deque_ops.h, 62
- skirstymas_s3_l
 - list_ops.cpp, 67
 - list_ops.h, 72
- spausdinti_lentele
 - testavimas.cpp, 89
- strategija
 - TestRow, 24
- Studentas, 7
 - ~Studentas, 11
 - addPazymys, 11
 - clearPazymiai, 11
 - egz_, 22
 - getEgz, 12
 - getPaz, 12
 - getPazSkaicius, 12
 - getRez, 13
 - islaike, 13
 - mediana, 13
 - operator!=, 14
 - operator<, 14
 - operator<<, 20
 - operator<=, 15
 - operator>, 17
 - operator>>, 20
 - operator>=, 17
 - operator+=, 14
 - operator=, 15, 16
 - operator==, 16
 - operator[], 18
 - paz_, 22
 - print, 18
 - rez_, 22
 - setEgz, 18
 - setRez, 19
 - skaiciuotiRez, 19
 - Studentas, 10
 - vidurkis, 19
- studentas.cpp, 81
 - operator<<, 81
 - operator>>, 81
- studentas.h, 85
- studentu_sk
 - TestRow, 24
- swap
 - Vector< T, Allocator >, 47
 - vector.h, 118
- TEST
 - unit_tests.cpp, 104–107
 - vector_unit_tests.cpp, 142–157
- testai_is_viso
 - testavimas_klases.cpp, 99
- testai_pavyke
 - testavimas_klases.cpp, 99
- testavimas.cpp, 88
 - atlikti_visus_testus, 89
 - spausdinti_lentele, 89
 - testuoti_deque, 90
 - testuoti_list, 90
 - testuoti_vector, 91
- testavimas_klases.cpp, 93
 - atlikti_klases_testus, 94
 - sekcija, 94
 - testai_is_viso, 99
 - testai_pavyke, 99
 - testuoti_konstruktorius, 95
 - testuoti_metodus, 95
 - testuoti_palyginimo_operatorius, 96
 - testuoti_priskyrima, 97
 - testuoti_srautu_operatorius, 97
 - testuoti_su_stl, 98
 - testuoti_zmogus_abstrakti, 98
 - tikrinti, 99
- TestRow, 23
 - bendras, 23
 - failas, 23
 - konteineris, 23
 - rusiavimas, 24
 - skaitymas, 24
 - skirstymas, 24
 - strategija, 24
 - studentu_sk, 24
- testuoti_deque
 - testavimas.cpp, 90
- testuoti_konstruktorius
 - testavimas_klases.cpp, 95

- testuoti_list
 - testavimas.cpp, 90
- testuoti_metodus
 - testavimas_klases.cpp, 95
- testuoti_palyginimo_operatorius
 - testavimas_klases.cpp, 96
- testuoti_priskyrima
 - testavimas_klases.cpp, 97
- testuoti_srautu_operatorius
 - testavimas_klases.cpp, 97
- testuoti_su_stl
 - testavimas_klases.cpp, 98
- testuoti_vector
 - testavimas.cpp, 91
- testuoti_zmogus_abstrakti
 - testavimas_klases.cpp, 98
- tikrinti
 - testavimas_klases.cpp, 99
- unit_tests.cpp, 104
 - main, 104
 - TEST, 104–107
- utils.cpp, 109
 - galutinis_vid, 110
 - investi_varda_ar_pavarde, 110
 - investies_skaicius, 110
 - investies_tikrinimas, 110
 - skaiciavimo_metodas, 111
- utils.h, 113
 - investi_varda_ar_pavarde, 113
 - investies_skaicius, 113
 - investies_tikrinimas, 113
 - skaiciavimo_metodas, 114
- value_type
 - Vector< T, Allocator >, 30
- vardas_
 - Zmogus, 53
- Vector
 - Vector< T, Allocator >, 31–33
- Vector< T, Allocator >, 24
 - ~Vector, 33
 - alloc_, 47
 - allocator_type, 29
 - AllocTraits, 29
 - assign, 34
 - at, 34, 35
 - back, 35
 - begin, 35, 36
 - capacity, 36
 - capacity_, 47
 - cbegin, 36
 - cend, 36
 - clear, 36
 - const_iterator, 29
 - const_pointer, 29
 - const_reference, 29
 - const_reverse_iterator, 29
 - crbegin, 37
 - crend, 37
 - data, 37
 - data_, 48
 - deallocate_all, 37
 - difference_type, 30
 - emplace, 38
 - emplace_back, 38
 - empty, 39
 - end, 39
 - erase, 39
 - front, 40
 - get_allocator, 40
 - insert, 40–42
 - iterator, 30
 - max_size, 42
 - operator=, 42, 43
 - operator[], 43
 - pointer, 30
 - pop_back, 44
 - push_back, 44
 - rbegin, 44, 45
 - reallocate, 45
 - reference, 30
 - rend, 45
 - reserve, 46
 - resize, 46
 - reverse_iterator, 30
 - shrink_to_fit, 46
 - size, 47
 - size_, 48
 - size_type, 30
 - swap, 47
 - value_type, 30
 - Vector, 31–33
- vector.cpp, 115
- vector.h, 115
 - erase, 116
 - erase_if, 116
 - operator!=, 116
 - operator<, 117
 - operator<=, 117
 - operator>, 117
 - operator>=, 118
 - operator==, 117
 - swap, 118
- vector_ops.cpp, 126
 - isvedimas, 127
 - isvedimas_i_faila, 127
 - pasirinkimas_rusiavimo_budo, 128
 - pasirinkimo_metodas, 128
 - rusiavimas, 128
 - skaitymas_is_failo, 129
 - skirstymas_i_grupes, 129
 - skirstymas_s1, 130
 - skirstymas_s2, 130
 - skirstymas_s3, 131
- vector_ops.h, 133
 - isvedimas, 134

- isvedimas_i_faila, [134](#)
- pasirinkimas_rusiavimo_budo, [136](#)
- pasirinkimo_metodas, [136](#)
- rusiavimas, [137](#)
- skaitymas_is_failo, [137](#)
- skirstymas_i_grupes, [138](#)
- skirstymas_s1, [138](#)
- skirstymas_s2, [138](#)
- skirstymas_s3, [139](#)
- vector_unit_tests.cpp, [141](#)
- TEST, [142–157](#)
- vidurkis
 - Studentas, [19](#)
- Zmogus, [48](#)
 - ~Zmogus, [51](#)
 - getPavarde, [51](#)
 - getVardas, [51](#)
 - operator=, [51](#), [52](#)
 - pavarde_, [53](#)
 - print, [52](#)
 - setPavarde, [52](#)
 - setVardas, [53](#)
 - vardas_, [53](#)
 - Zmogus, [50](#)
- zmogus.cpp, [165](#)
- zmogus.h, [165](#)
 - operator<<, [166](#)
 - ZMOGUS_H, [166](#)
- ZMOGUS_H
 - zmogus.h, [166](#)